Algoritmos e estrutura de dados

INICIAIS_ALGORITMOS E EST DE DADOS.indd 1 1/14/09 6:38:34 PM

INICIAIS_ALGORITMOS E EST DE DADOS.indd 2 1/14/09 6:38:34 PM

Algoritmos e estrutura de dados

Paulo Roberto Martins
(Org.)

UNOPAR VIRTUAL

PEARSON Education

CÓPIA NÃO AUTORIZADA É CRIME
ABDR
ASSOCIAÇÃO BRASILEIRA DE DIREITOS REPROGRÁFICOS
RESPEITE O DIREITO AUTORAL
EDITORA AFILIADA

© 2009 by Pearson Education do Brasil
Todos os direitos reservados. Nenhuma parte desta publicação
poderá ser reproduzida ou transmitida de qualquer modo
ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia,
gravação ou qualquer outro tipo de sistema de armazenamento e transmissão
de informação, sem prévia autorização, por escrito, da Pearson Education do Brasil.

Diretor editorial: Roger Trimer
Gerente editorial: Sabrina Cairo
Supervisora de produção editorial: Marcelo Françozo
Editora: Josie Rogero
Revisão: Juliana Campoi
Capa: Rafael Mazzo
Diagramação: Globaltec Artes Gráficas Ltda.

Dados Internacionais de catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Martins, Paulo Roberto (org.)
Algoritmos e estrutura de dados : análise e desenvolvimento de sistemas / Paulo Roberto Martins. -- São Paulo : Pearson Prentice Hall, 2009.

ISBN 978-85-7605-244-9

1. Algoritmos 2. Estrutura de dados (Ciência da computação) - Estudo e ensino
3. Programação (Computadores eletrônicos) I. Título.

09-00070 CDD-005.107

Índice para catálogo sistemático:
1. Computadores : Programas : Processamento de dados : Estudo e ensino 005.107
2. Programação de computadores : Processamento de dados : Estudo e ensino 005.107

2009
Direitos exclusivos para língua portuguesa cedidos à
Pearson Education do Brasil, uma empresa do grupo Pearson Education
Av. Ermano Marchetti, 1435
CEP: 05038-001 - São Paulo - SP
Tel.: (11) 2178-8686, Fax: (11) 2178-8688
e-mail: vendas@pearsoned.com

Sumário

Unidade 1 – Algoritmos de ordenação em vetores 1

Algoritmo de ordenação por troca 1

Algoritmo de ordenação por inserção 16

Algoritmo de ordenação por seleção 26

Algoritmo de ordenação por intercalação 34

Unidade 2 – Algoritmos de busca em vetores 46

Algoritmo de busca seqüencial 46

Algoritmo de busca binária 59

Unidade 3 – Estruturas de dados dos tipos pilha e fila 67

Pilha 67

Implementação estática de uma pilha 68

Implementação dinâmica de uma pilha 71

Aplicação de controle de estoque usando pilha dinâmica 75

Fila 86

Implementação estática de uma fila 87

Implementação dinâmica de uma fila 90

Aplicação de controle de senhas usando fila dinâmica 94

Unidade 4 – Listas estática e dinâmica simplesmente encadeadas 101

Implementacão de uma lista estática simplesmente encadeada e não ordenada 104

Implementação de uma lista estática simplesmente encadeada e ordenada 108

Implementação de uma lista dinâmica simplesmente encadeada e não ordenada 112

Implementação de uma lista dinâmica simplesmente encadeada e ordenada 119

Aplicacão de controle de alunos usando lista dinâmica simplesmente encadeada e não ordenada 124

Unidade 5 – Lista dinâmica duplamente encadeada 136

Implementação de uma lista dinâmica duplamente encadeada e não ordenada 140

Implementação de uma lista dinâmica duplamente encadeada e ordenada 147

Aplicação de controle de vôos usando lista dinâmica duplamente encadeada e não ordenada 153

Unidade 6 – Árvore binária 155

Implementação de uma árvore binária dinâmica 158

Aplicação de fluxo de caixa usando árvore binária 169

Carta ao aluno

O crescimento e a convergência do potencial das tecnologias da informação e da comunicação fazem com que a Educação a Distância, sem dúvida, contribua para a expansão do ensino superior no Brasil, além de favorecer a transformação dos métodos tradicionais de ensino em uma nova e inovadora proposta pedagógica.

Foram exatamente essas características que possibilitaram à Unopar ser o que é hoje: uma referência nacional em ensino superior. Além de oferecer cursos nas áreas de humanas, exatas e da saúde em três *campi* localizados no Paraná, é uma das maiores universidades de educação a distância do país, com mais de 350 pólos e um sistema de ensino diferenciado que engloba aulas ao vivo via satélite, Internet, ambiente Web e, agora, livros-texto como este.

Elaborados com base na idéia de que — embora a educação a distância tenha entre seus pilares o autodesenvolvimento — os alunos precisam de instrumentos didáticos que os apóiem, os livros-texto da Unopar têm como objetivo permitir que os estudantes ampliem seu conhecimento teórico ao mesmo tempo em que aprendem a partir de suas experiências, desenvolvendo a capacidade de analisar o mundo a seu redor.

Para tanto, além de possuírem um alto grau de dialogicidade — caracterizado por um texto claro e apoiado por elementos como "Saiba mais", "Links" e "Para saber mais" —, esses livros contam com a seção "Aprofundando o conhecimento", que proporciona acesso a materiais de jornais e revistas, artigos e textos extraídos de livros de autores consagrados, como Philip Kotler, Lawrence Gitman e Stephen Robbins.

E, como não deve haver limites para o aprendizado, os alunos que quiserem ampliar seus estudos poderão encontrar na íntegra na Biblioteca Digital, acessando a Biblioteca Virtual Universitária disponibilizada pela instituição, a grande maioria dos livros indicada na seção "Aprofundando o conhecimento". Essa biblioteca, que funciona 24 horas por dia durante os 7 dias da semana, conta com mais de 700 títulos em português das mais diversas áreas do conhecimento e pode ser acessada de qualquer computador conectado à Internet.

Somados à experiência dos professores e coordenadores pedagógicos da Unopar, esses recursos são uma parte do esforço da instituição para realmente fazer diferença na vida e na carreira de seus estudantes e também — por que não — para contribuir com o futuro de nosso país.

Bom estudo!
Pró-reitoria

INICIAIS_ALGORITMOS E EST DE DADOS.indd 7 1/14/09 6:38:35 PM

INICIAIS_ALGORITMOS E EST DE DADOS.indd 8 1/14/09 6:38:35 PM

Unidade 1

Algoritmos de ordenação em vetores*

Existem situações em que se tem um vetor de dados desordenados e se deseja ordená-los; para realizar o procedimento de ordenação, alguns algoritmos podem ser usados. Veja a seguir alguns algoritmos de ordenação.

Algoritmo de ordenação por troca

Neste algoritmo de ordenação serão efetuadas comparações entre os dados armazenados no vetor. Quando a situação procurada (crescente ou decrescente) é encontrada, uma troca de posições entre os dados é realizada. Assim, um laço com as comparações e as trocas será efetuado na quantidade de vezes igual ao número de dados do vetor menos 1, ou seja, na maior quantidade possível de trocas. Abaixo, o algoritmo de ordenação por troca será ilustrado com um vetor que possui dez posições, logo o laço será executado nove vezes e tendo como objetivo ordenar os dados do vetor de forma crescente.

Execução número 1 do laço:

Vet	5	4	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 1 (número 5) é maior que o número da posição 2 (número 4). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

* Esta unidade corresponde ao Capítulo 2 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005.

Vet	4	5	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 2 (número 5) é maior que o número da posição 3 (número 2). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	5	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 3 (número 5) é maior que o número da posição 4 (número 1). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 4 (número 5) é maior que o número da posição 5 (número 8). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 5 (número 8) é maior que o número da posição 6 (número 2). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	2	8	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 6 (número 8) é maior que o número da posição 7 (número 4). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	2	4	8	5	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 7 (número 8) é maior que o número da posição 8 (número 5). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	2	4	5	8	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 8 (número 8) é maior que o número da posição 9 (número 9). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	2	4	5	8	9	7
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 9 (número 9) é maior que o número da posição 10 (número 7). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	4	2	1	5	2	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10



Como não há mais a próxima posição para se comparar, a execução seguinte do laço é realizada.

Execução número 2 do laço:

Vet	4	2	1	5	2	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 1 (número 4) é maior que o número da posição 2 (número 2). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	2	4	1	5	2	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 2 (número 4) é maior que o número da posição 3 (número 1). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	2	1	4	5	2	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 3 (número 4) é maior que o número da posição 4 (número 5). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	2	1	4	5	2	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 4 (número 5) é maior que o número da posição 5 (número 2). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	5	4	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 5 (número 5) é maior que o número da posição 6 (número 4). Se for, então ocorre a troca e avança-se. Do contrário,

avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	4	5	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 6 (número 5) é maior que o número da posição 7 (número 5). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	4	5	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 7 (número 5) é maior que o número da posição 8 (número 8). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	4	5	5	8	7	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 8 (número 8) é maior que o número da posição 9 (número 7). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10

Compara-se se o número da posição 9 (número 8) é maior que o número da posição 10 (número 9). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	2	1	4	2	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10

Como não há mais a próxima posição para se comparar, a execução seguinte do laço é realizada.

Execução número 3 do laço:

Vet	2	1	4	2	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 1 (número 2) é maior que o número da posição 2 (número 1). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	1	2	4	2	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 2 (número 2) é maior que o número da posição 3 (número 4). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	4	2	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 3 (número 4) é maior que o número da posição 4 (número 2). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreram a troca e o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 4 (número 4) é maior que o número da posição 5 (número 4). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 5 (número 4) é maior que o número da posição 6 (número 5). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 6 (número 5) é maior que o número da posição 7 (número 5). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 7 (número 5) é maior que o número da posição 8 (número 7). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 8 (número 7) é maior que o número da posição 9 (número 8). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10



Compara-se se o número da posição 9 (número 8) é maior que o número da posição 10 (número 9). Se for, então ocorre a troca e avança-se. Do contrário, avança-se com a pesquisa. Neste caso ocorreu apenas o avanço, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10

Como não há mais a próxima posição para se comparar, a execução seguinte do laço é realizada. Assim, as execuções do laço continuarão até chegar ao número de elementos do vetor menos um, neste exemplo, 9. Pode-se perceber que, mesmo o vetor já estando ordenado, as comparações continuarão. Logo, propõe-se uma melhoria no método de ordenação por troca, ou seja, em vez de fazer um laço que repetirá uma quantidade de vezes igual ao tamanho do vetor menos 1, pode-se repetir o laço enquanto houver alguma troca.

A Figura 1.1 ilustra a aplicação que ordena um vetor de forma crescente utilizando o algoritmo de ordenação por troca; após essa figura, os seus respectivos códigos comentados aparecem.

Figura 1.1 Aplicação que ordena os dados de um vetor pelo algoritmo de ordenação por troca



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador VET, com 10 posições
     // para armazenar números inteiros
     vet:array[1..10] of integer;
     // variável com identificador POSLIVRE que mostra
     // se o vetor já foi carregado ou não
     poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Ordenação por troca','');
if num <> ''
then begin
      numero:=strtoint(num);
      if quantidade = 1
      then begin
           pos1.visible:=true;
           pos1.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 2
      then begin
           pos2.visible:=true;
           pos2.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 3
      then begin
           pos3.visible:=true;
           pos3.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 4
      then begin
           pos4.visible:=true;
           pos4.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 5
      then begin
           pos5.visible:=true;
           pos5.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 6
      then begin
           pos6.visible:=true;
           pos6.caption:=num;
           vet[quantidade]:=numero;
           end;
```

```
   if quantidade = 7
   then begin
        pos7.visible:=true;
        pos7.caption:=num;
        vet[quantidade]:=numero;
        end;
   if quantidade = 8
   then begin
        pos8.visible:=true;
        pos8.caption:=num;
        vet[quantidade]:=numero;
        end;
   if quantidade = 9
   then begin
          pos9.visible:=true;
          pos9.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 10
     then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
     quantidade:=quantidade+1;
    end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Ordenar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, j, aux: integer;
begin
// este procedimento vai executar o algoritmo de ordenação por troca;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     // o laço abaixo deve repetir uma quantidade de vezes
     // igual ao tamanho do vetor menos 1
     // neste caso, tem-se um vetor com 10 posições, logo, 9 repetições
    for i:=1 to 9 do
    begin
       // o laço abaixo vai comparar uma posição com a próxima.
       // Como a comparação é sempre com a próxima, o laço vai até 9,
       // pois, se fosse até 10, teríamos uma comparação
       // entre a posição 10 e 11 que não existe
       for j:=1 to 9 do
```

```
        begin
          if vet[j] > vet [j+1]
          then begin
               // a troca está sendo realizada
               // com a ajuda da variável AUX
               aux:=vet[j];
               vet[j]:=vet[j+1];
               vet[j+1]:=aux;
              end;
       end;
    end;
    // as linhas abaixo irão atualizar os números da tela
    // com o vetor ordenado
    for i:=1 to 10 do
    begin
    if i = 1
    then pos1.caption:=inttostr(vet[i]);
    if i = 2
    then pos2.caption:=inttostr(vet[i]);
    if i = 3
    then pos3.caption:=inttostr(vet[i]);
    if i = 4
    then pos4.caption:=inttostr(vet[i]);
    if i = 5
    then pos5.caption:=inttostr(vet[i]);
    if i = 6
    then pos6.caption:=inttostr(vet[i]);
    if i = 7
    then pos7.caption:=inttostr(vet[i]);
    if i = 8
    then pos8.caption:=inttostr(vet[i]);
    if i = 9
    then pos9.caption:=inttostr(vet[i]);
    if i = 10
    then pos10.caption:=inttostr(vet[i]);
    end;
   end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
```

```
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

A Figura 1.2 mostra a tela da aplicação que ordena um vetor de forma crescente utilizando o algoritmo de ordenação por troca melhorado. Após a Figura 1.2, veja a aplicação com os respectivos códigos comentados.

Figura 1.2 Tela da aplicação que ordena os dados de um vetor pelo algoritmo de ordenação por troca melhorado



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador VET, com 10 posições
     // para armazenar números inteiros
     vet:array[1..10] of integer;
     // variável com identificador POSLIVRE que mostra
     // se o vetor já foi carregado ou não
     poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero:integer;
```

```
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Ordenação por troca melhorado','');
if num <> ''
then begin
     numero:=strtoint(num);
     if quantidade = 1
     then begin
          pos1.visible:=true;
          pos1.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 2
     then begin
          pos2.visible:=true;
          pos2.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 3
     then begin
          pos3.visible:=true;
          pos3.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 4
     then begin
          pos4.visible:=true;
          pos4.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 5
     then begin
          pos5.visible:=true;
          pos5.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 6
     then begin
        pos6.visible:=true;
        pos6.caption:=num;
        vet[quantidade]:=numero;
        end;
    if quantidade = 7
    then begin
         pos7.visible:=true;
         pos7.caption:=num;
         vet[quantidade]:=numero;
```

```
          end;
      if quantidade = 8
      then begin
          pos8.visible:=true;
          pos8.caption:=num;
          vet[quantidade]:=numero;
          end;
      if quantidade = 9
      then begin
          pos9.visible:=true;
          pos9.caption:=num;
          vet[quantidade]:=numero;
          end;
      if quantidade = 10
      then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
      quantidade:=quantidade+1;
     end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Ordenar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, j, aux, achou: integer;
begin
// este procedimento vai executar o algoritmo de ordenação por troca melhorado;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                 no botão Carregar')
else begin
     // o laço a seguir deve repetir enquanto houver alguma troca
    // a variável ACHOU marca se alguma troca ocorreu
    // ACHOU = 0 - nenhuma troca ocorreu
    // ACHOU = 1 - pelo menos uma troca ocorreu
    achou:=1;
    while achou = 1 do
    begin
      // a variável ACHOU marca se alguma troca ocorreu
      // logo, deve ser reinicializada
      achou:=0;
      // o laço abaixo vai comparar uma posição com a próxima
      // como a comparação é sempre com a próxima, o laço vai até 9,
      // pois, se fosse até 10, teríamos uma comparação
```

```
      // entre a posição 10 e 11 que não existe
      for j:=1 to 9 do
       begin
         if vet[j] > vet [j+1]
         then begin
               // a troca está sendo realizada
               // com a ajuda da variável AUX
               achou:=1;
               aux:=vet[j];
               vet[j]:=vet[j+1];
               vet[j+1]:=aux;
              end;
       end;
    end;
     // as linhas abaixo irão atualizar os números da tela
     // com o vetor ordenado
    for i:=1 to 10 do
    begin
    if i = 1
    then pos1.caption:=inttostr(vet[i]);
    if i = 2
    then pos2.caption:=inttostr(vet[i]);
    if i = 3
    then pos3.caption:=inttostr(vet[i]);
    if i = 4
    then pos4.caption:=inttostr(vet[i]);
    if i = 5
    then pos5.caption:=inttostr(vet[i]);
    if i = 6
    then pos6.caption:=inttostr(vet[i]);
    if i = 7
    then pos7.caption:=inttostr(vet[i]);
    if i = 8
    then pos8.caption:=inttostr(vet[i]);
    if i = 9
    then pos9.caption:=inttostr(vet[i]);
    if i = 10
    then pos10.caption:=inttostr(vet[i]);
    end;
    end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
```

```
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Algoritmo de ordenação por inserção

Neste algoritmo será eleito o segundo número do vetor para iniciar a ordenação; assim, os elementos à esquerda do número eleito estão sempre ordenados de forma crescente ou decrescente. Logo, um laço com as comparações será efetuado na quantidade de vezes igual ao número de dados do vetor menos um, pois as comparações começam no segundo elemento do vetor. A seguir, o algoritmo de ordenação por inserção será ilustrado com um vetor que possui dez posições, desse modo, o laço será executado nove vezes e tendo como objetivo ordenar os dados do vetor de forma crescente.

Execução número 1 do laço:

Vet	5	4	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Para iniciar o algoritmo de ordenação por inserção, o primeiro número eleito é o da posição 2 (número 4) que será comparado com todos os números que estão à sua esquerda ou até encontrar um número maior que ele. Quando um número maior que o eleito for encontrado, ocorrerá um deslocamento dos elementos para a direita, abrindo espaço para a mudança de posição do número eleito. Do contrário, todos os números à esquerda do número eleito serão menores e este já estará na posição correta, e a eleição do próximo número acontecerá.

Nesse vetor ocorrerá a comparação do número da posição 2 (número 4) com o número da posição 1 (número 5), que é o único número à esquerda da posição 2. Como o número 4 é menor que o número 5 e se deseja realizar

a ordenação de forma crescente, acontecerá o deslocamento do número 5 para a direita, e o número 4 será recolocado no vetor.

eleito = 4 (número da posição 2 que será recolocado no vetor)

Todos os números à esquerda da posição 2 serão deslocados para a direita, nesse caso, apenas o número da posição 1 (número 5) será deslocado, o número eleito será recolocado, e o próximo número (número da posição 3 — número 2) será eleito para comparação.

Execução número 2 do laço:

Vet	4	5	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 3 (número 2) com os números das posições 1 (número 4) e 2 (número 5), que estão à esquerda da posição 3. Como o número 2 é menor que o número 4 e se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento do número 5 e do número 4 para a direita, e o número 2 será recolocado no vetor.

eleito = 2 (número da posição 3 que será recolocado no vetor)

Todos os números à esquerda da posição 3 serão deslocados para a direita; neste caso, o número da posição 2 (número 5) e o da posição 1 (número 4) serão deslocados, o número eleito será recolocado e o próximo número (número da posição 4 — número 1) será eleito para comparação.

Execução número 3 do laço:

Vet	2	4	5	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 4 (número 1) com os números das posições 1 (número 2), 2 (número 4) e 3 (número 5), que estão à esquerda da posição 4. Como o número 1 é menor que o número 2 e se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento dos números 5, 4 e 2 para a direita, e o número 1 será recolocado no vetor.

eleito = 1 (número da posição 4 que será recolocado no vetor)

Todos os números à esquerda da posição 4 serão deslocados para a direita; neste caso, o número da posição 3 (número 5), o da posição 2 (número 4) e o

da posição 1 (número 2) serão deslocados, o número eleito será recolocado e o próximo número (número da posição 5 — número 8) será eleito para comparação.

Execução número 4 do laço:

Vet	1	2	4	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 5 (número 8) com os números das posições 1 (número 1), 2 (número 2), 3 (número 4) e 4 (número 5), que estão à esquerda da posição 5. Como o número 8 não é menor que nenhum número que está à sua esquerda, não acontecerá o deslocamento dos números, e o próximo número (número da posição 6 — número 2) será eleito para comparação.

Execução número 5 do laço:

Vet	1	2	4	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 6 (número 2) com os números das posições 1 (número 1), 2 (número 2), 3 (número 4), 4 (número 5) e 5 (número 8), que estão à esquerda da posição 6. Lembre-se de que essas comparações param no momento em que for encontrado um número maior que o número eleito, pois os números à esquerda do número eleito já estão ordenados.

Quando acontece a comparação do número eleito (número 2) com o número da posição 1 (número 1), verifica-se que este não é maior que o número eleito, então a comparação ocorre com o número da posição 2 (número 2), que também não é maior que o eleito; desse modo, a comparação acontece com o número da posição 3 (número 4), que é maior que o número eleito, logo, como se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento dos números 8, 5 e 4 para a direita, e o número 2 será recolocado no vetor. Percebe-se que serão deslocados apenas os números 8, 5 e 4, que são maiores que o número eleito.

eleito = 2 (número da posição 6 que será recolocado no vetor)

Os números à esquerda da posição 6, maiores que o eleito, serão deslocados para a direita; nesse caso, o número da posição 5 (número 8), o da

posição 4 (número 5) e o da posição 3 (número 4) serão deslocados, o eleito será recolocado e o próximo número (número da posição 7 — número 4) será escolhido para comparação.

Execução número 6 do laço:

Vet	1	2	2	4	5	8	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 7 (número 4) com os números das posições 1 (número 1), 2 (número 2), 3 (número 2), 4 (número 4), 5 (número 5) e 6 (número 8), que estão à esquerda da posição 7. Lembre-se de que essas comparações param no momento em que for encontrado um número maior que o eleito, pois os números à esquerda do eleito já estão ordenados.

Quando acontece a comparação do eleito (número 4) com o número da posição 1 (número 1), verifica-se que este não é maior que o eleito, então a comparação ocorre com o número da posição 2 (número 2), que também não é maior que o número eleito; desse modo, a comparação acontece com o número da posição 3 (número 2), que também não é maior que o número eleito. Logo, a comparação acontece com o número da posição 4 (número 4) que também não é maior que o número eleito, então, a comparação acontece com o número da posição 5 (número 5), que é maior que o número eleito; assim, como se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento dos números 8 e 5 para a direita, e o número 4 será recolocado no vetor. Percebe-se que serão deslocados apenas os números 8 e 5, que são maiores que o número eleito.

eleito = 4 (número da posição 7 que será recolocado no vetor)

Os números à esquerda da posição 7, maiores que o número eleito, serão deslocados para a direita; nesse caso, o número da posição 6 (número 8) e o da posição 5 (número 5) serão deslocados, o número eleito será recolocado, e o próximo número (número da posição 8 — número 5) será escolhido para comparação.

Execução número 7 do laço:

Vet	1	2	2	4	4	5	8	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 8 (número 5) com os números das posições 1 (número 1), 2 (número 2), 3 (número 2), 4 (número 4), 5 (número 4), 6 (número 5) e 7 (número 8), que estão à esquerda da posição 8. Lembre-se de que essas comparações param no momento em que for encontrado um número maior que o eleito, pois os números à esquerda do eleito já estão ordenados.

Quando acontece a comparação do eleito (número 5) com o número da posição 1 (número 1), verifica-se que este não é maior que o eleito, então a comparação ocorre com o número da posição 2 (número 2), que também não é maior que o número eleito, assim a comparação acontece com o número da posição 3 (número 2), que também não é maior que o eleito; desse modo, a comparação acontece com o número da posição 4 (número 4), que também não é maior que o número eleito, então a comparação ocorre com o número da posição 5 (número 4), que também não é maior que o número eleito. Assim, a comparação acontece com o número da posição 6 (número 5), que também não é maior que o número eleito, então a comparação ocorre com o número da posição 7 (número 8), que é maior que o número eleito. Logo, como se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento do número 8 para a direita, e o número 5 será recolocado no vetor. Percebe-se que será deslocado apenas o número 8, que é o único maior que o número eleito.

eleito = 5 (número da posição 8 que será recolocado no vetor)

Os números à esquerda da posição 8, maiores que o número eleito, serão deslocados para a direita; nesse caso, apenas o número da posição 7 (número 8) será deslocado, o número eleito será recolocado, e o próximo número (número da posição 9 — número 9) será eleito para comparação.

Execução número 8 do laço:

Vet	1	2	2	4	4	5	5	8	9	7
	1	2	3	4	5	6	7	8	9	10

Nesse vetor ocorrerá a comparação do número da posição 9 (número 9) com os das posições 1 (número 1), 2 (número 2), 3 (número 2), 4 (número 4), 5 (número 4), 6 (número 5), 7 (número 5) e 8 (número 8), que estão à esquerda da posição 9. Como o número 9 não é menor que nenhum número que está à sua esquerda, não acontecerá o deslocamento dos números, e o próximo número (número da posição 10 — número 7) será eleito para comparação.

Execução número 9 do laço (última execução):

Vet	1	2	2	4	4	5	5	8	9	7
	1	2	3	4	5	6	7	8	9	10

Nesse vetor ocorrerá a comparação do número da posição 10 (número 7) com os números das posições 1 (número 1), 2 (número 2), 3 (número 2), 4 (número 4), 5 (número 4), 6 (número 5), 7 (número 5), 8 (número 8) e 9 (número 9), que estão à esquerda da posição 10. Lembre-se de que essas comparações param no momento em que for encontrado um número maior que o eleito, pois os números à esquerda do eleito já estão ordenados.

Quando acontece a comparação do número eleito (número 7) com o número da posição 1 (número 1), verifica-se que esse não é maior que o eleito, então a comparação ocorre com o número da posição 2 (número 2), que também não é maior que o eleito, assim a comparação acontece com o número da posição 3 (número 2), que também não é maior que o eleito, ou seja, a comparação acontece com o número da posição 4 (número 4), que também não é maior que o número eleito, logo a comparação acontece com o número da posição 5 (número 4), que também não é maior que o eleito; assim , a comparação acontece com o número da posição 6 (número 5), que também não é maior que o eleito, então a comparação ocorre com o número da posição 7 (número 5), que também não é maior que o número eleito, desse modo, a comparação acontece com o número da posição 8 (número 8), que é maior que o número eleito. Como se deseja realizar a ordenação de forma crescente, acontecerá o deslocamento dos números 8 e 9 para a direita, e o número 7 será recolocado no vetor. Percebe-se que serão deslocados os números 8 e 9, que são os números maiores que o eleito.

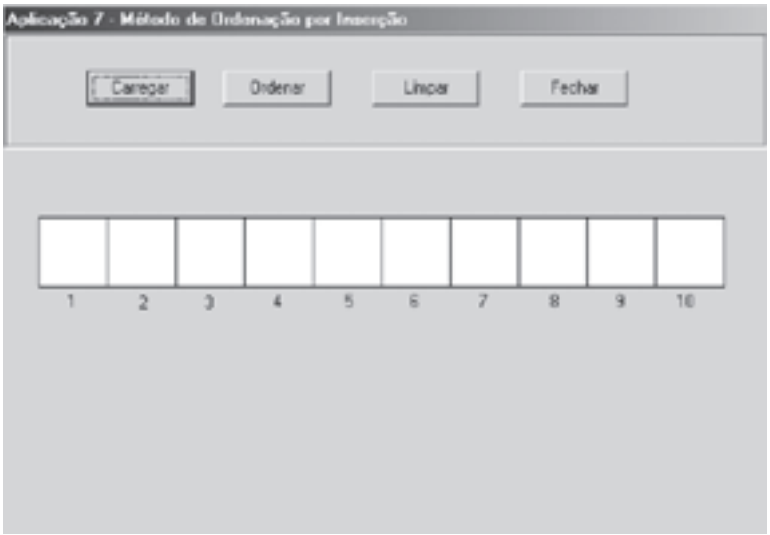
eleito = 7 (número da posição 10 que será recolocado no vetor)

Os números à esquerda da posição 10, maiores que o eleito, serão deslocados para a direita, nesse caso, os números das posições 8 (número 8) e 9 (número 9) serão deslocados, o número eleito recolocado, finalizando assim o processo de ordenação, como mostra o vetor a seguir.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10

A Figura 1.3 mostra a tela da aplicação que ordena um vetor de forma crescente utilizando o algoritmo de ordenação por inserção. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 1.3 Tela da aplicação que ordena os dados de um vetor pelo algoritmo de ordenação por inserção



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador VET, com 10 posições
     // para armazenar números inteiros
     vet:array[1..10] of integer;
     // variável com identificador POSLIVRE que mostra
     // se o vetor já foi carregado ou não
     poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Ordenação por inserção','');
if num <> ''
then begin
```

```
numero:=strtoint(num);
if quantidade = 1
then begin
     pos1.visible:=true;
     pos1.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 2
then begin
     pos2.visible:=true;
     pos2.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 3
then begin
     pos3.visible:=true;
     pos3.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 4
then begin
     pos4.visible:=true;
     pos4.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 5
then begin
     pos5.visible:=true;
     pos5.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 6
then begin
     pos6.visible:=true;
     pos6.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 7
then begin
     pos7.visible:=true;
     pos7.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 8
then begin
     pos8.visible:=true;
     pos8.caption:=num;
     vet[quantidade]:=numero;
     end;
if quantidade = 9
then begin
     pos9.visible:=true;
```

```
          pos9.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 10
     then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
     quantidade:=quantidade+1;
   end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Ordenar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, j, aux, achou, eleito, posi: integer;
begin
// este procedimento vai executar o algoritmo
// de ordenação por inserção:
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio e poslivre = 1 - vetor com dados
if POSLIVRE = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     // o laço abaixo vai repetir o número de vezes
     // igual ao número de elementos do vetor - 1;
     // neste caso, tem-se um vetor com 10 elementos,
     // logo, o laço será com 9 repetições
     // começando de 2, pois o primeiro número eleito
     // para as comparações será o da posição 2
    for i:=2 to 10 do
    begin
      // eleição do número que será comparado
      eleito:= vet[i];
      // começando as comparações sempre da posição 1, logo j = 1
      j:= 1;
      // a variável ACHOU marca se foi encontrado um
      // número maior que o número eleito;
      // como as comparações irão começar, a variável ACHOU
      // começa valendo zero
      achou:=0;
      // o laço abaixo será executado o número de vezes
      // igual à posição do número eleito menos 1,
      // e enquanto não encontrar um número maior
      // que o número eleito
      while (j < i) and (achou = 0)do
       begin
         if vet[j] > eleito
```

```
          then begin
                // a variável ACHOU será marcada
                achou:=1;
                // a variável POSI vai guardar
                // a posição do deslocamento
                posi:=j;
               end
          else begin
                // avança uma posição do vetor para comparação
                j:=j+1;
                end;
        end;
        if achou = 1
        then begin
             // ocorrerá o deslocamento dos números no vetor;
             // o laço vai da posição do número eleito
             // até a posição encontrada para deslocamento
             // que está na variável POSI
             for j:=i downto posi do
             begin
             vet[j]:=vet[j-1];
             end;
             // o número eleito será recolocado na posição ordenada
             vet[posi]:=eleito;
            end;
    end;
     // as linhas abaixo irão atualizar os números da tela
     // com o vetor ordenado
    for i:=1 to 10 do
    begin
    if i = 1
    then pos1.caption:=inttostr(vet[i]);
    if i = 2
    then pos2.caption:=inttostr(vet[i]);
    if i = 3
    then pos3.caption:=inttostr(vet[i]);
    if i = 4
    then pos4.caption:=inttostr(vet[i]);
    if i = 5
    then pos5.caption:=inttostr(vet[i]);
    if i = 6
    then pos6.caption:=inttostr(vet[i]);
    if i = 7
    then pos7.caption:=inttostr(vet[i]);
    if i = 8
    then pos8.caption:=inttostr(vet[i]);
    if i = 9
    then pos9.caption:=inttostr(vet[i]);
    if i = 10
    then pos10.caption:=inttostr(vet[i]);
    end;
    end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir:

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Algoritmo de ordenação por seleção

Neste algoritmo de ordenação será eleito o primeiro número do vetor e será realizada uma comparação do número eleito com todos os números que estão à sua direita. Nessas comparações procura-se um número menor que o número eleito (quando a ordenação for crescente) ou um número maior que o eleito (quando a ordenação for decrescente). Quando um número satisfaz as condições da ordenação desejada (menor ou maior que o número eleito), esse trocará de posição com o número eleito; ou seja, todos os números à esquerda do eleito ficarão sempre ordenados. Neste algoritmo, o laço com as comparações será efetuado na quantidade de vezes igual ao número de dados do vetor menos um, pois as comparações são realizadas com os elementos à direita do eleito, e o número da última posição não tem elementos à direita. A seguir, o algoritmo de ordenação por seleção será ilustrado com um vetor que possui dez posições; o laço será executado nove vezes, tendo como objetivo ordenar os dados do vetor de forma crescente.

Execução número 1 do laço:

Vet	5	4	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Para iniciar o algoritmo de ordenação por seleção, o primeiro número eleito é o número da posição 1 (número 5), que será comparado com o menor número existente entre os que estão à direita do número eleito (números das posições de 2 a 10). Se o menor número à direita for menor que o eleito, ocorrerá uma troca de posições entre o menor número encontrado e o número eleito; caso contrário, ocorrerá apenas o avanço do número eleito.

Nesse vetor ocorrerá a comparação do número da posição 1 (número 5) com o menor número entre os números das posições de 2 a 10. Neste caso, o número eleito (número 5) é comparado com o número da posição 4 (número 1) que é o menor número encontrado; ou seja, eles trocam de posição — o número 5 da posição 1 vai para a posição 4, o número 1 vai da posição 4 para a posição 1, e o próximo número é eleito.

Execução número 2 do laço:

Vet	1	4	2	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 2 (número 4) com o menor número entre os números das posições de 3 a 10. Neste caso, o número eleito (número 4) é comparado com o número da posição 3 (número 2), que é o menor número encontrado; ou seja, eles trocam de posição — o número 4 da posição 2 vai para a posição 3, o número 2 vai da posição 3 para a posição 2, e o próximo número é eleito.

Execução número 3 do laço:

Vet	1	2	4	5	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 3 (número 4) com o menor número entre os números das posições de 4 a 10. Neste caso, o número eleito (número 4) é comparado com o número da posição 6 (número 2),

que é o menor número encontrado; ou seja, eles trocam de posição — o número 4 da posição 3 vai para a posição 6, o número 2 vai da posição 6 para a posição 3, e o próximo número é eleito.

Execução número 4 do laço:

Vet	1	2	2	5	8	4	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 4 (número 5) com o menor número entre os números das posições de 5 a 10. Neste caso, o número eleito (número 5) é comparado com o número da posição 6 (número 4), que é o menor número encontrado; ou seja, eles trocam de posição — o número 5 da posição 4 vai para a posição 6, o número 4 vai da posição 6 para a posição 4, e o próximo número é eleito.

Execução número 5 do laço:

Vet	1	2	2	4	8	5	4	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 5 (número 8) com o menor número entre os números das posições de 6 a 10. Neste caso, o número eleito (número 8) é comparado com o número da posição 7 (número 4), que é o menor número encontrado; ou seja, eles trocam de posição — o número 8 da posição 5 vai para a posição 7, o número 4 vai da posição 7 para a posição 5, e o próximo número é eleito.

Execução número 6 do laço:

Vet	1	2	2	4	4	5	8	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 6 (número 5) com o menor número entre os números das posições de 7 a 10. Neste caso, o número eleito não é menor que o menor número encontrado, ou seja, não é menor que o número 5 da posição 8, então não ocorre nenhuma troca, e o próximo número é eleito.

Execução número 7 do laço:

Vet	1	2	2	4	4	5	8	5	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 7 (número 8) com o menor número entre os números das posições de 8 a 10. Neste caso, o número eleito (número 8) é comparado com o número da posição 8 (número 5), que é o menor número encontrado; ou seja, eles trocam de posição — o número 8 da posição 7 vai para a posição 8, o número 5 vai da posição 8 para a posição 7, e o próximo número é eleito.

Execução número 8 do laço:

Vet	1	2	2	4	4	5	5	8	9	7
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 8 (número 8) com o menor número entre os números das posições de 9 a 10. Neste caso, o número eleito (número 8) é comparado com o número da posição 10 (número 7), que é o menor número encontrado; ou seja, eles trocam de posição — o número 8 da posição 8 vai para a posição 10, o número 7 vai da posição 10 para a posição 8, e o próximo número é eleito.

Execução número 9 do laço (última execução):

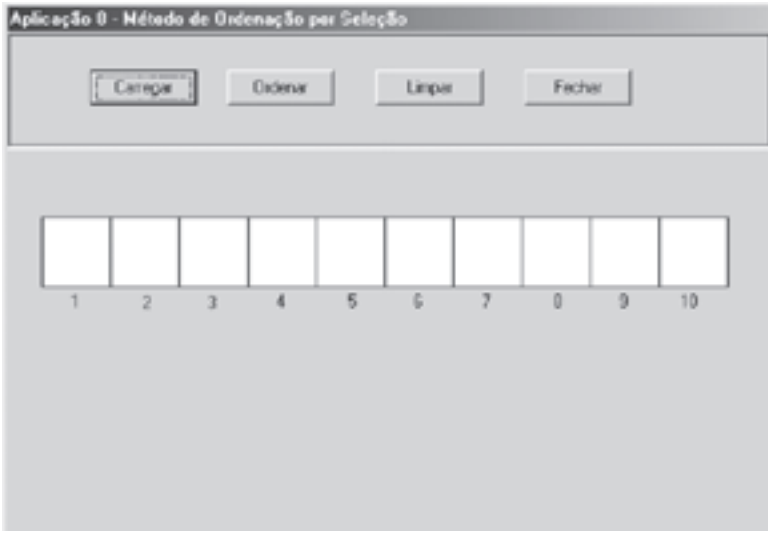
Vet	1	2	2	4	4	5	5	7	9	10
	1	2	3	4	5	6	7	8	9	10



Nesse vetor ocorrerá a comparação do número da posição 9 (número 9) com o número da posição 10, única posição à direita do número eleito. Neste caso, o número eleito não é menor que o menor número encontrado, logo, não haverá troca e, como o laço está na última execução, o vetor está ordenado.

A Figura 1.4 mostra a tela da aplicação que ordena um vetor de forma crescente utilizando o algoritmo de ordenação por seleção. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 1.4 Tela da aplicação que ordena os dados de um vetor pelo algoritmo de ordenação por seleção



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
      // vetor com identificador VET, com 10 posições
      // para armazenar números inteiros
      vet:array[1..10] of integer;
      // variável com identificador POSLIVRE que mostra
      // se o vetor já foi carregado ou não
      poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Ordenação por seleção','');
if num <> ''
then begin
      numero:=strtoint(num);
      if quantidade = 1
      then begin
```

```
          pos1.visible:=true;
          pos1.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 2
     then begin
          pos2.visible:=true;
          pos2.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 3
     then begin
          pos3.visible:=true;
          pos3.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 4
     then begin
          pos4.visible:=true;
          pos4.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 5
     then begin
          pos5.visible:=true;
          pos5.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 6
     then begin
          pos6.visible:=true;
          pos6.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 7
     then begin
          pos7.visible:=true;
          pos7.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 8
     then begin
          pos8.visible:=true;
          pos8.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 9
     then begin
          pos9.visible:=true;
          pos9.caption:=num;
          vet[quantidade]:=numero;
```

```
          end;
      if quantidade = 10
      then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
      quantidade:=quantidade+1;
     end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Ordenar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, j, eleito, posi, aux, menor: integer;
begin
// este procedimento vai executar o algoritmo de ordenação por seleção;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
      // o laço abaixo vai repetir o número de vezes
      // igual ao número de elementos do vetor - 1;
      // neste caso, tem-se um vetor com 10 elementos,
      // logo, o laço terá 9 repetições
      // a variável I guarda a posição do número eleito
     for i:=1 to 9 do
     begin
       // eleição do número que será comparado
       eleito:= vet[i];
       // a variável POSI guarda a posição do menor número
       // à direita do número eleito;
       // a variável MENOR guarda o menor número entre
       // os números que estão à direita do número eleito
       posi:= i+1;
       menor:=vet[i+1];
       // o laço abaixo será executado da posição posterior
       // a posição do número eleito;
       // logo, i + 1, até a última posição, neste exemplo
       // de vetor com 10 posições, até a posição 10
       for j:=i+1 to 10 do
        begin
          if vet[j] < menor
          then begin
                menor:= vet[j];
                posi:=j;
```

```
              end
         end;
        if vet[i] > menor
        then begin
             // as linhas abaixo mostram
             // a troca de posições dos números;
             // a variável AUX auxilia na troca
             aux:= vet[i];
             vet[i]:= menor ;
             vet[posi]:=aux;
             end;
    end;
     // as linhas abaixo irão atualizar os números da tela
     // com o vetor ordenado
    for i:=1 to 10 do
    begin
    if i = 1
    then pos1.caption:=inttostr(vet[i]);
    if i = 2
    then pos2.caption:=inttostr(vet[i]);
    if i = 3
    then pos3.caption:=inttostr(vet[i]);
    if i = 4
    then pos4.caption:=inttostr(vet[i]);
    if i = 5
    then pos5.caption:=inttostr(vet[i]);
    if i = 6
    then pos6.caption:=inttostr(vet[i]);
    if i = 7
    then pos7.caption:=inttostr(vet[i]);
    if i = 8
    then pos8.caption:=inttostr(vet[i]);
    if i = 9
    then pos9.caption:=inttostr(vet[i]);
    if i = 10
    then pos10.caption:=inttostr(vet[i]);
    end;
   end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
```

```
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Algoritmo de ordenação por intercalação

Neste algoritmo de ordenação o vetor a ser ordenado será particionado em dois outros vetores que serão ordenados por qualquer outro algoritmo de ordenação. Posteriormente, os dois subvetores ordenados terão seus números intercalados para formar o vetor digitado agora de maneira ordenada crescente ou decrescente. Neste algoritmo, o laço com as comparações será efetuado na quantidade de vezes igual ao número de dados do vetor. A seguir o algoritmo de ordenação por intercalação será ilustrado com um vetor que possui dez posições; o laço será executado dez vezes, tendo como objetivo ordenar os dados do vetor de forma crescente.

O vetor abaixo é o vetor digitado:

Vet	5	4	2	1	8	2	4	5	9	7
	1	2	3	4	5	6	7	8	9	10

Para iniciar o algoritmo de ordenação por intercalação, o vetor digitado é particionado em dois outros vetores, e esses são ordenados por qualquer outro algoritmo de ordenação existente.

A seguir estão os dois subvetores, ou seja, o vetor digitado particionado.

Vet1	5	4	2	1	8
	1	2	3	4	5

Vet2	2	4	5	9	7
	1	2	3	4	5

A seguir, os dois subvetores são ordenados por qualquer algoritmo de ordenação.

Vet1	1	2	4	5	8
	1	2	3	4	5

Vet1	2	4	5	7	9
	1	2	3	4	5

Por meio dos dois subvetores ordenados, são realizadas comparações para encontrar o menor número entre os dois subvetores (quando a ordenação é crescente) e colocá-lo no vetor resultante ordenado.

Execução número 1 do laço:

Vet1	1	2	4	5	8
	1	2	3	4	5



Vet2	2	4	5	7	9
	1	2	3	4	5



Nesses vetores ocorrerá a comparação do número da posição 1 do subvetor1 (número 1) com o número da posição 1 do subvetor2 (número 2). Nessa comparação o número do subvetor1 é menor e será armazenado no vetor resultante.

Vet	1									
	1	2	3	4	5	6	7	8	9	10



Execução número 2 do laço:

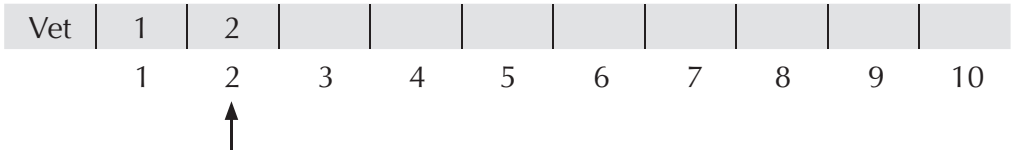
Vet1	1	2	4	5	8
	1	2	3	4	5



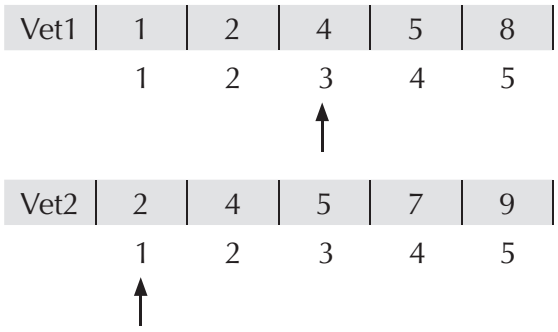
Vet2	2	4	5	7	9
	1	2	3	4	5



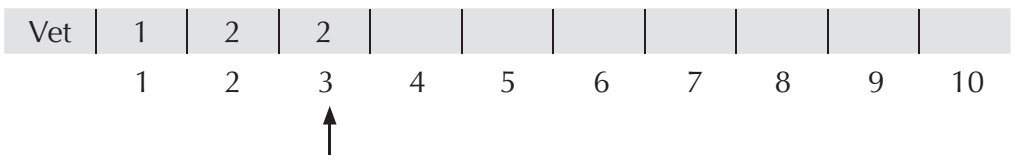
Nesses vetores ocorrerá a comparação do número da posição 2 do subvetor1 (número 2) com o número da posição 1 do subvetor2 (número 2). Nessa comparação os números são iguais, logo, tanto faz armazenar um ou outro, assim, o número do subvetor1 será armazenado no vetor resultante.



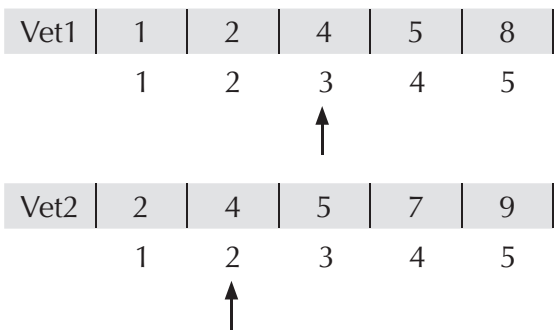
Execução número 3 do laço:



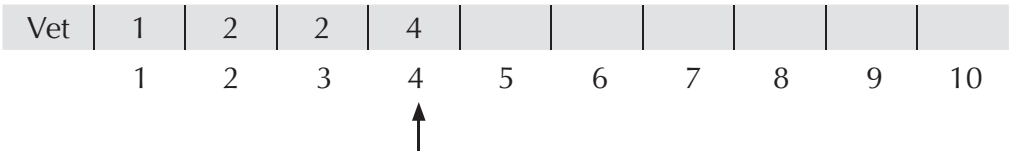
Nesses vetores ocorrerá a comparação do número da posição 3 do subvetor1 (número 4) com o número da posição 1 do subvetor2 (número 2). Nessa comparação o número do subvetor2 é menor e será armazenado no vetor resultante.



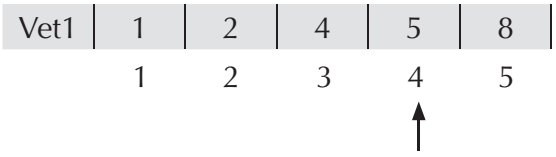
Execução número 4 do laço:

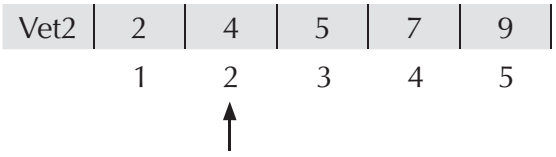


Nesses vetores ocorrerá a comparação do número da posição 3 do subvetor1 (número 4) com o número da posição 2 do subvetor2 (número 4). Nessa comparação os números são iguais, logo, tanto faz armazenar um ou outro, assim, o número do subvetor1 será armazenado no vetor resultante.

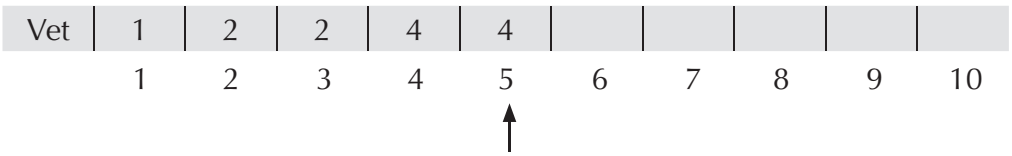


Execução número 5 do laço:

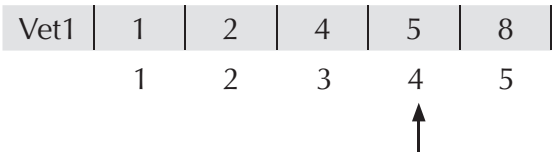


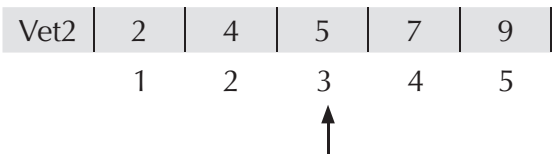


Nesses vetores ocorrerá a comparação do número da posição 4 do subvetor1 (número 5) com o número da posição 2 do subvetor2 (número 4). Nessa comparação o número do subvetor2 é menor e será armazenado no vetor resultante.

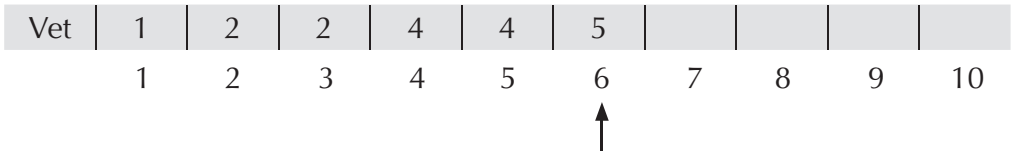


Execução número 6 do laço:

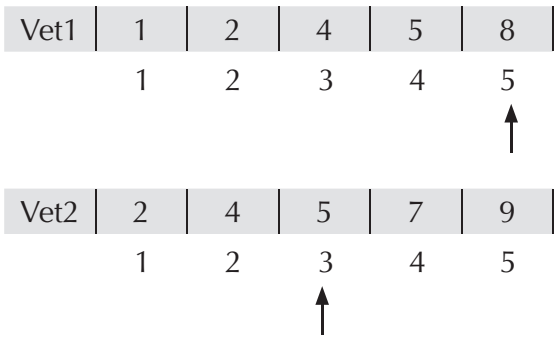




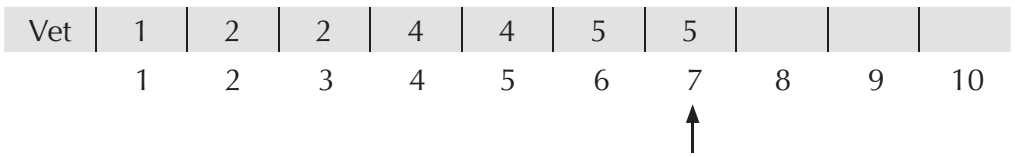
Nos vetores acima ocorrerá a comparação do número da posição 4 do subvetor1 (número 5) com o número da posição 3 do subvetor2 (número 5). Nessa comparação os números são iguais, logo, tanto faz armazenar um ou outro, assim, o número do subvetor1 será armazenado no vetor resultante.



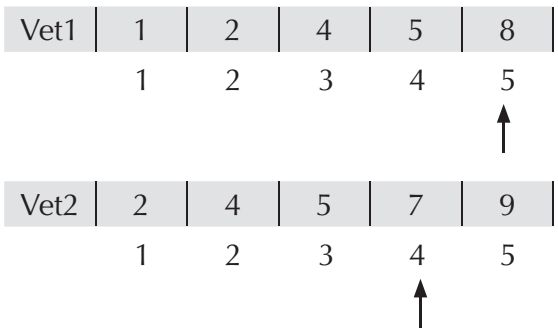
Execução número 7 do laço:



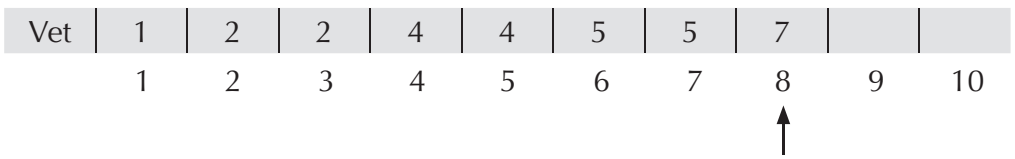
Nesses vetores ocorrerá a comparação do número da posição 5 do subvetor1 (número 8) com o número da posição 3 do subvetor2 (número 5). Nessa comparação o número do subvetor2 é menor e será armazenado no vetor resultante.



Execução número 8 do laço:



Nesses vetores ocorrerá a comparação do número da posição 5 do subvetor1 (número 8) com o número da posição 4 do subvetor2 (número 7). Nessa comparação o número do subvetor2 é menor e será armazenado no vetor resultante.



Execução número 9 do laço:

Vet1	1	2	4	5	8
	1	2	3	4	5

Vet2	2	4	5	7	9
	1	2	3	4	5

Nesses vetores ocorrerá a comparação do número da posição 5 do subvetor1 (número 8) com o número da posição 5 do subvetor2 (número 9). Nessa comparação o número do subvetor1 é menor e será armazenado no vetor resultante.

Vet	1	2	2	4	4	5	5	7	8	
	1	2	3	4	5	6	7	8	9	10

Execução número 10 do laço (última execução):

Vet1	1	2	4	5	8
	1	2	3	4	5

Vet2	2	4	5	7	9
	1	2	3	4	5

Como todos os números do subvetor1 já foram armazenados, automaticamente o número do subvetor2 será armazenado no vetor resultante. Como esta é a última execução do laço, o vetor a seguir já se encontra ordenado.

Vet	1	2	2	4	4	5	5	7	8	9
	1	2	3	4	5	6	7	8	9	10

A Figura 1.5 mostra a tela da aplicação que ordena um vetor de forma crescente utilizando o algoritmo de ordenação por intercalação. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 1.5 Tela da aplicação que ordena os dados de um vetor pelo algoritmo de ordenação por intercalação



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador VET, com 10 posições
     // para armazenar números inteiros
     vet:array[1..10] of integer;
     // variável com identificador POSLIVRE que mostra
     // se o vetor já foi carregado ou não
     poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Ordenação por intercalação','');
if num <> ''
then begin
      numero:=strtoint(num);
```

```
     if quantidade = 1
     then begin
          pos1.visible:=true;
          pos1.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 2
     then begin
          pos2.visible:=true;
          pos2.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 3
     then begin
          pos3.visible:=true;
          pos3.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 4
     then begin
          pos4.visible:=true;
          pos4.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 5
     then begin
          pos5.visible:=true;
          pos5.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 6
     then begin
          pos6.visible:=true;
          pos6.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 7
     then begin
          pos7.visible:=true;
          pos7.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 8
     then begin
          pos8.visible:=true;
          pos8.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 9
```

```
      then begin
           pos9.visible:=true;
           pos9.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 10
      then begin
           pos10.visible:=true;
           pos10.caption:=num;
           vet[quantidade]:=numero;
           end;
      quantidade:=quantidade+1;
     end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Ordenar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, j, k, eleito, posi, aux: integer;
begin
// este procedimento vai executar o algoritmo
// de ordenação por intercalação;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     // o vetor digitado será particionado em dois subvetores
     // j - índice do subvetor1
     j:=1;
     // k - índice do subvetor2
     k:=1;
     for i:=1 to 10 do
     begin
      if i <= 5
      then begin
             vet1[j]:=vet[i];
             j:=j+1;
           end
      else begin
             vet2[k]:=vet[i];
             k:=k+1;
           end;
     end;
```

```
// ordenando o subvetor1 pelo algoritmo da ordenação por troca
for i:=1 to 4 do
begin
 for j:=1 to 4 do
   begin
     if vet1[j] > vet1 [j+1]
     then begin
           aux:=vet1[j];
           vet1[j]:=vet1[j+1];
           vet1[j+1]:=aux;
          end;
   end;
end;
// ordenando o subvetor2 pelo algoritmo da ordenação por troca
for i:=6 to 9 do
begin
 for k:=1 to 4 do
   begin
     if vet2[k] > vet2 [k+1]
     then begin
           aux:=vet2[k];
           vet2[k]:=vet2[k+1];
           vet2[k+1]:=aux;
          end;
   end;
end;
// o laço abaixo vai intercalar os dois subvetores
// gerando o vetor ordenado que possui 10 posições
// j - índice do subvetor1
j:=1;
// j = 1 para que a primeira posição
// do subvetor1 seja analisada
// k - índice do subvetor2
k:=1;
// k = 1 para que a primeira posição do subvetor2
// seja analisada
for i:=1 to 10 do
begin
if j = 6
then begin
      // todos os números do subvetor1 já foram
      // colocados no vetor resultante;
      // logo, sobram apenas números do subvetor2
      vet[i]:=vet2[k];
      k := k+1;
     end
else if k = 6
      then begin
             // todos os números do subvetor2 já foram colocados
```

```
                // no vetor resultante;
                // logo, sobram apenas números do subvetor1
                vet[i]:=vet1[j];
                j := j+1;
              end
         else if vet1[j] <= vet2[k]
                then begin
                     // ainda existem números nos dois subvetores
                     // para serem colocados no vetor resultante;
                     // neste caso, o número analisado no subvetor1
                     // é menor ou igual ao número analisado
                     // no subvetor2, logo, o número do subvetor1
                     // será colocado no vetor resultante
                      vet[i]:=vet1[j];
                      j := j+1;
                      end
                else begin
                     // ainda existem números nos dois subvetores
                     // para serem colocados no vetor resultante;
                     // neste caso, o número analisado no subvetor2
                     // é menor que o número analisado no subvetor1,
                     // logo, o número do subvetor2
                     // será colocado no vetor resultante
                     vet[i]:=vet2[k];
                     k := k+1;
                     end;
    end;
     // as linhas abaixo irão atualizar os números da tela
     // com o vetor ordenado
    for i:=1 to 10 do
    begin
    if i = 1
    then pos1.caption:=inttostr(vet[i]);
    if i = 2
    then pos2.caption:=inttostr(vet[i]);
    if i = 3
    then pos3.caption:=inttostr(vet[i]);
    if i = 4
    then pos4.caption:=inttostr(vet[i]);
    if i = 5
    then pos5.caption:=inttostr(vet[i]);
    if i = 6
    then pos6.caption:=inttostr(vet[i]);
    if i = 7
    then pos7.caption:=inttostr(vet[i]);
    if i = 8
    then pos8.caption:=inttostr(vet[i]);
    if i = 9
    then pos9.caption:=inttostr(vet[i]);
```

```
     if i = 10
     then pos10.caption:=inttostr(vet[i]);
     end;
    end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Unidade 2

Algoritmos de busca em vetores*

Existem situações em que se tem um vetor com dados desordenados ou ordenados e se deseja localizar um dado específico, logo, para realizar o procedimento de busca, alguns algoritmos podem ser usados. Veja a seguir alguns algoritmos de busca.

Algoritmo de busca seqüencial

O algoritmo de busca seqüencial é utilizado para localizar um dado dentro de um vetor e este vetor pode estar ordenado ou não. Para iniciar o algoritmo, o dado que se deseja localizar é digitado pelo usuário, e um laço com as comparações é efetuado na quantidade de vezes igual ao número de dados do vetor ou até encontrar o dado desejado. A seguir, o algoritmo de busca seqüencial é ilustrado com um vetor que possui dez posições, logo, o laço é executado dez vezes ou até encontrar o dado desejado.

O vetor apresentado abaixo não possui números repetidos, ou seja, o dado procurado é o número 6, e este é comparado com todas as posições do vetor digitado ou até que uma ocorrência seja encontrada.

Execução número 1 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 1 (número 5) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

* Esta unidade corresponde ao Capítulo 3 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005.

Execução número 2 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 2 (número 4) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 3 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 3 (número 7) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 4 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 4 (número 8) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 5 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 5 (número 2) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 6 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 6 (número 0) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 7 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 7 (número 14) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 8 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 8 (número 9) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 9 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 9 (número 15) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 10 do laço (última execução):

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10

Compara-se o número da posição 10 (número 3) com o número procurado (número 6). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes e, como esta é a última execução do laço, conclui-se que o número digitado não foi encontrado.

O dado procurado agora é o número 4, logo, este será comparado com todas as posições do vetor digitado ou até uma ocorrência ser encontrada.

Execução número 1 do laço:

Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10

Compara-se o número da posição 1 (número 5) com o número procurado (número 4). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são diferentes, então ocorre o avanço.

Execução número 2 do laço:

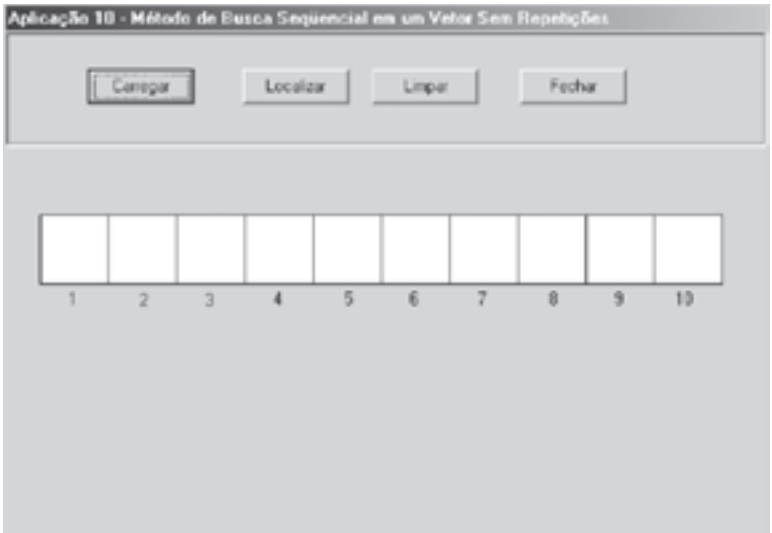
Vet	5	4	7	8	2	0	14	9	15	3
	1	2	3	4	5	6	7	8	9	10

Compara-se o número da posição 2 (número 4) com o número procurado (número 4). Se forem iguais, pára-se a execução do laço. Do contrário, avança-se com a pesquisa. Neste caso, os números são iguais, logo, o laço é interrompido, pois o número foi encontrado na posição 2 do vetor.

É importante lembrar que a interrupção do laço só pode acontecer se o vetor não possuir números repetidos, pois, quando a repetição é permitida, mesmo quando o número é encontrado, a busca precisa continuar até a última posição do vetor.

A Figura 2.1 mostra a tela da aplicação que utiliza o algoritmo de busca seqüencial para localizar determinado dado em um vetor não ordenado e que não possui dados repetidos. Após a figura, têm-se os respectivos códigos comentados.

Figura 2.1 Tela da aplicação que busca um dado em um vetor não ordenado e sem repetições pelo algoritmo de busca seqüencial



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
      // vetor com identificador VET, com 10 posições
      // para armazenar números inteiros
      vet:array[1..10] of integer;
      // variável com identificador POSLIVRE que mostra
      // se o vetor já foi carregado ou não
      poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero, achou, j:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Busca seqüencial','');
if num <> ''
then begin
      numero:=strtoint(num);
```

```
if quantidade = 1
then begin
     pos1.visible:=true;
     pos1.caption:=num;
     vet[quantidade]:=numero;
     end;
// verifica se o número digitado já existe no vetor
//  pois este não pode aceitar repetições;
// a variável ACHOU marca se o número já existe ou não no vetor
// ACHOU = 0 - número não existe
// ACHOU = 1 - número existe
achou:=0;
for j:=1 to quantidade-1 do
begin
if vet[j] = numero
then achou := 1;
end;
if achou = 0
then begin
     if quantidade = 2
      then begin
           pos2.visible:=true;
           pos2.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 3
      then begin
           pos3.visible:=true;
           pos3.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 4
      then begin
           pos4.visible:=true;
           pos4.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 5
      then begin
           pos5.visible:=true;
           pos5.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 6
      then begin
           pos6.visible:=true;
           pos6.caption:=num;
           vet[quantidade]:=numero;
           end;
```

```
            if quantidade = 7
            then begin
                 pos7.visible:=true;
                 pos7.caption:=num;
                 vet[quantidade]:=numero;
                 end;
            if quantidade = 8
            then begin
                 pos8.visible:=true;
                 pos8.caption:=num;
                 vet[quantidade]:=numero;
                 end;
            if quantidade = 9
            then begin
                 pos9.visible:=true;
                 pos9.caption:=num;
                 vet[quantidade]:=numero;
                 end;
            if quantidade = 10
            then begin
                 pos10.visible:=true;
                 pos10.caption:=num;
                 vet[quantidade]:=numero;
                 end;
            quantidade:=quantidade+1;
          end
     else showmessage('Este número já existe no vetor');
     end;
   end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Localizar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, achou, numero, posi: integer;
    num: string;
begin
// este procedimento vai executar o algoritmo de busca seqüencial;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     num:=inputbox('Busca Seqüencial','Digite o número a ser
                  procurado','');
```

```
     if num <> ''
      then begin
            numero:=strtoint(num);
            i:=1;
            // a variável ACHOU é que marca se o número
            // foi encontrado ou não
            // se ACHOU = 0 então o número não foi encontrado
            // se ACHOU = 1 então o número foi encontrado
            achou:=0;
            // o laço deve procurar até a posição 10
            // que é o tamanho do vetor
            // ou até encontrar o número procurado;
            // se encontrar o número procurado, a busca
            // deve ser finalizada,
            // pois não existem números repetidos
            while (i <= 10) and (achou = 0) do
            begin
              if vet[i] = numero
                then begin
                       achou:=1;
                        // a variável POSI marca a posição
                        // em que o número foi encontrado
                       posi:=i;
                     end
                else i:=i+1;
            end;
            if achou = 0
            then showmessage(' O número '+ num+' não foi encontrado no
                              vetor')
            else showmessage(' O número '+ num+' foi encontrado na
                             posição '+ inttostr(posi));
          end;
     end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
```

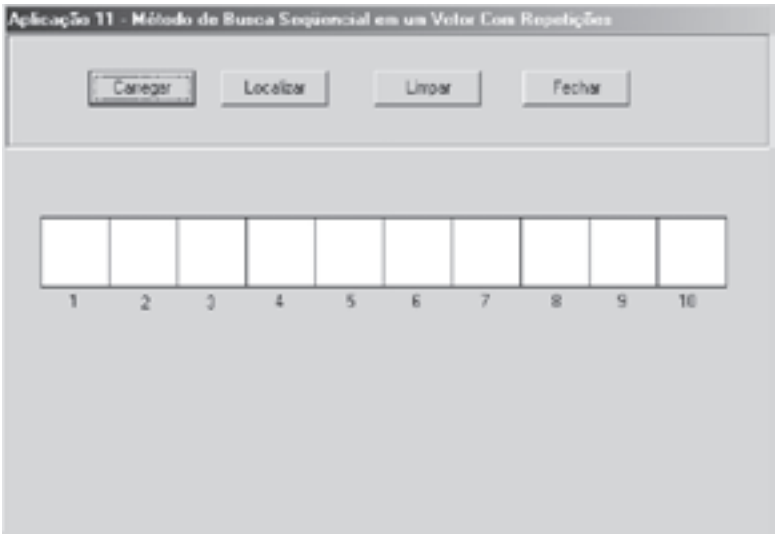
```
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

A Figura 2.2 mostra a tela da aplicação que utiliza o algoritmo de busca seqüencial para localizar determinado dado em um vetor não ordenado e que possui dados repetidos. Após a figura, têm-se seus respectivos códigos comentados.

Figura 2.2 Tela da aplicação que busca um dado em um vetor não ordenado e com repetições pelo algoritmo de busca seqüencial



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
      // vetor com identificador VET, com 10 posições
      // para armazenar números inteiros;
      // vetor com identificador POSI, com 10 posições
```

```
      // para armazenar as posições do número
      // procurado no vetor digitado
      vet, posi:array[1..10] of integer;
      // a variável POSLIVRE marca se os números
      // foram digitados no vetor;
      // a variável POSI_LIVRE marca se o vetor de posições
      // possui algum elemento
      poslivre, posi_livre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero, achou, j:integer;
begin
// este procedimento vai carregar o vetor
// com os números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Busca seqüencial','');
if num <> ''
then begin
      numero:=strtoint(num);
      if quantidade = 1
      then begin
           pos1.visible:=true;
           pos1.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 2
      then begin
           pos2.visible:=true;
           pos2.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 3
      then begin
           pos3.visible:=true;
           pos3.caption:=num;
           vet[quantidade]:=numero;
           end;
      if quantidade = 4
      then begin
           pos4.visible:=true;
           pos4.caption:=num;
```

```
          vet[quantidade]:=numero;
          end;
     if quantidade = 5
     then begin
          pos5.visible:=true;
          pos5.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 6
     then begin
          pos6.visible:=true;
          pos6.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 7
     then begin
          pos7.visible:=true;
          pos7.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 8
     then begin
          pos8.visible:=true;
          pos8.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 9
     then begin
          pos9.visible:=true;
          pos9.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 10
     then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
     quantidade:=quantidade+1;
    end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
end;
```

O código do botão *Localizar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, achou, numero: integer;
```

```
     num, texto: string;
begin
// este procedimento vai executar o algoritmo de busca seqüencial;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     num:=inputbox('Busca Seqüencial','Digite o número a ser
                  procurado','');
     if num <> ''
      then begin
            numero:=strtoint(num);
            i:=1;
            achou:=0;
            // o vetor POSI vai armazenar as posições dos números
            // iguais ao número procurado,
            // pois podem existir dois ou mais números iguais
            // ao número procurado;
            // a variável POSI_LIVRE marca a posição livre do vetor
            // das posições
            posi_livre:=1;
            while (i <= 10) do
            begin
              if vet[i] = numero
                then begin
                       achou:=achou + 1;
                       posi[posi_livre]:=i;
                       posi_livre:=posi_livre+1;
                     end;
            i:=i+1;
            end;
            if achou = 0
            then showmessage(' O número '+ num+' não foi encontrado no
                             vetor')
            else if achou = 1
                  then showmessage(' O número '+ num+' foi encontrado
                                   na posição '+ inttostr(posi[1]))
                  else begin
                         texto:='';
                         for i:=1 to posi_livre-1 do
                         begin
                         texto:= texto+ inttostr(posi[i]);
                         if i <= achou - 2
                         then texto:=texto+', ';
                         if i = achou - 1
                         then texto:= texto+' e ';
```

```
                        end;
                       showmessage(' O número '+ num+' foi encontrado
                                   nas posições: '+ texto);
                     end;
          end;
     end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

As aplicações descritas anteriormente com o algoritmo de busca seqüencial para localizar um dado em um vetor não ordenado, com ou sem dados repetidos, podem ser executadas para um vetor ordenado; a única mudança que deve ocorrer é no laço das comparações do botão *Localizar*.

Na aplicação para vetor não ordenado e sem números repetidos tem-se o seguinte laço no botão *Localizar*: while (i <= 10) and (achou = 0) do, laço este que irá percorrer todas as posições do vetor ou até encontrar o dado procurado. Para executar essa aplicação em um vetor ordenado de forma crescente, por exemplo, tem-se o seguinte laço: while (i <= 10) and (achou = 0) and (vet[i] <= numero) do, laço este que irá percorrer todas as posições do vetor até encontrar o dado procurado ou até encontrar um número que seja maior que o número procurado.

Na aplicação para vetor não ordenado e com números repetidos tem-se o seguinte laço no botão *Localizar*: while (i <= 10) do, laço este que irá percorrer todas as posições do vetor. Para executar essa aplicação em um vetor ordenado de forma crescente, por exemplo, tem-se o seguinte laço: while (i <= 10) and (vet[i] <= numero) do, laço este que irá percorrer todas as posições do vetor ou até encontrar um número que seja maior que o número procurado.

Algoritmo de busca binária

O algoritmo de busca binária é utilizado para localizar um dado dentro de um vetor ordenado e sem repetições. Para iniciar o algoritmo, o dado que se deseja localizar é digitado pelo usuário e um laço com as comparações é efetuado. A seguir, o algoritmo de busca binária é ilustrado com um vetor que possui dez posições e o dado procurado é o número 9.

Execução número 1 do laço:
inicio = 1
fim = 10
posi = (inicio+ fim)/2
posi = (1+10)/2 = 11/2 = 5,5
parte inteira de posi = 5

Vet	2	4	7	9	12	15	18	20	24	30
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 5 (número 12) com o número procurado (número 9). Se forem iguais, a execução do laço será encerrada; do contrário, uma nova posição será calculada. Como o número procurado é menor que o número da posição 5, temos um novo fim.

Execução número 2 do laço:
inicio = 1
fim = posi — 1 = 5 — 1 = 4
posi = (inicio+ fim)/2
posi = (1+4)/2 = 5/2 = 2,5
parte inteira de posi = 2

Vet	2	4	7	9	12	15	18	20	24	30
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 2 (número 4) com o número procurado (número 9). Se forem iguais, a execução do laço será encerrada; do contrário, uma nova posição será calculada. Como o número procurado é maior que o número da posição 2, então temos um novo início.

Execução número 3 do laço:

inicio = posi + 1 = 2 + 1 = 3
fim = 4
posi = (inicio+ fim)/2
posi = (3+4)/2 = 7/2 = 3,5
parte inteira de posi = 3

Vet	2	4	7	9	12	15	18	20	24	30
	1	2	3	4	5	6	7	8	9	10



Compara-se o número da posição 3 (número 7) com o número procurado (número 9). Se forem iguais, a execução do laço será encerrada; do contrário, uma nova posição será calculada. Como o número procurado é maior que o número da posição 3, então temos um novo início.

Execução número 4 do laço:

inicio = posi + 1 = 3 + 1 = 4
fim = 4
posi = (inicio+ fim)/2
posi = (4+4)/2 = 8/2 = 4
parte inteira de posi = 4

Vet	2	4	7	9	12	15	18	20	24	30
	1	2	3	4	5	6	7	8	9	10

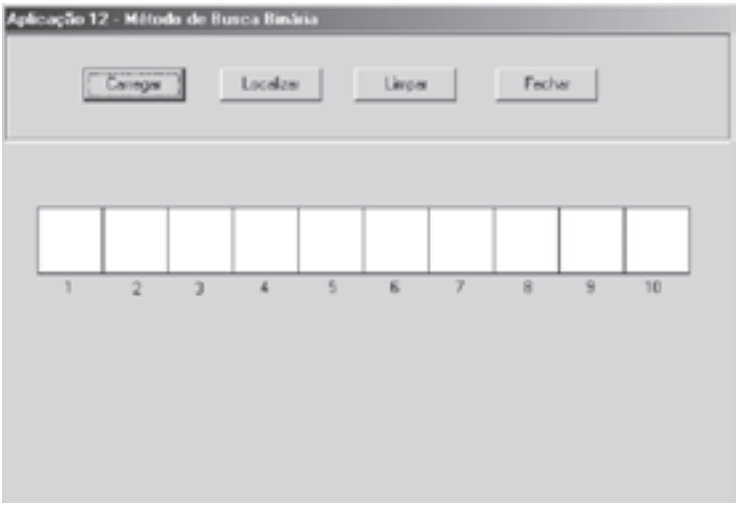


Compara-se o número da posição 4 (número 9) com o número procurado (número 9). Se forem iguais, a execução do laço será encerrada. Como o número da posição 4 é igual ao número procurado, a busca será encerrada.

As comparações serão realizadas até o número procurado ser encontrado ou até a variável FIM ser menor que a variável INICIO.

A Figura 2.3 mostra a tela da aplicação que utiliza o algoritmo de busca binária para localizar um determinado dado em um vetor ordenado e sem dados repetidos. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 2.3 Tela da aplicação que busca um dado em um vetor pelo algoritmo de busca binária



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
      // vetor com identificador VET, com 10 posições
      // para armazenar números inteiros
      vet:array[1..10] of integer;
      // a variável POSLIVRE mostra se o vetor já foi carregado ou não
      poslivre:integer;
```

O código do botão *Carregar* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var quantidade: integer;
    num, texto: string;
    numero, achou, j, i, aux:integer;
begin
// este procedimento vai carregar o vetor
// com números digitados pelo usuário;
// a variável QUANTIDADE exige a digitação de 10 números
quantidade:=1;
while quantidade <= 10 do
begin
texto:='Digite o '+inttostr(quantidade)+'° número';
num:=inputbox(texto,'Busca binária','');
if num <> ''
then begin
      numero:=strtoint(num);
      if quantidade = 1
      then begin
```

```
          pos1.visible:=true;
          pos1.caption:=num;
          vet[quantidade]:=numero;
          end;
     // verifica se o número digitado já existe no vetor,
     //  pois este não pode aceitar repetições;
     // a variável ACHOU marca se o número já existe ou não no vetor
     // ACHOU = 0 - número não existe
     // ACHOU = 1 - número existe
     achou:=0;
     for j:=1 to quantidade-1 do
     begin
     if vet[j] = numero
     then achou := 1;
     end;
     if achou = 0
     then begin
     if quantidade = 2
     then begin
          pos2.visible:=true;
          pos2.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 3
     then begin
          pos3.visible:=true;
          pos3.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 4
     then begin
          pos4.visible:=true;
          pos4.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 5
     then begin
          pos5.visible:=true;
          pos5.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 6
     then begin
          pos6.visible:=true;
          pos6.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 7
     then begin
          pos7.visible:=true;
          pos7.caption:=num;
```

```
          vet[quantidade]:=numero;
          end;
     if quantidade = 8
     then begin
          pos8.visible:=true;
          pos8.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 9
     then begin
          pos9.visible:=true;
          pos9.caption:=num;
          vet[quantidade]:=numero;
          end;
     if quantidade = 10
     then begin
          pos10.visible:=true;
          pos10.caption:=num;
          vet[quantidade]:=numero;
          end;
     quantidade:=quantidade+1;
    end
    else showmessage('Este número já existe no vetor');
    end;
end;
// a variável POSLIVRE marca que o vetor foi carregado
poslivre:=1;
// ordenação do vetor pelo algoritmo de ordenação por troca
for i:=1 to 9 do
 begin
  for j:=1 to 9 do
   begin
   if vet[j] > vet [j+1]
   then begin
        aux:=vet[j];
        vet[j]:=vet[j+1];
        vet[j+1]:=aux;
        end;
   end;
 end;
     // as linhas abaixo irão atualizar os números da tela
     // com o vetor ordenado
     for i:=1 to 10 do
     begin
     if i = 1
     then pos1.caption:=inttostr(vet[i]);
     if i = 2
     then pos2.caption:=inttostr(vet[i]);
     if i = 3
     then pos3.caption:=inttostr(vet[i]);
     if i = 4
     then pos4.caption:=inttostr(vet[i]);
```

```
     if i = 5
     then pos5.caption:=inttostr(vet[i]);
     if i = 6
     then pos6.caption:=inttostr(vet[i]);
     if i = 7
     then pos7.caption:=inttostr(vet[i]);
     if i = 8
     then pos8.caption:=inttostr(vet[i]);
     if i = 9
     then pos9.caption:=inttostr(vet[i]);
     if i = 10
     then pos10.caption:=inttostr(vet[i]);
    end;
end;
```

O código do botão *Localizar* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var i, numero, achou, posi : integer;
    inicio, fim, conta: real;
    num: string;
begin
// este procedimento vai executar o algoritmo de busca binária;
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
if poslivre = 0
then showmessage('O vetor está vazio, é necessário digitar os números
                no botão Carregar')
else begin
     num:=inputbox('Busca Binária','Digite o número a ser
                 procurado','');
     if num <> ''
      then begin
           numero:=strtoint(num);
           // a variável ACHOU é que marca se o número
           // foi encontrado ou não
           // se ACHOU = 0 então o número não foi encontrado
           // se ACHOU = 1 então o número foi encontrado
           achou:=0;
           // o laço deve procurar até a variável FIM
           // ser menor que a variável INICIO
           // ou até encontrar o número procurado;
           // se encontrar o número procurado, a busca
           // deve ser finalizada,
           // pois não existem números repetidos
           inicio:=1;
           // o vetor tem 10 posições, logo, fim = 10
           fim:=10;
           conta:=(inicio+fim)/2;
```

```
          // a função TRUNC retorna a parte inteira do número
          posi:= trunc(conta);
          while (achou = 0) and (inicio <= fim) do
          begin
            if vet[posi] = numero
              then begin
                     achou:=1;
                   end
              else if numero < vet[posi]
                   then begin
                          fim:= posi-1;
                          conta:=(inicio+fim)/2;
                          posi:= trunc(conta);
                        end
                   else begin
                          inicio:= posi+1;
                          conta:=(inicio+fim)/2;
                          posi:= trunc(conta);
                        end;
          end;
          if achou = 0
          then showmessage(' O número '+ num+' não foi encontrado no
                           vetor')
          else showmessage(' O número '+ num+' foi encontrado na
                           posição '+ inttostr(posi));
        end;
    end;
end;
```

O código do botão *Limpar* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
// a variável POSLIVRE mostra se o vetor está vazio ou não
// POSLIVRE = 0 - vetor vazio
// POSLIVRE = 1 - vetor com dados
poslivre:=0;
pos1.visible:=false;
pos2.visible:=false;
pos3.visible:=false;
pos4.visible:=false;
pos5.visible:=false;
pos6.visible:=false;
pos7.visible:=false;
pos8.visible:=false;
pos9.visible:=false;
pos10.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Unidade 3

Estruturas de dados dos tipos pilha e fila*

Este capítulo ilustra o funcionamento das estruturas de dados dos tipos pilha e fila. Em seguida, são apresentadas as implementações estática e dinâmica. Finalmente, aplicações-exemplos são documentadas.

Pilha

A pilha é uma estrutura de dados muito simples e, possivelmente, por essa razão, é a estrutura de dados mais utilizada em programação. A idéia fundamental da pilha é que todo acesso a seus elementos seja feito através do topo. Quando um elemento novo é introduzido na pilha, passa a ser o do topo; o único elemento que pode ser removido da pilha é o do topo. Isso faz com que os elementos da pilha sejam retirados na ordem inversa àquela em que foram introduzidos: o primeiro elemento que entrou é o último a sair; a sigla FILO (First In Last Out) é usada para descrever esta estratégia.

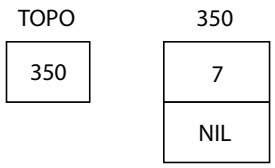
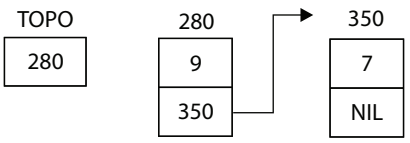
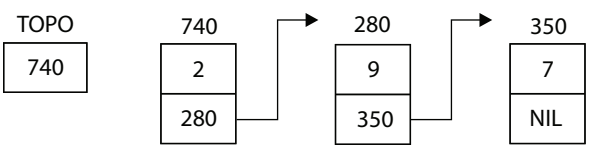
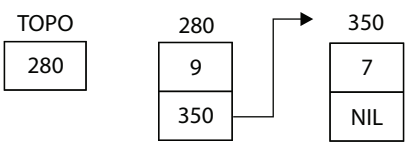
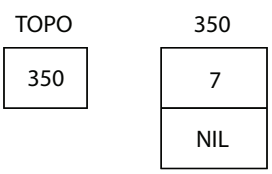
Para ilustrarmos o funcionamento da estrutura de dados do tipo pilha, podemos fazer analogia com uma pilha de pratos. Um novo prato será adicionado à pilha no seu topo; para retirarmos um prato da pilha, escolhemos o do topo, ou seja, o último prato colocado.

A estrutura de dados do tipo pilha pode ser realizada utilizando-se a implementação estática, ou seja, com vetores, ou por meio da implementação dinâmica, ou seja, componteiros.

Veja a seguir os passos na execução de uma pilha dinâmica.

Estrutura de dados do tipo pilha para armazenar um número.	350	endereço inicial de memória onde os dados serão armazenados
		espaço de memória onde o número será armazenado
		ponteiro para o próximo endereço de memória da pilha

* Esta unidade corresponde ao Capítulo 4 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005.

Inicialmente a estrutura de dados do tipo pilha está vazia, logo, o ponteiro TOPO aponta para vazio.	
Primeiro elemento inserido na pilha. Este terá o ponteiro próximo apontando para vazio e o TOPO passará a ter o valor da nova posição de memória alocada.	
Segundo elemento inserido na pilha. Este terá o ponteiro apontando para a posição de memória que era o TOPO, e o TOPO passará a ter o valor da nova posição de memória alocada.	
Terceiro elemento inserido na pilha. Este terá o ponteiro apontando para a posição de memória que era o TOPO, e o TOPO passará a ter o valor da nova posição de memória alocada.	
Um elemento será retirado da pilha, logo, o elemento do TOPO será excluído, e o TOPO passará a ter o valor do ponteiro para a próxima posição de memória alocada.	
Outro elemento será retirado da pilha, logo, o elemento do TOPO será excluído, e o TOPO passará a ter o valor do ponteiro para a próxima posição de memória alocada.	
Outro elemento será retirado da pilha, logo, o elemento do TOPO será excluído, e o TOPO passará a ter o valor do ponteiro para a próxima posição de memória alocada.	

Implementação estática de uma pilha

A Figura 3.1 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo pilha: insere na pilha (empilha), retira da pilha (desempilha), consulta o topo da pilha, consulta toda a pilha e esvazia a pilha. Todas essas operações serão realizadas pela implementação estática, ou seja, utilizando vetores; nesta aplicação, a quantidade de dados que podem ser empilhados limita-se ao tamanho do vetor. Após a Figura 3.1, têm-se os seus respectivos códigos comentados.

Figura 3.1 Tela da aplicação com implementação estática de uma pilha



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador PILHA, com 5 posições
     //que armazenarão números inteiros
     pilha: array[1..5] of integer;
     // variável com identificador TOPO que será utilizada
     // para controlar o topo da pilha
     topo: integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
topo:=0;
edit1.text:=inttostr(topo);
end;
```

O código do botão *Inserir na Pilha* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
if topo = 5
then showmessage('A pilha está cheia')
else begin
     num:=inputbox('Pilha Estática','Digite o número a ser inserido na
                   pilha','');
```

```
     if num <> ''
     then begin
            numero:=strtoint(num);
            topo:=topo+1;
            pilha[topo]:=numero;
            edit1.text:=inttostr(topo);
            showmessage('O número '+ num+ ' foi inserido com sucesso
                        na pilha');
          end;
     end;
end;
```

O código do botão *Retirar da Pilha* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero: integer;
begin
if topo = 0
then showmessage('A pilha está vazia')
else begin
     numero:= pilha[topo];
     topo:=topo-1;
     edit1.text:=inttostr(topo);
     showmessage('O número '+ inttostr(numero)+ ', que estava no topo
                 da pilha, foi retirado com sucesso');
     end;
end;
```

O código do botão *Consultar o Topo da Pilha* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
begin
if topo = 0
then showmessage('A pilha está vazia')
else showmessage('O número do topo da pilha é o número:
                 '+inttostr(pilha[topo]));
end;
```

O código do botão *Consultar Toda a Pilha* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if topo = 0
then showmessage('A pilha está vazia')
else begin
      texto:='';
      for i:= topo downto 1 do
```

```
      begin
        texto:=texto+ '   '+inttostr(pilha[i]);
      end;
      showmessage('A pilha toda: '+texto);
     end;
end;
```

O código do botão *Esvaziar a Pilha* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if topo = 0
then showmessage('A pilha já está vazia')
else begin
      topo:=0;
      edit1.text:=inttostr(topo);
      showmessage('A pilha foi esvaziada com sucesso');
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação dinâmica de uma pilha

A Figura 3.2 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo pilha: insere na pilha (empilha), retira da pilha (desempilha), consulta o topo da pilha, consulta toda a pilha e esvazia a pilha. Todas essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando ponteiros; nesta aplicação, a quantidade de dados que podem ser empilhados limita-se ao tamanho da memória. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 3.2 Tela da aplicação com implementação dinâmica de uma pilha



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da pilha;
// a variável NUM armazenará um número inteiro e
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo: pointer;
     end;

// as variáveis PILHA, TOPO e AUX são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável QTDE armazena a quantidade de números
// armazenados na pilha

var Form1: TForm1;
    pilha, topo, aux : ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
// a variável QTDE armazenará a quantidade de números
// armazenados na pilha
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro TOPO aponta para NIL, ou seja, para vazio,
```

```
// pois a pilha ainda está vazia
topo:= NIL;
end;
```

O código do botão *Inserir na Pilha* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
 // a variável NUM receberá o número que será armazenado na pilha
 num:=inputbox('Pilha Dinâmica','Digite o número a ser inserido na
              pilha','');
 if num <> ''
 then begin
      numero:=strtoint(num);
      // um novo endereço de memória será alocado
      // para armazenar o número digitado
      new(pilha);
      // no endereço criado, o campo NUM armazenará o número digitado
      pilha^.num:=numero;
      // e o campo PROXIMO, que é um apontador,
      // armazenará o endereço do próximo endereço da pilha
      pilha^.proximo:=topo;
      // o apontador TOPO armazena o endereço de
      // memória do último número inserido,
      // pois este será o primeiro a ser acessado
      // (FILO - First In Last Out)
      topo:= pilha;
      // a variável QTDE será atualizada com a quantidade
      // de números armazenados na pilha
      qtde:=qtde+1;
      edit1.text:=inttostr(qtde);
      showmessage('O número '+ num+ ' foi inserido com sucesso na
                  pilha');
      end;
end;
```

O código do botão *Retirar da Pilha* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var num : integer;
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('A pilha está vazia')
  else begin
       // a variável NUM receberá o número que
       // está armazenado no endereço
       // do topo da pilha, pois este será o número excluído
       num:=topo^.num;
```

```
        // o apontador AUX será atualizado com o endereço
        // do próximo endereço
        // de memória alocado para a pilha
        aux:=topo^.proximo;
        // o endereço de memória do topo será desalocado
        dispose(topo);
        // o apontador TOPO será atualizado
        // com o endereço do apontador AUX
        topo:=aux;
        // a variável QTDE será atualizada com a quantidade
        // de números armazenados na pilha
        qtde:=qtde-1;
        edit1.text:=inttostr(qtde);
        showmessage('O número '+ inttostr(num) +', que estava no topo
                    da pilha, foi retirado com sucesso');
      end;
end;
```

O código do botão *Consultar o Topo da Pilha* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('A pilha está vazia')
  else showmessage('O número do topo da pilha é o número: '
                  +inttostr(topo^.num));
end;
```

O código do botão *Consultar Toda a Pilha* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('A pilha está vazia')
  else begin
       texto:='';
       // o ponteiro AUX receberá o endereço do topo
       aux:= topo;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado
       while aux <> nil do
       begin
        texto:=texto+ '   '+inttostr(aux^.num);
        aux:=aux^.proximo;
      end;
      showmessage('A pilha toda: '+texto);
     end;
end;
```

O código do botão *Esvaziar a Pilha* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('A pilha está vazia')
  else begin
       // o ponteiro AUX receberá o endereço do topo
       aux:= topo;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado
       while aux <> nil do
       begin
        // o apontador TOPO será atualizado com o próximo endereço
        topo:=topo^.proximo;
        // o endereço AUX será desalocado da memória
        dispose(aux);
        //o apontador AUX será atualizado com o novo TOPO
        aux:=topo;
      end;
      // a variável QTDE será atualizada
      qtde:=0;
      edit1.text:=inttostr(qtde);
      showmessage('A pilha foi esvaziada com sucesso');
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Aplicação de controle de estoque usando pilha dinâmica

A Figura 3.3 mostra a tela da aplicação que implementa um controle de estoque por meio de uma estrutura de dados do tipo pilha com implementação dinâmica. Nesta aplicação serão cadastradas operações de compra, venda, perda ou devolução de produtos; para cada operação serão registrados o código da operação, o código do produto, a quantidade e a data da operação. Após a Figura 3.3, têm-se os seus respectivos códigos comentados.

Figura 3.3 Tela da aplicação de controle de estoque utilizando pilha dinâmica



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da pilha;
// a variável COD_OP armazenará o código da operação;
// a variável COD_PROD armazenará o código do produto;
// a variável QTDE armazenará a quantidade de produtos
// envolvidos em cada operação;
// a variável DATA armazenará a data da operação;
// a variável PROXIMO é um ponteiro para um endereço de memória do
// tipo REGISTRO

type registro = record
      cod_op, cod_prod, qtde: integer;
      data: tdate;
      proximo: pointer;
     end;

// as variáveis PILHA, TOPO e AUX são ponteiros para endereços
// de memória do tipo REGISTRO
// definido anteriormente

var Form1: TForm1;
    pilha, topo, aux : ^registro;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

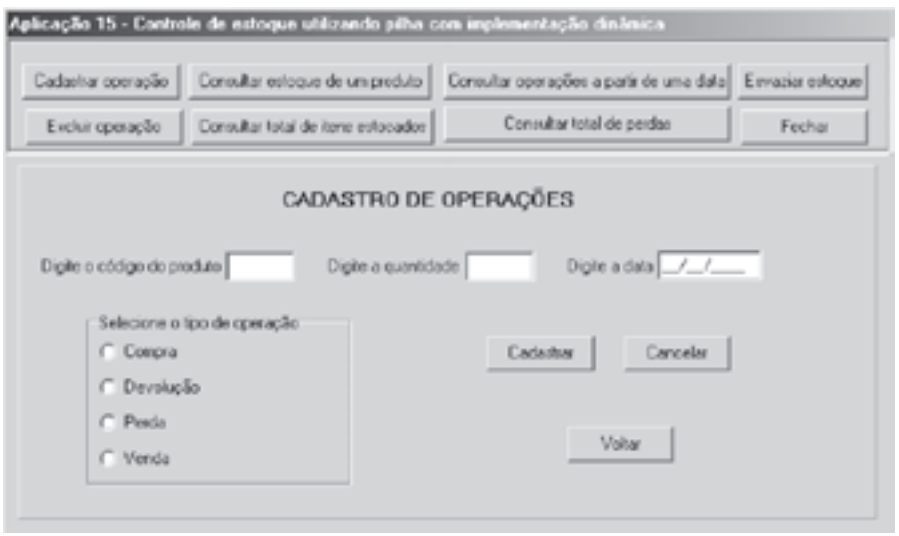
```
procedure TForm1.FormActivate(Sender: TObject);
begin
// o ponteiro TOPO aponta para NIL, ou seja, para vazio,
// pois a pilha ainda está vazia
topo:= NIL;
end;
```

O código do botão *Cadastrar Operação* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
begin
 // desabilita o painel que possui os botões
 // das operações da aplicação
 panel1.enabled:=false;
 // exibe o painel que possui os componentes de cadastro
 // de uma operação
 panel3.visible:=true;
 // limpa os componentes do painel de cadastro de uma operação
 edit1.clear;
 edit2.clear;
 maskedit1.clear;
 radiogroup1.itemIndex:=-1;
 edit1.setfocus;
end;
```

Quando o botão *Cadastrar Operação* é clicado, a tela da Figura 3.4 é apresentada.

Figura 3.4 Tela do cadastro de operações da aplicação de controle de estoque



O código do botão *Cadastrar* da tela de cadastro de operações é apresentado a seguir.

```
procedure TForm1.Button9Click(Sender: TObject);
var qt: integer;
begin
if edit1.text = ''
then begin
```

```
      showmessage('É necessário digitar o código do produto');
      edit1.setfocus;
     end
else if edit2.text= ''
      then begin
           showmessage('É necessário digitar a quantidade do
                       produto');
           edit2.setfocus;
          end
     else if maskedit1.text = '  /  /    '
           then begin
                showmessage('É necessário digitar a data da
                            operação');
                maskedit1.setfocus;
               end
           else if radiogroup1.itemindex = -1
                then showmessage('É necessário selecionar o tipo da
                                  operação')
                else begin
                     // as operações de perda (30) ou venda (40)
                     // só podem ser cadastradas se existir
                     // estoque suficiente
                     if (radiogroup1.itemindex = 2) or
                        (radiogroup1.itemindex = 3)
                     then begin
                          // o ponteiro AUX receberá
                          // o endereço do TOPO
                          aux:= topo;
                          // enquanto existir um próximo endereço,
                          // o laço abaixo será executado.
                          // a variável QT armazenará a
                          // quantidade estocada do produto
                          qt:=0;
                          while aux <> nil do
                            begin
                            if aux^.cod_prod = strtoint(edit1.text)
                            then begin
                                 if (aux^.cod_op = 10) or
                                    (aux^.cod_op = 20)
                                 then qt:=qt+aux^.qtde;
                                 if (aux^.cod_op = 30) or
                                    (aux^.cod_op = 40)
                                 then qt:=qt-aux^.qtde;
                                end;
                            aux:=aux^.proximo;
                            end;
                         end;
                     if ((radiogroup1.itemindex = 2) or
                         (radiogroup1.itemindex = 3))
                     and (qt < strtoint(edit2.text))
                     then begin
                           showmessage('Não existe estoque
```

```
                                        suficiente para esta
                                        operação');
                          end
                     else begin
                          // um novo endereço de memória
                          // será alocado
                          // para armazenar os dados
                          // da nova operação
                             new(pilha);
                          // no endereço criado, o campo COD_OP
                          // armazenará o código da operação
                          // código 10 - compra
                          // código 20 - devolução
                          // código 30 - perda
                          // código 40 - venda
                          if radiogroup1.ItemIndex = 0
                          then pilha^.cod_op:=10;
                          if radiogroup1.ItemIndex = 1
                          then pilha^.cod_op:=20;
                          if radiogroup1.ItemIndex = 2
                          then pilha^.cod_op:=30;
                          if radiogroup1.ItemIndex = 3
                          then pilha^.cod_op:=40;
                          // no endereço criado, o campo COD_PROD
                          // armazenará o código do produto
                          pilha^.cod_prod:=strtoint(edit1.text);
                          // no endereço criado, o campo QTDE
                          // armazenará a quantidade do produto
                          pilha^.qtde:=strtoint(edit2.text);
                          // no endereço criado, o campo DATA
                          // armazenará a data da operação
                          pilha^.data:=strtodate(maskedit1.text);
                          // e o campo PROXIMO, que é um apontador,
                          // armazenará o endereço
                          // do próximo endereço da pilha
                          pilha^.proximo:=topo;
                          // o apontador TOPO armazena o
                          // endereço de memória
                          // do último número inserido,
                          // pois este será o primeiro
                          // a ser acessado
                          //(FILO - First In Last Out)
                          topo:= pilha;
                          showmessage('Operação cadastrada com
                                      sucesso');
                          edit1.clear;
                          edit2.clear;
                          maskedit1.clear;
                          radiogroup1.itemIndex:=-1;
                          edit1.setfocus;
                          end;
      end;
end;
```

O código do botão *Cancelar* da tela de cadastro de operações é apresentado a seguir.

```
procedure TForm1.Button10Click(Sender: TObject);
begin
edit1.clear;
edit2.clear;
maskedit1.clear;
radiogroup1.itemIndex:=-1;
edit1.setfocus;
end;
```

O código do botão *Voltar* da tela de cadastro de operações é apresentado a seguir.

```
procedure TForm1.Button11Click(Sender: TObject);
begin
// esconde o painel que possui os componentes de cadastro
// de uma operação
panel3.visible:=false;
// habilita o painel que possui os botões das operações da aplicação
panel1.enabled:=true;
end;
```

O código do botão *Excluir Operação* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('Não há operações cadastradas')
  else begin
       // o apontador AUX será atualizado com o
       // endereço do próximo endereço
       // de memória alocado para a pilha
       aux:=topo^.proximo;
       // o endereço de memória do topo será desalocado
       dispose(topo);
       // o apontador TOPO será atualizado
       // com o endereço do apontador AUX
       topo:=aux;
       showmessage('A última operação cadastrada foi excluída com
                   sucesso');
     end;
end;
```

O código do botão *Consultar Estoque de um Produto* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var cod: string;
    codigo, qt: integer;
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('Não há operações cadastradas')
  else begin
       cod:=inputbox('Aplicação 15','Digite o código do produto a ser
                     consultado','');
       if cod <> ''
       then begin
            codigo:=strtoint(cod);
            // o ponteiro AUX receberá o endereço do TOPO
            aux:= topo;
            // enquanto existir um próximo endereço,
            // o laço abaixo será executado.
            // a variável QT armazenará a
            // quantidade estocada do produto
            qt:=0;
            while aux <> nil do
            begin
              if aux^.cod_prod = codigo
              then begin
                   if (aux^.cod_op = 10) or (aux^.cod_op = 20)
                   then qt:=qt+aux^.qtde;
                   if (aux^.cod_op = 30) or (aux^.cod_op = 40)
                   then qt:=qt-aux^.qtde;
                   end;
            aux:=aux^.proximo;
            end;
            if qt = 1
            then showmessage('A quantidade estocada é de '+
                             inttostr(qt)+' unidade')
            else showmessage('A quantidade estocada é de '+
                            inttostr(qt)+' unidades');
            end;
     end;
end;
```

O código do botão *Consultar Total de Itens Estocados* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var qt: integer;
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
```

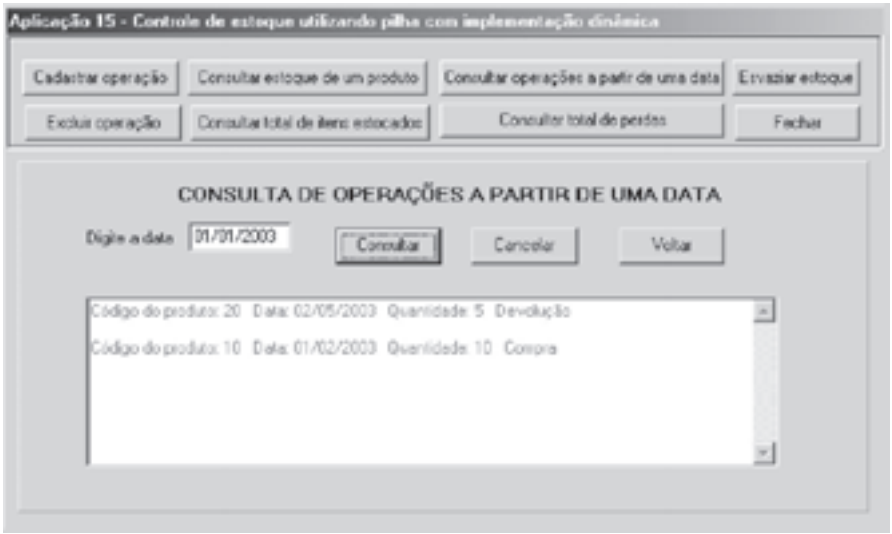
```
  then showmessage('Não há operações cadastradas')
  else begin
       // o ponteiro AUX receberá o endereço do TOPO
       aux:= topo;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado.
       // a variável QT armazenará a quantidade estocada do produto
       qt:=0;
       while aux <> nil do
       begin
        if (aux^.cod_op = 10) or (aux^.cod_op = 20)
        then qt:=qt+aux^.qtde;
        if (aux^.cod_op = 30) or (aux^.cod_op = 40)
        then qt:=qt-aux^.qtde;
       aux:=aux^.proximo;
       end;
       if qt = 1
       then showmessage('A quantidade estocada é de '+ inttostr(qt)+'
                        unidade')
       else showmessage('A quantidade estocada é de '+ inttostr(qt)+'
                        unidades');
      end;
end;
```

O código do botão *Consultar Operações a Partir de uma Data* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('Não há operações cadastradas')
  else begin
        // desabilita o painel que possui os botões
        // das operações da aplicação
        panel1.enabled:=false;
        // exibe o painel que possui os componentes de consulta
        // de operações a partir de uma data
        panel4.visible:=true;
        // limpa os componentes do painel de consulta
        // de operações a partir de uma data
        maskedit2.clear;
        memo1.clear;
        maskedit2.setfocus;
       end;
end;
```

Quando o botão *Consultar Operações a Partir de uma Data* é clicado, a tela da Figura 3.5 é apresentada.

Figura 3.5 Tela da consulta de operações a partir de uma data da aplicação de controle de estoque



O código do botão *Consultar* é apresentado a seguir.

```
procedure TForm1.Button12Click(Sender: TObject);
var d:tdate;
    texto:string;
begin
if maskedit2.text = '  /  /    '
then begin
     showmessage('Digite a data a ser consultada');
     end
else begin
     d:=strtodate(maskedit2.Text);
     // o ponteiro AUX receberá o endereço do TOPO
     aux:= topo;
     // enquanto existir um próximo endereço,
     // o laço abaixo será executado
     while aux <> nil do
      begin
      if aux^.data >= d
      then begin
           texto:='';
           texto:='Código do produto: '+ inttostr(aux^.cod_prod);
           texto:=texto + '   Data: '+ datetostr(aux^.data);
           texto:=texto + '   Quantidade: '+
           inttostr(aux^.qtde);
           if aux^.cod_op = 10
```

```
          then texto:= texto +'   Compra';
          if aux^.cod_op = 20
          then texto:= texto +'   Devolução';
          if aux^.cod_op = 30
          then texto:= texto +'   Perda';
          if aux^.cod_op = 40
          then texto:= texto +'   Venda';
          memo1.lines.add(texto);
          memo1.lines.add('');
          end;
      aux:=aux^.proximo;
      end;
      end;
end;
```

O código do botão *Cancelar* é apresentado a seguir.

```
procedure TForm1.Button13Click(Sender: TObject);
begin
maskedit2.clear;
memo1.clear;
maskedit2.setfocus;
end;
```

O código do botão *Voltar* é apresentado a seguir.

```
procedure TForm1.Button14Click(Sender: TObject);
begin
 // esconde o painel que possui os componentes de consulta
 // de operações a partir de uma data
 panel4.visible:=false;
 // habilita o painel que possui os botões das operações da aplicação
 panel1.enabled:=true;
end;
```

O código do botão *Consultar Total de Perdas* é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
var qt: integer;
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('Não há operações cadastradas')
  else begin
       // o ponteiro AUX receberá o endereço do TOPO
       aux:= topo;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado.
       // a variável QT armazenará a quantidade de perdas
```

```
      qt:=0;
      while aux <> nil do
      begin
       if (aux^.cod_op = 30)
       then qt:=qt+aux^.qtde;
      aux:=aux^.proximo;
      end;
      if qt = 1
      then showmessage('A quantidade de perdas é de '+ inttostr(qt)+'
                       unidade')
      else showmessage('A quantidade de perdas é de '+ inttostr(qt)+'
                       unidades');
     end;
end;
```

O código do botão *Esvaziar Estoque* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
// o ponteiro TOPO aponta para o endereço do topo da pilha
if topo = nil
  then showmessage('Não há operações cadastradas')
  else begin
       // o ponteiro AUX receberá o endereço do topo
       aux:= topo;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado
       while aux <> nil do
       begin
        // o apontador TOPO será atualizado com o próximo endereço
        topo:=topo^.proximo;
        // o endereço AUX será desalocado da memória
        dispose(aux);
        //o apontador AUX será atualizado com o novo TOPO
        aux:=topo;
      end;
      showmessage('O estoque foi esvaziado com sucesso');
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

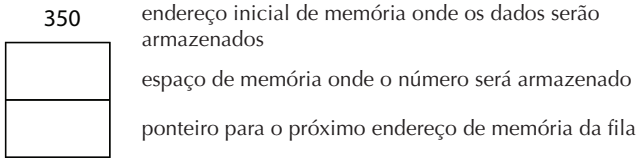
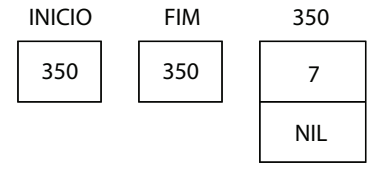
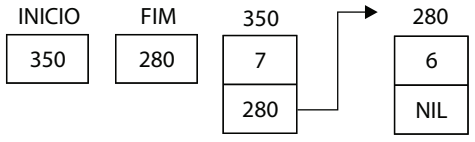
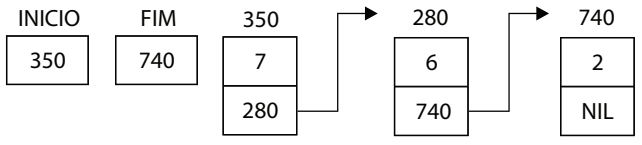
```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

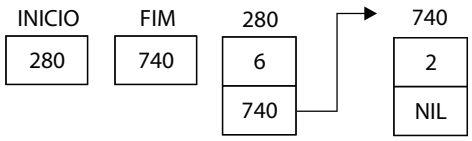
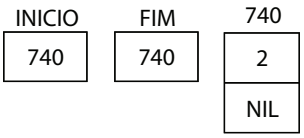
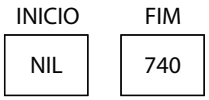
Fila

A fila é uma estrutura de dados simples e muito utilizada em programação. A idéia fundamental é que o acesso a seus elementos seja feito pelo seu topo; eles são acessados na ordem em que são inseridos. Assim, quando um elemento novo é introduzido na fila, este será colocado no final, e, quando é removido da fila, este será o primeiro elemento inserido; ou seja, o primeiro elemento que entrou é o primeiro a sair. A sigla FIFO — First In First Out — é usada para descrever esta estratégia.

Para ilustrarmos o funcionamento da estrutura de dados do tipo fila, podemos fazer analogia com uma fila de espera em um banco. Um novo cliente que chega será colocado no fim da fila de espera, e o próximo a ser atendido será sempre o primeiro cliente da fila.

A estrutura de dados do tipo fila pode ser realizada utilizando-se a implementação estática, ou seja, com vetores, ou por meio da implementação dinâmica, ou seja, com ponteiros. A tabela a seguir mostra os passos na execução de uma fila dinâmica.

Estrutura de dados do tipo fila para armazenar um número.	
Inicialmente a estrutura de dados do tipo fila está vazia, logo, o ponteiro INICIO aponta para vazio.	
Primeiro elemento inserido na fila. Este terá o ponteiro próximo apontando para vazio e, por ser o primeiro elemento, os ponteiros INICIO e FIM passam a apontar para a nova posição de memória alocada.	
Segundo elemento inserido na fila. O ponteiro do elemento FIM passará a ter o valor da nova posição de memória e o FIM passará a ter o valor da nova posição de memória alocada.	
Terceiro elemento inserido na fila. O ponteiro do elemento FIM passará a ter o valor da nova posição de memória e o FIM passará a ter o valor da nova posição de memória alocada.	

Um elemento será retirado da fila, logo, o elemento do INICIO será excluído e o INICIO passa a ter o valor do ponteiro para a próxima posição de memória alocada.	
Outro elemento será retirado da fila, logo, o elemento do INICIO será excluído e o INICIO passará a ter o valor do ponteiro para a próxima posição de memória alocada.	
Outro elemento será retirado da fila, logo, o elemento do INICIO será excluído e o INICIO passará a ter o valor do ponteiro para a próxima posição de memória alocada.	

Implementação estática de uma fila

A Figura 3.6 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo fila: insere na fila, retira da fila, consulta o topo da fila, consulta toda a fila e esvazia a fila. Todas essas operações serão realizadas por meio da implementação estática, ou seja, utilizando-se vetores; logo, nesta aplicação, a quantidade de dados que podem ser colocados na fila limita-se ao tamanho do vetor. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 3.6 Tela da aplicação com implementação estática de uma fila



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
    // vetor com identificador FILA, com 5 posições
    // que armazenarão números inteiros
    fila: array[1..5] of integer;
    // a variável QTDE identifica a quantidade de números
    // já inseridos na fila
    qtde: integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde:=0;
edit1.text:=inttostr(qtde);
end;
```

O código do botão *Inserir na Fila* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
if qtde = 5
then showmessage('A fila está cheia')
else begin
     num:=inputbox('Fila Estática','Digite o número a ser inserido na
                   fila','');
     if num <> ''
     then begin
            numero:=strtoint(num);
            qtde:=qtde+1;
            fila[qtde]:=numero;
            edit1.text:=inttostr(qtde);
            showmessage('O número '+ num+ ' foi inserido com sucesso
                        na fila');
          end;
     end;
end;
```

O código do botão *Retirar da Fila* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, i: integer;
begin
if qtde = 0
then showmessage('A fila está vazia')
else begin
     numero:=fila[1];
     // será necessário deslocar os elementos dentro do vetor
```

```
    // porque o número a ser excluído está no início da fila
    for i:=1 to qtde-1 do
    begin
    fila[i]:=fila[i+1];
    end;
    qtde:=qtde-1;
    edit1.text:=inttostr(qtde);
    showmessage('O número '+ inttostr(numero)+ ', que estava no
                início da fila, foi retirado com sucesso');
    end;
end;
```

O código do botão *Consultar o Topo da Fila* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
begin
if qtde = 0
then showmessage('A fila está vazia')
else showmessage('O número do início da fila é o número: '
                +inttostr(fila[1]));
end;
```

O código do botão *Consultar Toda a Fila* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if qtde = 0
then showmessage('A fila está vazia')
else begin
     texto:='';
     for i:= 1 to qtde do
     begin
       texto:=texto+ '   '+inttostr(fila[i]);
     end;
     showmessage('A fila toda: '+texto);
    end;
end;
```

O código do botão *Esvaziar a Fila* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if qtde = 0
then showmessage('A fila já está vazia')
else begin
     qtde:=0;
     edit1.text:=inttostr(qtde);
     showmessage('A fila foi esvaziada com sucesso');
    end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação dinâmica de uma fila

A Figura 3.7 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo fila: insere na fila, retira da fila, consulta o topo da fila, consulta toda a fila e esvazia a fila. Todas essas operações serão realizadas por meio da implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação, a quantidade de dados que podem ser colocados na fila limita-se ao tamanho da memória. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 3.7 Tela da aplicação com implementação dinâmica de uma fila



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da fila;
// a variável NUM armazenará um número inteiro,
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo: pointer;
     end;
```

```
// as variáveis FILA, INICIO, FIM e AUX são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável QTDE armazena a quantidade de números
// armazenados na fila
var Form1: TForm1;
    fila, inicio, fim, aux: ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
// a variável QTDE armazenará a quantidade de números
// armazenados na fila
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro INICIO aponta para NIL, ou seja, para vazio,
// pois a fila ainda está vazia
inicio:= NIL;
end;
```

O código do botão *Inserir na Fila* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
// a variável NUM receberá o número que será armazenado na fila
 num:=inputbox('Fila Dinâmica','Digite o número a ser inserido na
               fila','');
 if num <> ''
 then begin
      numero:=strtoint(num);
      // um novo endereço de memória será alocado
      // para armazenar o número digitado
      new(fila);
      // no endereço criado, o campo NUM armazenará o número digitado
      fila^.num:=numero;
      // e o campo PROXIMO, que é um apontador, armazenará NIL,
      // ou seja, vazio, pois cada novo elemento é inserido
      // no final da fila, logo, não aponta para nenhum endereço
      fila^.proximo:=nil;
      // o apontador INICIO armazena o endereço
      // de memória do primeiro número inserido,
      // pois este será o primeiro a ser
      // acessado (FIFO —— First In First Out)
      if inicio = nil
      then inicio:= fila
      else fim^.proximo:=fila;
```

```
       // atualizando o apontador FIM com o último endereço criado
       fim:=fila;
       // a variável QTDE será atualizada com a quantidade
       // de números armazenados na fila
       qtde:=qtde+1;
       edit1.text:=inttostr(qtde);
       showmessage('O número '+ num+ ' foi inserido com sucesso na
                   fila');
     end;
end;
```

O código do botão *Retirar da Fila* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var num : integer;
begin
// o ponteiro INICIO aponta para o endereço do topo da fila
if INICIO = nil
  then showmessage('A fila está vazia')
  else begin
       // a variável NUM receberá o número que está armazenado
       // no endereço do início da fila, pois este será o número
       // a ser excluído, ou seja, o primeiro número inserido
       // será o excluído
       num:=inicio^.num;
       // o apontador AUX será atualizado com o endereço do próximo
       // endereço de memória alocado para a fila
       aux:=inicio^.proximo;
       // o endereço de memória do INICIO será desalocado
       dispose(inicio);
       // o apontador INICIO será atualizado
       // com o endereço do apontador AUX
       inicio:=aux;
       // a variável QTDE será atualizada com a
       // quantidade de números armazenados na fila
       qtde:=qtde-1;
       edit1.text:=inttostr(qtde);
       showmessage('O número '+ inttostr(num) +', que estava no início
                    da fila, foi retirado com sucesso');
     end;
end;
```

O código do botão *Consultar o Topo da Fila* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
  then showmessage('A fila está vazia')
  else showmessage('O número do início da fila é o número: '
                  +inttostr(inicio^.num));
end;
```

O código do botão *Consultar Toda a Fila* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
  then showmessage('A fila está vazia')
  else begin
       texto:='';
       // o ponteiro AUX receberá o endereço do início da fila
       aux:= inicio;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado.
       while aux <> nil do
       begin
        texto:=texto+ '   '+inttostr(aux^.num);
        aux:=aux^.proximo;
      end;
      showmessage('A fila toda: '+texto);
     end;
end;
```

O código do botão *Esvaziar a Fila* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
  then showmessage('A fila está vazia')
  else begin
       // o ponteiro AUX receberá o endereço do INICIO
       aux:= inicio;
       // enquanto existir um próximo endereço,
       // o laço abaixo será executado.
       while aux <> nil do
       begin
        // o apontador INICIO será atualizado com o próximo endereço
        inicio:=inicio^.proximo;
        // o endereço AUX será desalocado da memória
        dispose(aux);
        //o apontador AUX será atualizado com o novo INICIO
        aux:=inicio;
      end;
      // a variável QTDE será atualizada
      qtde:=0;
      edit1.text:=inttostr(qtde);
      showmessage('A fila foi esvaziada com sucesso');
     end;
end;
```

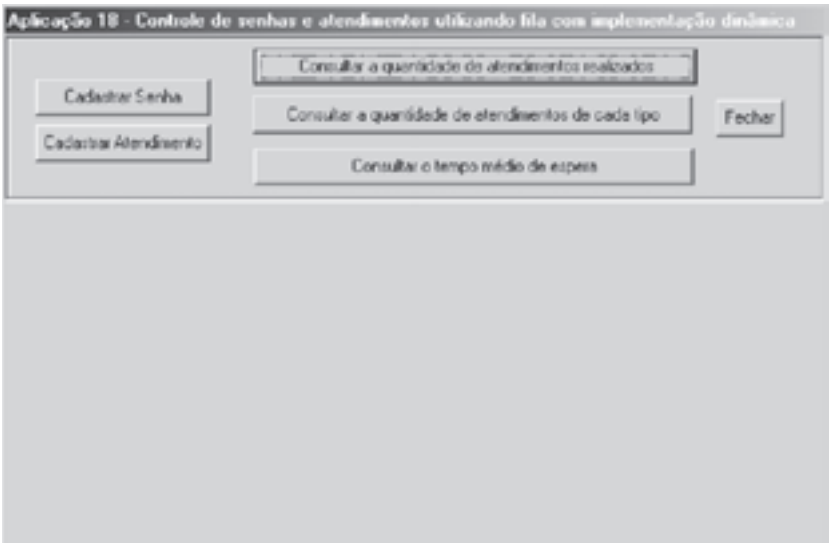
O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Aplicação de controle de senhas usando fila dinâmica

A Figura 3.8 mostra a tela da aplicação que implementa um controle de senhas de uma fila de atendimento utilizando uma estrutura de dados do tipo fila com implementação dinâmica. Nesta aplicação serão cadastradas novas senhas, geradas automaticamente, e atendimentos com hora e tipo de atendimento. Entre algumas consultas implementadas estão: quantidade de atendimentos realizados, quantidade de atendimentos separados por tipo e o tempo médio de espera no atendimento. Após a figura, têm-se seus respectivos códigos comentados.

Figura 3.8 Tela da aplicação de controle de senhas utilizando fila dinâmica



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da fila;
// a variável SENHA armazenará o número da senha
// que será gerado automaticamente;
// a variável HORA_SENHA armazenará a hora
// em que a senha foi cadastrada;
```

```
// a variável HORA_ATENDIMENTO armazenará a hora
// em que o atendimento foi cadastrado;
// a variável CODIGO armazenará o código do atendimento,
// e estes códigos são: 10 ou 20;
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO

type registro = record
      senha, codigo: integer;
      hora_senha, hora_atendimento: ttime;
      proximo: pointer;
     end;

// as variáveis FILA, INICIO, FIM, AUX, PROX_ATEND são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável SENHA_GERADA armazenará a senha gerada automaticamente

var Form1: TForm1;
    fila, inicio, fim, aux, prox_atend: ^registro;
    senha_gerada: integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
// o ponteiro INICIO aponta para NIL, ou seja, para vazio,
// pois a fila ainda está vazia
inicio:= NIL;
// o ponteiro PROX_ATEND aponta para NIL, ou seja, para vazio,
// pois a fila ainda está vazia
prox_atend:= NIL;
// a primeira senha será sempre a de número 1
senha_gerada:=1;
end;
```

O código do botão *Cadastrar Senha* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
begin
// um novo endereço de memória será alocado
// para armazenar a senha gerada
new(fila);
// no endereço criado, o campo SENHA armazenará a senha gerada,
fila^.senha:=senha_gerada;
// a hora do sistema será armazenada no campo HORA_SENHA
fila^.hora_senha:=time;
// e o campo PROXIMO, que é um apontador, armazenará NIL
// ou seja, vazio, pois cada novo elemento é inserido
// no final da fila, logo, não aponta para nenhum endereço
```

```
fila^.proximo:=nil;
// o apontador INICIO armazena o endereço
// de memória da primeira senha gerada,
// pois este será o primeiro a ser
// acessado (FIFO —— First In First Out)
// o apontador PROX ATEND também aponta para o primeiro endereço
// armazenado, pois os atendimentos começam do primeiro
// da fila
if inicio = nil
then begin
      inicio:= fila;
      prox_atend:=fila;
     end
else fim^.proximo:=fila;
// se o ponteiro PROX ATEND estiver valendo NIL,
// então a nova senha será o próximo atendimento
if prox atend = nil
then prox atend:=fila;
// atualizando o apontador FIM com o último endereço criado
fim:=fila;
showmessage('Senha cadastrada com sucesso');
// a próxima senha será gerada abaixo
senha gerada:=senha gerada+1;
end;
```

O código do botão *Cadastrar Atendimento* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
  then showmessage('Não há senhas cadastradas')
  else if prox_atend = nil
        then showmessage('Não há senhas na espera')
        else begin
              // desabilita o painel que possui
              // os botões das operações da aplicação
              panel1.enabled:=false;
              // exibe o painel que possui os componentes do cadastro
              // de atendimento
              panel3.visible:=true;
              // limpa os componentes do painel
              // de cadastro de atendimento
              combobox1.itemindex:=-1;
              combobox1.setfocus;
             end;
end;
```

Quando o botão *Cadastrar Atendimento* é clicado, a tela da Figura 3.9 é apresentada.

Figura 3.9 Tela de cadastro de atendimento da aplicação de controle de senhas



O código do botão *Cadastrar* da tela de cadastro de atendimento é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
begin
if combobox1.itemindex = -1
 then showmessage('É necessário selecionar o tipo do atendimento')
 else begin
      // grava a hora do atendimento
      prox_atend^.hora_atendimento:=time;
      // grava o código do atendimento
      if combobox1.ItemIndex = 0
      then prox_atend^.codigo:=10
      else prox_atend^.codigo:=20;
      // atualiza o próximo a ser atendido
      prox_atend:=prox_atend^.proximo;
      showmessage('Atendimento cadastrado com sucesso');
      // esconde o painel que possui os componentes
      // de cadastro de atendimento
      panel3.Visible:=false;
      // habilita o painel que possui os botões
      // das operações da aplicação
      panel1.enabled:=true;
      end;
end;
```

O código do botão *Cancelar* da tela de cadastro de atendimento é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
begin
// desmarca o tipo do atendimento
combobox1.itemindex:=-1;
// coloca o cursor no campo que seleciona o tipo de atendimento
combobox1.setfocus;
end;
```

O código do botão *Voltar* da tela de cadastro de atendimento é apresentado a seguir.

```
procedure TForm1.Button9Click(Sender: TObject);
begin
// esconde o painel que possui os componentes de cadastro de atendimento
panel3.visible:=false;
// habilita o painel que possui os botões das operações da aplicação
panel1.enabled:=true;
end;
```

O código do botão *Consultar a Quantidade de Atendimentos Realizados* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var qtde:integer;
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
then showmessage('Não há senhas cadastradas')
else begin
       // a variável QTDE armazenará a quantidade
       // de atendimentos realizados
       qtde:=0;
       // o ponteiro AUX receberá o endereço do primeiro
       // endereço de memória da FILA
       aux:=inicio;
       // enquanto o ponteiro AUX for diferente do
       // ponteiro PROX_ATEND, o laço abaixo será executado
       // ou seja, enquanto não encontrar o endereço
       // do próximo atendimento, executará o laço
       while aux <> prox_atend do
       begin
         qtde:=qtde+1;
         // avança para o próximo endereço da fila
         aux:=aux^.proximo;
       end;
       showmessage('Quantidade de atendimentos = '+inttostr(qtde));
     end;
end;
```

O código do botão *Consultar a Quantidade de Atendimentos Realizados de cada tipo* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
var qtde1,qtde2:integer;
begin
// o ponteiro INICIO aponta para o endereço do início da fila
if inicio = nil
then showmessage('Não há senhas cadastradas')
else begin
       // a variável QTDE1 armazenará a quantidade de atendimentos
       // realizados do tipo 10
       qtde1:=0;
       // a variável QTDE2 armazenará a quantidade de atendimentos
       // realizados do tipo 20
       qtde2:=0;
       // o ponteiro AUX receberá o endereço do primeiro
       // endereço de memória da fila
       aux:=inicio;
       // enquanto o ponteiro AUX for diferente do
       // ponteiro PROX_ATEND, o laço abaixo será executado
       // ou seja, enquanto não encontrar o endereço
       // do próximo atendimento, executará o laço
       while aux <> prox_atend do
       begin
         if aux^.codigo = 10
         then qtde1:=qtde1+1
         else qtde2:=qtde2+1;
         // avança para o próximo endereço da fila
         aux:=aux^.proximo;
       end;
       showmessage('Quantidade de atendimentos com código 10 = '
                   +inttostr(qtde1));
       showmessage('Quantidade de atendimentos com código 20 = '
                   +inttostr(qtde2));
     end;
end;
```

O código do botão *Consultar o Tempo Médio de Espera* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var qtde:integer;
    total, tempo:ttime;
    medio, espera: real;
    h,m,s,cs: word;
begin
// o ponteiro INICIO aponta para o endereço do topo da fila
if inicio = nil
then showmessage('Não há senhas cadastradas')
else if prox_atend = inicio
```

```
       then showmessage('Não há atendimentos cadastrados')
       else begin
        // o ponteiro AUX receberá o endereço do primeiro
        // endereço de memória da fila
        aux:=inicio;
        // enquanto o ponteiro AUX for diferente do
        // ponteiro PROX_ATEND, o laço abaixo será executado
        // ou seja, enquanto não encontrar o endereço
        // do próximo atendimento, executará o laço,
        // a variável QTDE armazenará a quantidade de atendimentos
        // realizados, e a variável TOTAL armazenará a soma dos tempos
        // de espera, ou seja, tempo entre senha e atendimento
        qtde:=0;
        total:=0;
        while aux <> prox_atend do
        begin
          qtde:=qtde+1;
          tempo:=aux^.hora_atendimento - aux^.hora_senha;
          total:=total+tempo;
          // avança para o próximo endereço da fila
          aux:=aux^.proximo;
        end;
        decodetime(total,h,m,s,cs);
        espera:=(h*60)+ m;
        medio:=espera/qtde;
      showmessage('Tempo médio de espera = '+floattostr(medio));
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

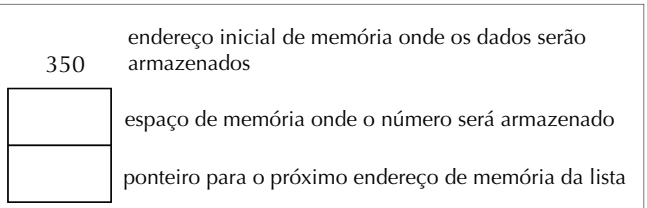
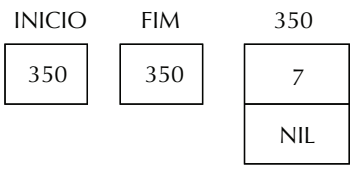
```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Unidade 4

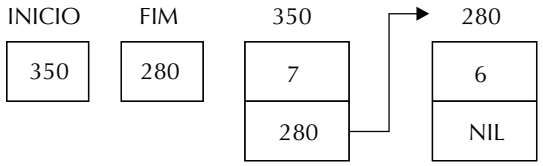
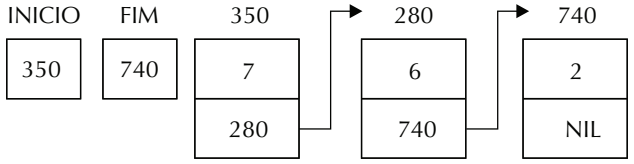
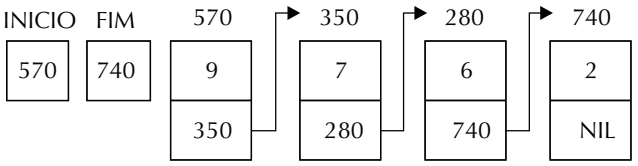
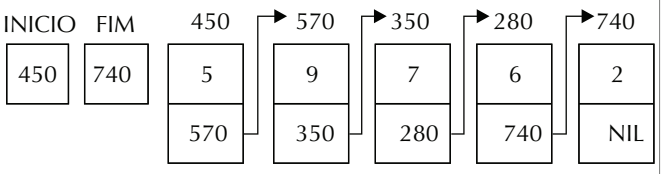
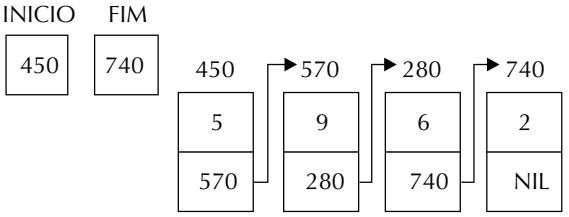
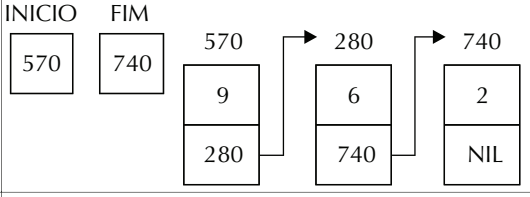
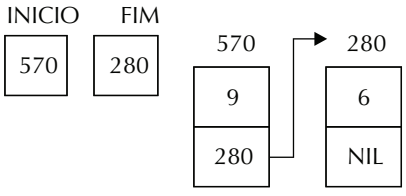
Listas estática e dinâmica simplesmente encadeadas*

A lista é uma estrutura de dados que pode armazenar dados ordenados e não ordenados. A idéia fundamental da lista é que todos os seus elementos possam ser acessados. Assim, quando um elemento novo é introduzido na lista, este poderá ser colocado no início, no final ou no meio dela. É possível fazer as seguintes implementações de uma lista: simplesmente encadeada e não ordenada, simplesmente encadeada e ordenada, duplamente encadeada e não ordenada e duplamente encadeada e ordenada. As listas simplesmente encadeadas serão abordadas neste capítulo, já as listas duplamente encadeadas serão abordadas na Unidade 5.

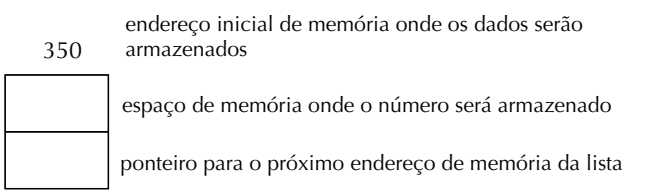
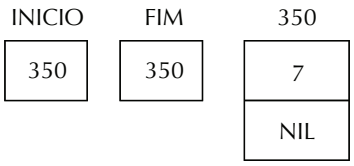
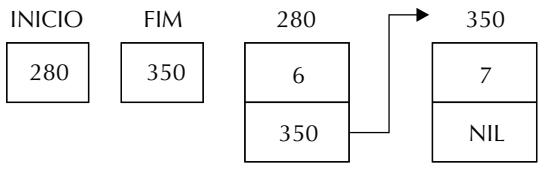
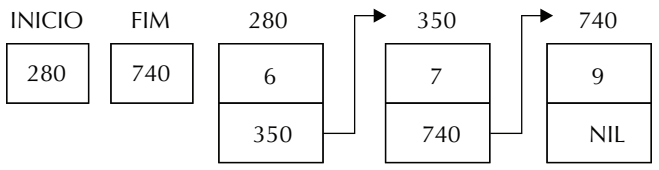
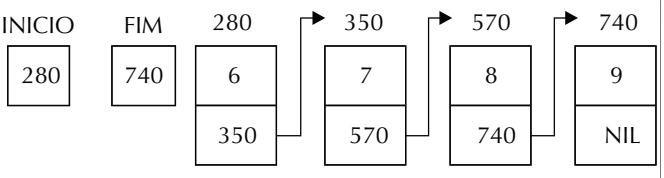
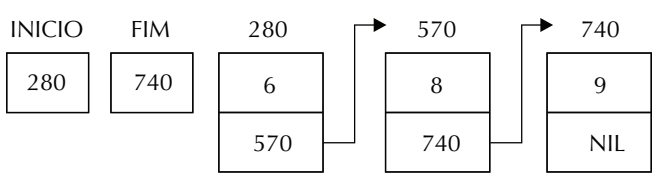
As estruturas de dados do tipo lista, descritas acima, podem ser realizadas utilizando-se a implementação estática, ou seja, com vetores, ou utilizando-se a implementação dinâmica, ou seja, com ponteiros. A tabela abaixo mostra os passos na execução de uma lista dinâmica simplesmente encadeada e não ordenada.

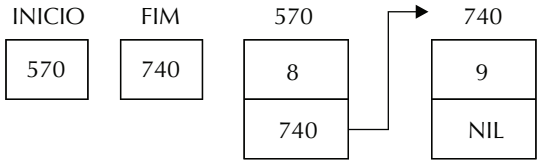
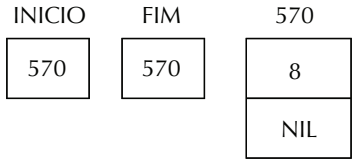
Estrutura de dados do tipo lista simplesmente encadeada e não ordenada para armazenar um número. Como a lista não é ordenada, o usuário pode escolher se deseja inserir novos números no início ou no fim da lista.	
Inicialmente a estrutura de dados do tipo lista está vazia, logo, o ponteiro INICIO aponta para vazio.	
Primeiro elemento inserido na lista. Este terá o ponteiro próximo apontando para vazio e, por ser o primeiro elemento, os ponteiros INICIO e FIM passarão a apontar para a nova posição de memória alocada.	

* Esta unidade corresponde ao Capítulo 5 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005.

Segundo elemento inserido na lista. Decide-se fazer a inserção no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória e o FIM passa a ter o valor da nova posição de memória alocada.	
Terceiro elemento inserido na lista. Decide-se fazer a inserção no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória e o FIM passa a ter o valor da nova posição de memória alocada.	
Quarto elemento inserido na lista. Decide-se fazer a inserção no início da lista, assim, o ponteiro próximo do novo elemento passa a ter o valor do INICIO e o INICIO passa a ter o valor da nova posição de memória alocada.	
Quinto elemento inserido na lista. Decide-se fazer a inserção no início da lista, assim, o ponteiro próximo do novo elemento passa a ter o valor do INICIO e o INICIO passa a ter o valor da nova posição de memória alocada.	
Qualquer elemento pode ser retirado da lista. Escolhe-se retirar o elemento 7 que está no meio da lista. Os ponteiros INICIO e FIM não mudam, mas o elemento que apontava para o 7 passa a apontar para o endereço do ponteiro próximo da posição do número 7.	
Escolhe-se agora retirar o elemento 5 que está no início da lista, logo, o ponteiro INICIO passa a apontar para o endereço do ponteiro próximo da posição do INICIO.	
Escolhe-se agora retirar o elemento 2 que está no final da lista, logo, o ponteiro FIM passa a apontar para o endereço que apontava para o elemento que será retirado e o ponteiro próximo do elemento que agora é o FIM passa a apontar para NIL.	

A tabela abaixo mostra os passos na execução de uma lista dinâmica simplesmente encadeada e ordenada.

Estrutura de dados do tipo lista simplesmente encadeada e ordenada para armazenar um número. Como a lista é ordenada, o programa é que decide onde inserir novos números: no início, no fim ou no meio da lista.	
Inicialmente a estrutura de dados do tipo lista está vazia, logo, o ponteiro INICIO aponta para vazio.	
Primeiro elemento inserido na lista é o número 7. Este terá o ponteiro próximo apontando para vazio e, por ser o primeiro elemento, os ponteiros INICIO e FIM passam a apontar para a nova posição de memória alocada.	
Segundo elemento inserido na lista, o número 6; logo, a inserção será no início da lista, assim, o ponteiro próximo do novo elemento passa a ter o valor do INICIO e o INICIO passa a ter o valor da nova posição de memória alocada.	
Terceiro elemento inserido na lista, o número 9; logo, a inserção será no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória e o FIM passa a ter o valor da nova posição de memória alocada.	
Quarto elemento inserido na lista, o número 8; logo, a inserção será no meio da lista, assim, o ponteiro próximo do elemento anterior passa a apontar para a nova posição e o ponteiro próximo da nova posição passa a apontar para o valor do próximo da posição anterior à inserção.	
Escolhe-se retirar o elemento 7 que está no meio da lista. Os ponteiros INICIO e FIM não mudam, mas o elemento que apontava para o 7 passa a apontar para o endereço do ponteiro próximo à posição do número 7.	

Escolhe-se agora retirar o elemento 6 que está no início da lista, logo, o ponteiro INICIO passa a apontar para o endereço do ponteiro próximo à posição do INICIO.	
Escolhe-se agora retirar o elemento 9 que está no final da lista, logo, o ponteiro FIM passa a apontar para o endereço que apontava para o elemento que será retirado e o ponteiro próximo do elemento que agora é o FIM passa a apontar para NIL.	

Implementacão de uma lista estática simplesmente encadeada e não ordenada

A Figura 4.1 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista simplesmente encadeada e não ordenada: insere no início da lista, insere no final da lista, retira da lista, consulta um elemento da lista, consulta toda a lista e esvazia a lista. Todas essas operações serão realizadas pela implementação estática, ou seja, utilizando vetores; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho do vetor. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 4.1 Tela da aplicação com implementação estática de uma lista simplesmente encadeada e não ordenada



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador LISTA, com 5 posições
     // que armazenarão números inteiros
     lista: array[1..5] of integer;
     // a variável QTDE identifica a quantidade de números
     // já inseridos na lista
     qtde: integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde:=0;
edit1.text:=inttostr(qtde);
end;
```

O código do botão *Inserir no Início da Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero,i: integer;
begin
if qtde = 5
then showmessage('A lista está cheia')
else begin
     num:=inputbox('Lista simplesmente encadeada e não ordenada com
                   implementação estática','Digite o número a ser
                   inserido no início da lista','');
     if num <> ''
     then begin
           numero:=strtoint(num);
           // como o novo número será inserido no início
           // da lista, deve ocorrer o deslocamento dos
           // números já existentes para a direita
           for i:=qtde+1 downto 2 do
             lista[i]:=lista[i-1];
           qtde:=qtde+1;
           lista[1]:=numero;
           edit1.text:=inttostr(qtde);
           showmessage('O número '+ num+ ' foi inserido com sucesso
                       no início da lista');
         end;
     end;
end;
```

O código do botão *Inserir no Final da Lista* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
var num: string;
    numero: integer;
begin
if qtde = 5
```

```
then showmessage('A lista está cheia')
else begin
     num:=inputbox('Lista simplesmente encadeada e não ordenada com
                    implementação estática','Digite o número a ser
                    inserido no final da lista','');
     if num <> ''
     then begin
           numero:=strtoint(num);
           // o novo número será inserido no final da lista
           qtde:=qtde+1;
           lista[qtde]:=numero;
           edit1.text:=inttostr(qtde);
           showmessage('O número '+ num+ ' foi inserido com sucesso
                       no final da lista');
         end;
     end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, i, achou: integer;
    num: string;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
      // a variável ACHOU verifica se
      // o número procurado existe na lista
      achou:=0;
      num:=inputbox('Lista simplesmente encadeada e não ordenada com
                    implementação estática','Digite o número a ser
                    procurado na lista','');
      if num <> ''
         then begin
              numero:=strtoint(num);
              for i:=1 to qtde do
              begin
              if lista[i] = numero
              then achou:=i;
             end;
             if achou = 0
             then showmessage('O número '+num+' não foi encontrado na
                              lista')
             else showmessage('O número '+num+' foi encontrado na
                              posição '+inttostr(achou)+' da lista');
             end;
     end;
end;
```

O código do botão *Consultar Toda a Lista* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
      texto:='';
      for i:= 1 to qtde do
      begin
        texto:=texto+ '   '+inttostr(lista[i]);
      end;
      showmessage('A lista toda: '+texto);
     end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, i, achou: integer;
   num: string;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
     // será necessário encontrar o número na lista
     // e deslocar os elementos dentro do vetor;
     // a variável ACHOU marca se o número a ser retirado
     // da lista foi encontrado
     achou:=0;
     num:=inputbox('Lista simplesmente encadeada e não ordenada com
                  implementação estática','Digite o número a ser
                  retirado da lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             for i:=1 to qtde do
              begin
               if lista[i] = numero
               then achou:=i;
              end;
            if achou <> 0
            then begin
                 // o número a ser retirado da lista foi encontrado,
                 // logo, haverá deslocamento
                 for i:=achou to qtde-1 do
                 begin
                 lista[i]:=lista[i+1];
                 end;
                 qtde:=qtde-1;
                 edit1.text:=inttostr(qtde);
```

```
                showmessage('O número '+ inttostr(numero)+ ' foi
                            retirado com sucesso da lista');
                end
            else showmessage('O número '+ inttostr(numero)+ ' não foi
                            encontrado na lista');
            end;
    end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if qtde = 0
then showmessage('A lista já está vazia')
else begin
      qtde:=0;
      edit1.text:=inttostr(qtde);
      showmessage('A lista foi esvaziada com sucesso');
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação de uma lista estática simplesmente encadeada e ordenada

A Figura 4.2 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista simplesmente encadeada e ordenada: insere na lista, retira da lista, consulta um elemento da lista, consulta toda a lista e esvazia a lista. Todas essas operações serão realizadas pela implementação estática, ou seja, utilizando-se vetores; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho do vetor. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 4.2 Tela da aplicação com implementação estática de uma lista simplesmente encadeada e ordenada



A seguir é apresentada a declaração das variáveis globais.

```
var Form1: TForm1;
     // vetor com identificador LISTA, com 5 posições
     // que armazenarão números inteiros
     lista: array[1..5] of integer;
     // a variável QTDE identifica a quantidade de números
     // já inseridos na lista
     qtde: integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde:=0;
edit1.text:=inttostr(qtde);
end;
```

O código do botão *Inserir na Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero, i, achou : integer;
begin
if qtde = 5
then showmessage('A lista está cheia')
else begin
     num:=inputbox('Lista simplesmente encadeada e ordenada com
                  implementação estática','Digite o número a ser
                  inserido na lista','');
     if num <> ''
     then begin
           numero:=strtoint(num);
```

```
           // como o novo número será inserido de forma ordenada
           // na lista, deve ocorrer o deslocamento dos
           // números já existentes para a direita; se necessário.
           // a variável ACHOU marca a posição
           // em que o número deve ser inserido
           achou:=0;
           // a variável I será utilizada para percorrer o vetor
           // que representa a lista e contém os números
           i:=1;
           while (i <= qtde) and (achou = 0) do
           begin
            if lista[i] > numero
            then achou:=i
            else i:=i+1;
           end;
           if achou = 0
           then begin
                // o novo número será inserido no final da lista
                // pois é maior que todos os números já cadastrados
                qtde:=qtde+1;
                lista[qtde]:=numero;
                edit1.text:=inttostr(qtde);
                showmessage('O número '+ num+ ' foi inserido com
                            sucesso na lista');
               end
           else begin
                // haverá o deslocamento dos números para a direita
                for i:=qtde+1 downto achou do
                  lista[i]:=lista[i-1];
                qtde:=qtde+1;
                lista[achou]:=numero;
                edit1.text:=inttostr(qtde);
                showmessage('O número '+ num+ ' foi inserido com
                            sucesso na lista');
               end;
         end;
    end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, i, achou: integer;
    num: string;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
     // a variável ACHOU verifica se
     // o número procurado existe na lista
     achou:=0;
     num:=inputbox('Lista simplesmente encadeada e ordenada com
                   implementação estática','Digite o número a ser
                   procurado na lista','');
```

```
     if num <> ''
        then begin
             numero:=strtoint(num);
             i:=1;
             while (i <= qtde) and (achou = 0) do
             begin
               if lista[i] = numero
                 then achou:=i
                 else i:=i+1;
             end;
             if achou = 0
             then showmessage('O número '+num+' não foi encontrado na
                              lista')
             else showmessage('O número '+num+' foi encontrado na
                              posição '+inttostr(achou)+' da lista');
             end;
     end;
end;
```

O código do botão *Consultar Toda a Lista* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
     texto:='';
     for i:= 1 to qtde do
     begin
       texto:=texto+ '   '+inttostr(lista[i]);
     end;
     showmessage('A lista toda: '+texto);
    end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, i, achou: integer;
    num: string;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
      // será necessário encontrar o número da lista
      // e deslocar os elementos dentro do vetor;
      // a variável ACHOU marca se o número a ser retirado
      // da lista foi encontrado
      achou:=0;
      num:=inputbox('Lista simplesmente encadeada e ordenada com
                    implementação estática','Digite o número a ser
                    retirado da lista','');
```

```
      if num <> ''
         then begin
              numero:=strtoint(num);
              for i:=1 to qtde do
              begin
                if lista[i] = numero
                then achou:=i;
              end;
              if achou <> 0
              then begin
                   // o número a ser retirado da lista foi encontrado,
                   // logo, haverá deslocamento
                   for i:=achou to qtde-1 do
                   begin
                   lista[i]:=lista[i+1];
                   end;
                   qtde:=qtde-1;
                   edit1.text:=inttostr(qtde);
                   showmessage('O número '+ inttostr(numero)+ ' foi
                               retirado com sucesso da lista');
                   end
              else showmessage('O número '+ inttostr(numero)+ ' não foi
                               encontrado na lista');
              end;
    end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if qtde = 0
then showmessage('A lista já está vazia')
else begin
     qtde:=0;
     edit1.text:=inttostr(qtde);
     showmessage('A lista foi esvaziada com sucesso');
    end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação de uma lista dinâmica simplesmente encadeada e não ordenada

A Figura 4.3 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista simplesmente encadeada e não ordenada: insere no início da lista, insere no final da lista, retira da lista, consulta um elemento da lista, consulta toda a lista e esvazia a lista. Todas essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho da memória. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 4.3 **Tela da aplicação com implementação dinâmica de uma lista simplesmente encadeada e não ordenada**



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da lista;
// a variável NUM armazenará um número inteiro;
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo: pointer;
     end;

// as variáveis LISTA, INICIO, FIM, AUX e AUX2 são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável QTDE armazena a quantidade de números
// armazenados na lista
var Form1: TForm1;
    lista, inicio, fim, aux, aux2 : ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro INICIO aponta para vazio, pois a lista está vazia
inicio:=nil;
end;
```

O código do botão *Inserir no Início da Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
num:=inputbox('Lista simplesmente encadeada e não ordenada com
               implementação dinâmica','Digite o número a ser
               inserido no início da lista','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(lista);
      if inicio = nil
      then begin
           // a lista está vazia, logo,
           // este número será o primeiro a ser inserido
           lista^.num := numero;
           lista^.proximo:=nil;
           inicio:=lista;
           fim:=lista;
          end
      else begin
          // já existe algum número na lista
          // e o novo número será inserido no início
          lista^.num:= numero;
          lista^.proximo:=inicio;
          inicio:=lista;
          end;
      qtde:=qtde+1;
      edit1.text:=inttostr(qtde);
      showmessage('O número '+ num+ ' foi inserido com sucesso no
                  início da lista');
      end;
end;
```

O código do botão *Inserir no Final da Lista* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
var num: string;
    numero: integer;
begin
num:=inputbox('Lista simplesmente encadeada e não ordenada com
              implementação dinâmica','Digite o número a ser inserido
              no final da lista','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(lista);
      if inicio = nil
      then begin
           // a lista está vazia, logo,
           // este número será o primeiro a ser inserido
           lista^.num := numero;
           lista^.proximo:=nil;
           inicio:=lista;
           fim:=lista;
          end
      else begin
          // já existe algum número na lista
          // e o novo número será inserido no final
          lista^.num:= numero;
          lista^.proximo:=nil;
          fim^.proximo:=lista;
          fim:=lista;
          end;
      qtde:=qtde+1;
      edit1.text:=inttostr(qtde);
      showmessage('O número '+ num+ ' foi inserido com sucesso no
                  final da lista');
      end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
      // a variável ACHOU verifica se
      // o número procurado existe na lista
      achou:=0;
```

```
     num:=inputbox('Lista simplesmente encadeada e não ordenada com
                   implementação dinâmica','Digite o número a ser
                   procurado na lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             aux:=inicio;
             while (aux <> nil) do
             begin
               if aux^.num = numero
               then achou:=achou+1;
             aux:=aux^.proximo;
             end;
             if achou = 0
             then showmessage('O número '+num+' não foi encontrado na
                              lista')
             else if achou = 1
                  then showmessage('O número '+num+' foi encontrado'
                                   +inttostr(achou)+' vez')
                  else showmessage('O número '+num+' foi encontrado'
                                   +inttostr(achou)+' vezes');
           end;
    end;
end;
```

O código do botão *Consultar Toda a Lista* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if inicio =  nil
then showmessage('A lista está vazia')
else begin
     texto:='';
     aux:=inicio;
     while (aux <> nil) do
     begin
       texto:=texto+ '   '+inttostr(aux^.num);
       aux:=aux^.proximo;
     end;
     showmessage('A lista toda: '+texto);
    end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, achou: integer;
```

```
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
     // será necessário encontrar o número na lista e
     // a variável ACHOU marca se o número a ser retirado
     // da lista foi encontrado
     achou:=0;
     num:=inputbox('Lista simplesmente encadeada e não ordenada com
                  implementação dinâmica','Digite o número a ser
                  retirado da lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             aux:=inicio;
             while (aux <> nil) do
             begin
               if aux^.num = numero
                 then begin
                      achou:=1;
                      qtde:=qtde-1;
                      edit1.text:=inttostr(qtde);
                      // se o número a ser excluído for o primeiro
                      if aux = inicio
                       then begin
                            inicio:=aux^.proximo;
                            dispose(aux);
                            aux:=inicio;
                            end
                       else if aux = fim
                            // se o número a ser excluído
                            // for o último
                            then begin
                                 aux2:=inicio;
                                 while (aux2^.proximo <> aux) do
                                 begin
                                   aux2:=aux2^.proximo;
                                 end;
                                 aux2^.proximo:=nil;
                                 dispose(fim);
                                 fim:=aux2;
                                 aux:=nil;
                                 end
                            else begin
                                 // se o número a ser excluído
                                 // estiver no meio da lista
                                 aux2:=inicio;
                                 while (aux2^.proximo <> aux) do
```

```
                                begin
                                  aux2:=aux2^.proximo;
                                end;
                                aux2^.proximo:=aux^.proximo;
                                dispose(aux);
                                aux:=aux2;
                                end;
                        end
              else aux:=aux^.proximo;
              end;
      if achou = 1
      then showmessage('O número '+ inttostr(numero)+ ' foi retirado
                       com sucesso da lista')
      else showmessage('O número '+ inttostr(numero)+ ' não foi
                       encontrado na lista');
     end;
  end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if inicio = nil
then showmessage('A lista já está vazia')
else begin
      aux:=inicio;
      while (aux <> nil) do
      begin
        inicio:=inicio^.proximo;
        dispose(aux);
        aux:= inicio;
      end;
      showmessage('A lista foi esvaziada com sucesso');
      qtde:=0;
      edit1.text:=inttostr(qtde);
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação de uma lista dinâmica simplesmente encadeada e ordenada

A Figura 4.4 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista simplesmente encadeada e ordenada: insere na lista, retira da lista, consulta um elemento da lista, consulta toda a lista e esvazia a lista. Todas essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho da memória. Após a figura têm-se os seus respectivos códigos comentados.

Figura 4.4 Tela da aplicação com implementação dinâmica de uma lista simplesmente encadeada e ordenada



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da lista;
// a variável NUM armazenará um número inteiro;
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo: pointer;
     end;

// as variáveis LISTA, INICIO, FIM, AUX e AUX2 são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
```

```
// a variável QTDE armazena a quantidade de números
// armazenados na lista

var Form1: TForm1;
    lista, inicio, fim, aux, aux2 : ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro INICIO aponta para vazio, pois a lista está vazia
inicio:=nil;
end;
```

O código do botão *Inserir na Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
num:=inputbox('Lista simplesmente encadeada e ordenada com
               implementação dinâmica','Digite o número a ser
               inserido na lista','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(lista);
      if inicio = nil
      then begin
           // a lista está vazia, logo,
           // este número será o primeiro a ser inserido
           lista^.num := numero;
           lista^.proximo:=nil;
           inicio:=lista;
           fim:=lista;
          end
      else begin
          // já existe algum número na lista
          // e o novo número será inserido no lugar ordenado
          if (inicio^.num >= numero)
          then begin
               // o novo número é menor que o primeiro,
               // logo, será inserido no início da lista
               lista^.num:= numero;
               lista^.proximo:=inicio;
               inicio:=lista;
               end
```

```
          else if (fim^.num <= numero)
                then begin
                     // o novo número é maior que o último
                     // logo, será inserido no final da lista
                     lista^.num:= numero;
                     lista^.proximo:=nil;
                     fim^.proximo:=lista;
                     fim:=lista;
                     end
                 else begin
                      // o novo número será inserido no meio da lista
                      aux:=inicio;
                      while (aux^.num < numero) do
                      begin
                        aux:=aux^.proximo;
                      end;
                      aux2:=inicio;
                      while (aux2^.proximo <> aux) do
                      begin
                      aux2:=aux2^.proximo;
                      end;
                      lista^.num:= numero;
                      aux2^.proximo:=lista;
                      lista^.proximo:=aux;
                      end;
          end;
      qtde:=qtde+1;
      edit1.text:=inttostr(qtde);
      showmessage('O número '+ num+ ' foi inserido com sucesso na
                  lista');
      end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
      // a variável ACHOU verifica se
      // o número procurado existe na lista
      achou:=0;
      num:=inputbox('Lista simplesmente encadeada e ordenada com
                    implementação dinâmica','Digite o número a ser
                    procurado na lista','');
      if num <> ''
         then begin
```

```
            numero:=strtoint(num);
            aux:=inicio;
            while (aux <> nil) and (aux^.num <= numero) do
            begin
              if aux^.num = numero
              then achou:=achou+1;
            aux:=aux^.proximo;
            end;
            if achou = 0
            then showmessage('O número '+num+' não foi encontrado na
                             lista')
            else if achou = 1
                 then showmessage('O número '+num+' foi encontrado'
                                  +inttostr(achou)+' vez')
                 else showmessage('O número '+num+' foi encontrado'
                                  +inttostr(achou)+' vezes');
          end;
    end;
end;
```

O código do botão *Consultar Toda a Lista* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if inicio =  nil
then showmessage('A lista está vazia')
else begin
     texto:='';
     aux:=inicio:
     while (aux <> nil) do
     begin
       texto:=texto+ '   '+inttostr(aux^.num);
       aux:=aux^.proximo;
     end;
     showmessage('A lista toda: '+texto);
    end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
```

```
// será necessário encontrar o número na lista e
// a variável ACHOU marca se o número a ser retirado
// da lista foi encontrado
achou:=0;
num:=inputbox('Lista simplesmente encadeada e ordenada com
              implementação dinâmica','Digite o número a ser
              retirado da lista','');
if num <> ''
   then begin
        numero:=strtoint(num);
        aux:=inicio;
        while (aux <> nil) and (aux^.num <= numero) do
         begin
          if aux^.num = numero
            then begin
                 achou:=1;
                 qtde:=qtde-1;
                 edit1.text:=inttostr(qtde);
                 // se o número a ser excluído for o primeiro
                 if aux = inicio
                  then begin
                       inicio:=aux^.proximo;
                       dispose(aux);
                       aux:=inicio;
                       end
                  else if aux = fim
                       // se o número a ser excluído
                       // for o último
                       then begin
                            aux2:=inicio;
                            while (aux2^.proximo <> aux) do
                            begin
                              aux2:=aux2^.proximo;
                            end;
                            aux2^.proximo:=nil;
                            dispose(fim);
                            fim:=aux2;
                            aux:=nil;
                            end
                       else begin
                            // se o número a ser excluído
                            // estiver no meio da lista
                            aux2:=inicio;
                            while (aux2^.proximo <> aux) do
                            begin
                              aux2:=aux2^.proximo;
                            end;
                            aux2^.proximo:=aux^.proximo;
                            dispose(aux);
```

```
                                aux:=aux2;
                                end;
                        end
              else aux:=aux^.proximo;
              end;
      if achou = 1
      then showmessage('O número '+ inttostr(numero)+ ' foi retirado
                       com sucesso da lista')
      else showmessage('O número '+ inttostr(numero)+ ' não foi
                       encontrado na lista');
     end;
  end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if inicio = nil
then showmessage('A lista já está vazia')
else begin
      aux:=inicio;
      while (aux <> nil) do
      begin
        inicio:=inicio^.proximo;
        dispose(aux);
        aux:=inicio;
      end;
      showmessage('A lista foi esvaziada com sucesso');
      qtde:=0;
      edit1.text:=inttostr(qtde);
     end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

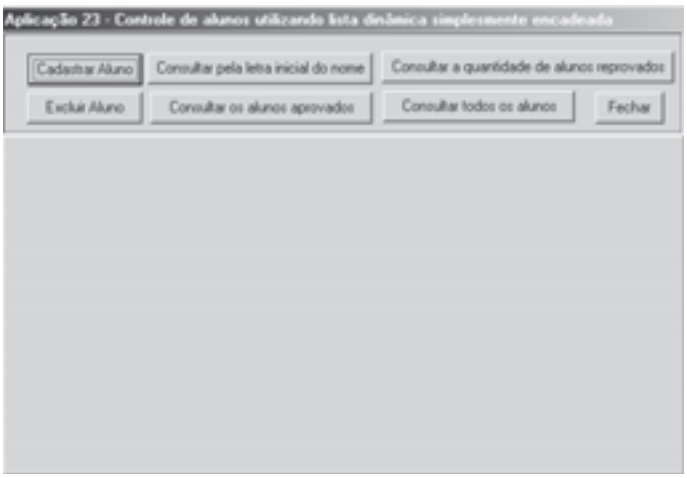
```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Aplicacão de controle de alunos usando lista dinâmica simplesmente encadeada e não ordenada

A Figura 4.5 mostra a tela da aplicação de um controle de alunos utilizando uma estrutura de dados do tipo lista simplesmente encadeada e não

ordenada com implementação dinâmica. Nesta aplicação são cadastrados novos alunos (não é permitido o cadastro de mais de um aluno com o mesmo código), excluídos alunos já cadastrados, e algumas consultas implementadas são: consulta por letra inicial do nome do aluno, consulta de alunos aprovados, consulta da quantidade de alunos reprovados e consulta geral de todos os alunos com suas respectivas médias e situações. Após a figura, têm-se seus respectivos códigos comentados.

Figura 4.5 Tela da aplicação de controle de alunos utilizando lista simplesmente encadeada e não ordenada com implementação dinâmica



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da lista;
// a variável NOME armazenará o nome do aluno;
// a variável NOTA1 armazenará a primeira nota do aluno;
// a variável NOTA2 armazenará a segunda nota do aluno;
// a variável PROXIMO é um ponteiro para um endereço
// de memória do tipo REGISTRO
type registro = record
      nome: string;
      nota1,nota2: real;
      proximo: pointer;
     end;

// as variáveis LISTA, INICIO, FIM, AUX e AUX2 são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente
```

```
var Form1: TForm1;
    lista, inicio, fim, aux, aux2: ^registro;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

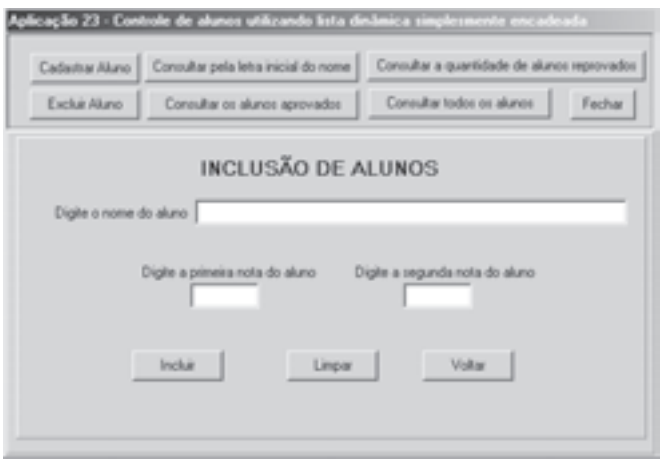
```
procedure TForm1.FormActivate(Sender: TObject);
begin
// o ponteiro INICIO aponta para vazio, pois a lista está vazia
inicio:=nil;
end;
```

O código do botão *Cadastrar Aluno* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
begin
// desabilita o acesso ao panel1 que possui
// os botões das operações
panel1.enabled:=false;
// mostra o panel3 que possui os componentes para
// a inclusão de novos alunos
panel3.visible:=true;
// prepara os componentes do panel3 para que
// estes apareçam limpos
edit1.clear;
edit2.clear;
edit3.clear;
edit1.setfocus;
end;
```

Quando o botão *Cadastrar Aluno* é clicado, a tela da Figura 4.6 é apresentada.

Figura 4.6 Tela de cadastro de alunos da aplicação de controle de alunos



O código do botão *Incluir* da tela de cadastro de alunos é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
begin
if trim(edit1.text) = ''
then begin
     showmessage('Digite o nome do aluno');
     edit1.setfocus;
    end
else if edit2.text = ''
     then begin
          showmessage('Digite a primeira nota do aluno');
          edit2.setfocus;
         end
     else if edit3.text = ''
          then begin
               showmessage('Digite a segunda nota do aluno');
               edit3.setfocus;
              end
          else if strtofloat(edit2.text) > 10
               then begin
                    showmessage('Primeira nota inválida');
                    edit2.setfocus;
                   end
               else if strtofloat(edit3.text) > 10
                    then begin
                         showmessage('Segunda nota inválida');
                         edit3.setfocus;
                        end
                    else begin
                         new(lista);
                         if inicio = nil
                         then begin
                              // a lista está vazia e
                              // este aluno será o primeiro
                              // a ser inserido
                              lista^.nome := edit1.text;
                              lista^.nota1:= strtofloat(edit2.text);
                              lista^.nota2:= strtofloat(edit3.text);
                              lista^.proximo:=nil;
                              inicio:=lista;
                              fim:=lista;
                              end
                         else begin
                              // já existe algum aluno na lista
                              // e o novo aluno será inserido
                              // no final
                              lista^.nome:= edit1.text;
```

```
                                lista^.nota1:= strtofloat(edit2.text);
                                lista^.nota2:= strtofloat(edit3.text);
                                lista^.proximo:=nil;
                                fim^.proximo:=lista;
                                fim:=lista;
                                end;
                          showmessage('Aluno incluído com sucesso');
                          // prepara os componentes
                          // do panel3 para que
                          // estes fiquem limpos
                          edit1.clear;
                          edit2.clear;
                          edit3.clear;
                          edit1.setfocus;
                        end;
end;
```

O código do botão *Limpar* da tela de cadastro de alunos é apresentado a seguir.

```
procedure TForm1.Button9Click(Sender: TObject);
begin
// faz com que todos os componentes
// do panel3 fiquem limpos
edit1.clear;
edit2.clear;
edit3.clear;
edit1.setfocus;
end;
```

O código do botão *Voltar* da tela de cadastro de alunos é apresentado a seguir.

```
procedure TForm1.Button10Click(Sender: TObject);
begin
// habilita o acesso ao panel1 que possui
// os botões das operações
panel1.enabled:=true;
// esconde o panel3 que possui os componentes para
// a inclusão de novos alunos
panel3.visible:=false;
end;
```

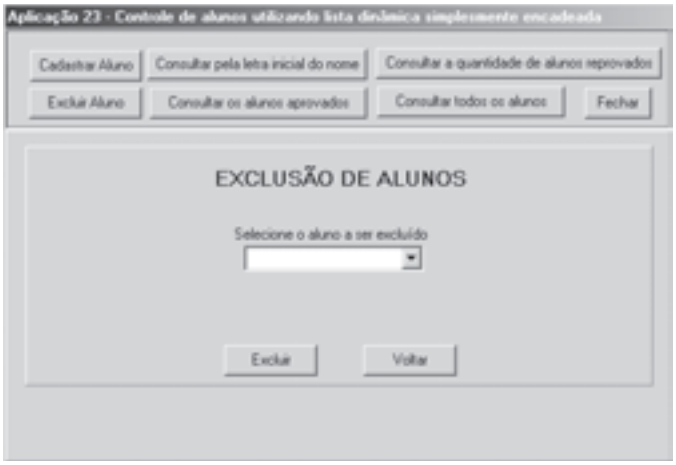
O código do botão *Excluir Aluno* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
begin
if inicio = nil
then showmessage('Nenhum aluno foi cadastrado')
else begin
```

```
     // desabilita o acesso ao panel1 que possui
     // os botões das operações
     panel1.enabled:=false;
     // mostra o panel4 que possui os componentes para
     // a exclusão de alunos já cadastrados
     panel4.visible:=true;
     // limpa o componente que mostrará os alunos cadastrados
     combobox1.clear;
     // mostra os alunos já cadastrados para seleção
     // do aluno que será excluído
     aux:=inicio;
     while (aux <> nil) do
       begin
        combobox1.items.add(aux^.nome);
        aux:=aux^.proximo;
       end;
    end;
end;
```

Quando o botão *Excluir Aluno* é clicado, a tela da Figura 4.7 é apresentada.

Figura 4.7 Tela de exclusão de alunos da aplicação de controle de alunos



O código do botão *Excluir* da tela de exclusão de alunos é apresentado a seguir.

```
procedure TForm1.Button11Click(Sender: TObject);
begin
if combobox1.itemIndex = -1
then begin
       showmessage('Selecione o aluno a ser excluído');
       combobox1.setfocus;
    end
```

```
else begin
      // será necessário encontrar o aluno na lista
      aux:=inicio;
      while (aux <> nil) do
       begin
       if aux^.nome = combobox1.text
        then begin
             if aux = inicio
              then begin
                    // o aluno a ser excluído é o primeiro
                    inicio:=aux^.proximo;
                    dispose(aux);
                    aux:=inicio;
                   end
               else if aux = fim
                    then begin
                         // o aluno a ser excluído é o último
                         aux2:=inicio;
                         while (aux2^.proximo <> aux) do
                          begin
                           aux2:=aux2^.proximo;
                          end;
                         aux2^.proximo:=nil;
                         dispose(fim);
                         fim:=aux2;
                         aux:=nil;
                         end
                    else begin
                         // se o aluno a ser excluído
                         // estiver no meio da lista
                         aux2:=inicio;
                         while (aux2^.proximo <> aux) do
                          begin
                           aux2:=aux2^.proximo;
                          end;
                         aux2^.proximo:=aux^.proximo;
                         dispose(aux);
                         aux:=aux2;
                         end;
             end
       else aux:=aux^.proximo;
       end;
       showmessage('Aluno excluído com sucesso');
       combobox1.itemindex:=-1;
       combobox1.clear;
       // é necessário fazer a atualização dos alunos
       // listados no combobox1
       aux:=inicio;
      while (aux <> nil) do
        begin
         combobox1.items.add(aux^.nome);
```

```
        aux:=aux^.proximo;
       end;
     end;
end;
```

O código do botão *Voltar* da tela de exclusão de alunos é apresentado a seguir.

```
procedure TForm1.Button12Click(Sender: TObject);
begin
// habilita o acesso ao panel1 que possui
// os botões das operações
panel1.enabled:=true;
// esconde o panel4 que possui os componentes para
// a exclusão de alunos
panel4.visible:=false;
end;
```

O código do botão *Consultar pela Letra Inicial do Nome* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var letra:string;
    td, achou: integer;
begin
if inicio = nil
then showmessage('Nenhum aluno foi cadastrado')
else begin
      // desabilita o acesso ao panel1 que possui
      // os botões das operações
      panel1.enabled:=false;
      // mostra o panel5 que possui os componentes para
      // a consulta de alunos pela letra inicial do nome
      panel5.visible:=true;
      // limpa o memo que mostrará o resultado da consulta
      memo1.clear;
      letra:=inputbox('Letra inicial do nome para consulta','Digite a
                    letra','');
      if letra <> ''
      then begin
            // converte a letra digitada para maiúscula
            letra:=uppercase(letra);
            // a variável TD armazena a quantidade de
            // caracteres digitados
            td:=length(letra);
            // percorre toda a lista
            // para encontrar alunos com nomes
            // começados pelas letras digitadas;
            // a variável ACHOU marca se algum aluno foi encontrado
            achou:=0;
            aux:=inicio;
```

```
            while (aux <> nil) do
              begin
                if copy(aux^.nome,1,td) = letra
                then begin
                      achou:=1;
                      memo1.lines.add(aux^.nome+' NOTA1 = '+
                                  floattostr(aux^.nota1)+
                                  ' NOTA2= '+
                                  floattostr(aux^.nota2));
                    end;
                aux:=aux^.proximo;
              end;
            if achou = 0
            then showmessage('Nenhum aluno com nome com as letras
                           digitadas');
          end;
      end;
end;
```

Quando o botão *Consultar pela Letra Inicial do Nome* é clicado, a tela da Figura 4.8 é apresentada. Em seguida à digitação da letra, a tela da Figura 4.9 mostra o resultado da consulta, se esta existir.

Figura 4.8 Tela para a digitação da letra inicial do nome para consulta dos alunos

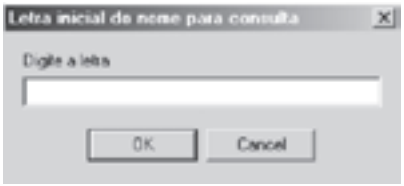
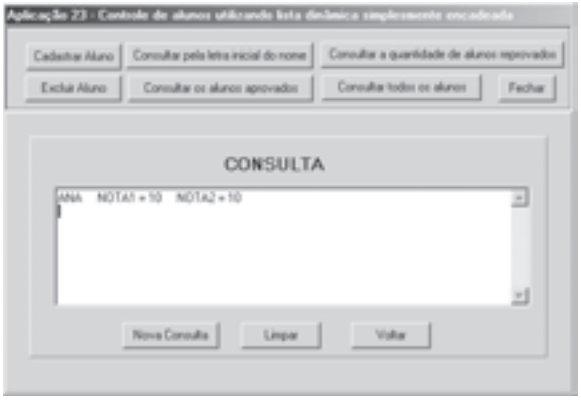


Figura 4.9 Tela da consulta de alunos pelas letras iniciais do nome da aplicação de controle de alunos



O código do botão *Nova Consulta* da tela de consulta de alunos pela letra inicial do nome é apresentado a seguir.

```
procedure TForm1.Button13Click(Sender: TObject);
var letra:string;
    td, achou: integer;
begin
memo1.clear;
letra:=inputbox('Letra inicial do nome para consulta','Digite a
               letra','');
if letra <> ''
 then begin
      // converte a letra digitada para maiúscula
     letra:=uppercase(letra);
     // a variável TD armazena a quantidade de caracteres digitados
     td:=length(letra);
     // percorre toda a lista
     // para encontrar alunos com nomes
     // começados pelas letras digitadas;
     // a variável ACHOU marca se algum aluno foi encontrado
     achou:=0;
     aux:=inicio;
     while (aux <> nil) do
      begin
      if copy(aux^.nome,1,td) = letra
      then begin
           achou:=1;
           memo1.lines.add(aux^.nome+'     NOTA1 = '+
                          floattostr(aux^.nota1)+
                          'NOTA2 = '+ floattostr(aux^.nota2));
           end;
      aux:=aux^.proximo;
      end;
     if achou = 0
     then showmessage('Nenhum aluno com nome com as letras
                      digitadas');
    end;
end;
```

O código do botão *Limpar* da tela de consulta de alunos pela letra inicial do nome é apresentado a seguir.

```
procedure TForm1.Button14Click(Sender: TObject);
begin
// limpa o componente que mostra o resultado da consulta
memo1.clear;
end;
```

O código do botão *Voltar* da tela de consulta de alunos pela letra inicial do nome é apresentado a seguir.

```
procedure TForm1.Button15Click(Sender: TObject);
begin
// habilita o acesso ao panel1 que possui
// os botões das operações
panel1.enabled:=true;
// mostra o button Nova Consulta
button13.visible:=true;
// mostra o button Limpar
button14.visible:=true;
// esconde o panel5 que possui os componentes para
// a consulta de alunos
panel5.visible:=false;
end;
```

O código do botão *Consultar a Quantidade de Alunos Reprovados* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
var achou: integer;
    media:real;
begin
if inicio = nil
then showmessage('Nenhum aluno foi cadastrado')
else begin
      // percorre toda a lista
      // para encontrar a quantidade de alunos reprovados;
      // a variável ACHOU conta a quantidade de alunos reprovados
      achou:=0;
      aux:=inicio;
      while (aux <> nil) do
       begin
       media:=(aux^.nota1+aux^.nota2)/2;
       if media < 7
       then achou:=achou + 1;
       aux:=aux^.proximo;
       end;
       if achou = 0
            then showmessage('Nenhum aluno reprovado')
            else showmessage('Quantidade de reprovados = '
                           +inttostr(achou));
     end;
end;
```

O código do botão *Consultar Todos os Alunos* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
var achou: integer;
    media: real;
begin
if inicio = nil
```

```
then showmessage('Nenhum aluno foi cadastrado')
else begin
      // desabilita o acesso ao panel1 que possui
      // os botões das operações
      panel1.enabled:=false;
      // mostra o panel5 que possui os componentes para
      // a consulta de alunos
      panel5.visible:=true;
      // limpa o memo que mostrará a consulta
      memo1.clear;
     // esconde o button Nova Consulta
     button13.visible:=false;
     // esconde o button Limpar
     button14.visible:=false;
     // percorre toda a lista
     aux:=inicio;
          while (aux <> nil) do
            begin
              media:=(aux^.nota1+aux^.nota2)/2;
              if media >= 7
              then memo1.lines.add(aux^.nome+'     MÉDIA = '
                                   +floattostr(media)+' APROVADO')
              else memo1.lines.add(aux^.nome+'     MÉDIA = '
                                   +floattostr(media)+' REPROVADO');
            aux:=aux^.proximo;
            end;
        end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Unidade 5

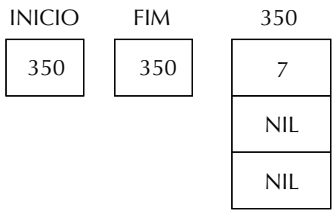
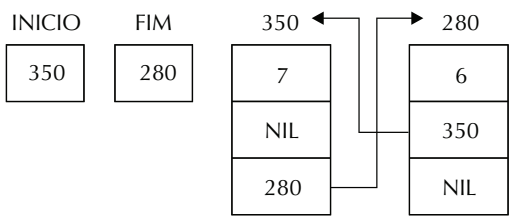
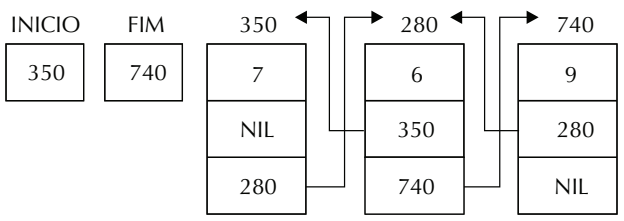
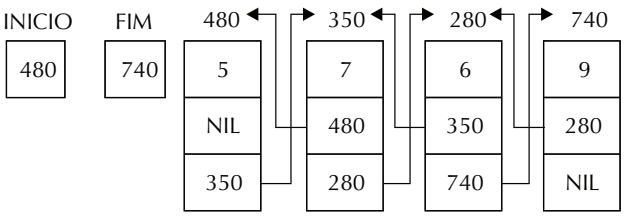
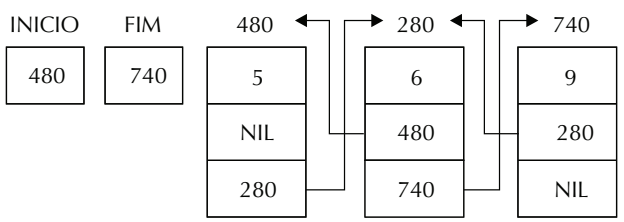
Lista dinâmica duplamente encadeada*

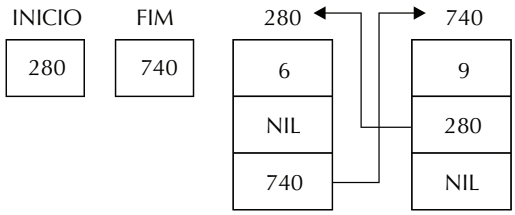
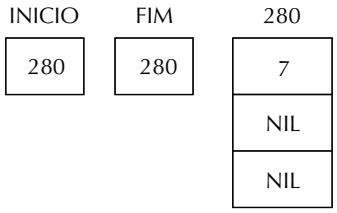
Como sabemos, a lista é uma estrutura de dados que pode armazenar dados ordenados e não ordenados. A idéia fundamental da lista é que todos os seus elementos possam ser acessados. Assim, quando um elemento novo é introduzido na lista, este poderá ser colocado no início, no final ou no meio da lista. É possível fazer as seguintes implementações de uma lista: simplesmente encadeada e não ordenada, simplesmente encadeada e ordenada, duplamente encadeada e não ordenada e duplamente encadeada e ordenada. As listas duplamente encadeadas serão abordadas nesta unidade, e as listas simplesmente encadeadas foram abordadas na Unidade 4. Normalmente, utiliza-se apenas a implementação dinâmica, ou seja, com ponteiros, para lista duplamente encadeada.

A tabela abaixo mostra os passos na execução de uma lista dinâmica duplamente encadeada e não ordenada.

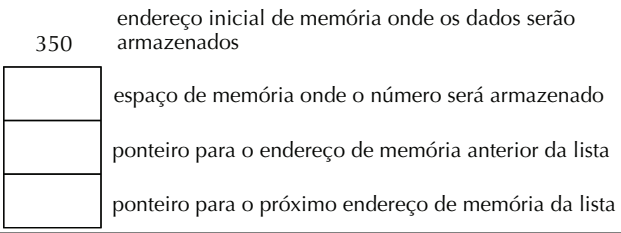
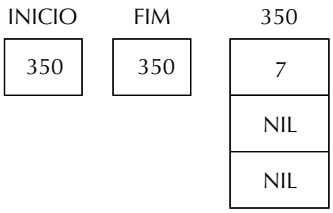
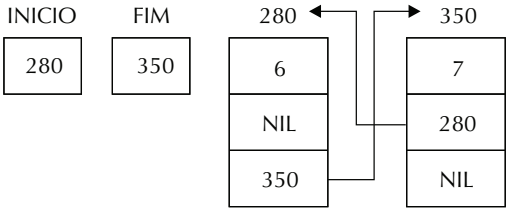
Estrutura de dados do tipo lista duplamente encadeada e não ordenada para armazenar um número. Como a lista não é ordenada, o usuário pode escolher se deseja inserir novos números no início ou no fim da lista.	350 endereço inicial de memória onde os dados serão armazenados espaço de memória onde o número será armazenado ponteiro para o endereço de memória anterior da lista ponteiro para o próximo endereço de memória da lista
Inicialmente a estrutura de dados do tipo lista está vazia, logo, o ponteiro INICIO aponta para vazio.	INICIO NIL

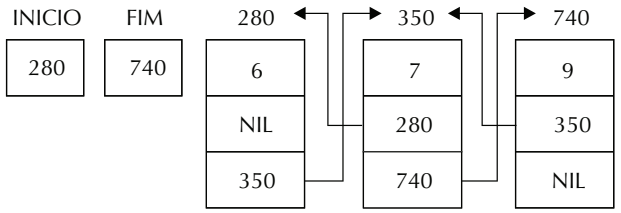
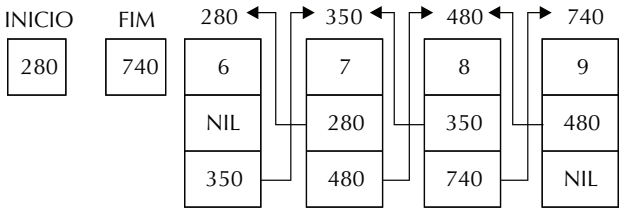
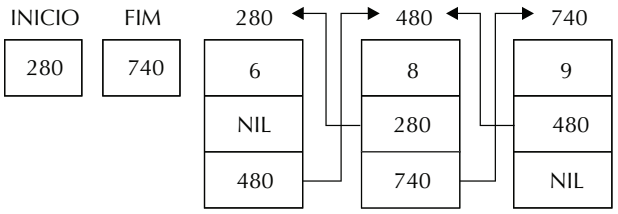
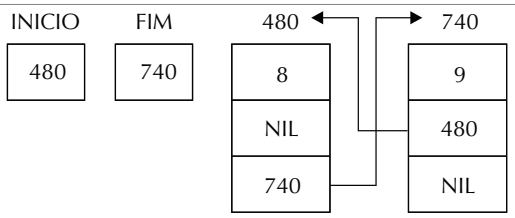
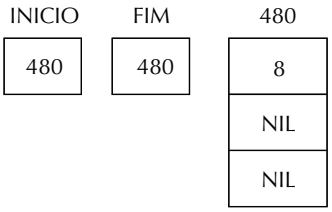
* Esta unidade corresponde ao Capítulo 6 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005

Primeiro elemento inserido na lista. Este terá o ponteiro próximo e o ponteiro anterior apontando para vazio e, por ser o primeiro elemento, os ponteiros INICIO e FIM passam a apontar para a nova posição de memória alocada.	
Segundo elemento inserido na lista. Decide-se fazer a inserção no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória, o ponteiro anterior da nova posição de memória passa a ter o valor do FIM e o FIM passa a ter o valor da nova posição de memória alocada.	
Terceiro elemento inserido na lista. Decide-se fazer a inserção no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória, o ponteiro anterior da nova posição de memória passa a ter o valor do FIM e o FIM passa a ter o valor da nova posição de memória alocada.	
Quarto elemento inserido na lista. Decide-se fazer a inserção no início da lista, assim, o ponteiro anterior do elemento INICIO passa a ter o valor da nova posição de memória, o ponteiro próximo da nova posição de memória passa a ter o valor do INICIO e o INICIO passa a ter o valor da nova posição de memória alocada.	
Qualquer elemento pode ser retirado da lista. Escolhe-se retirar o elemento 7 que está no meio da lista. Os ponteiros INICIO e FIM não mudam, mas o elemento que apontava par a o 7 passa a apontar para o endereço do ponteiro próximo à posição do número 7. O elemento que possuía o ponteiro anterior apontando para o 7 passa a apontador para o ponteiro anterior do número 7.	

Escolhe-se agora retirar o elemento 5 que está no início da lista, logo, o ponteiro INICIO passa a apontar para o endereço do ponteiro próximo à posição do INICIO. O ponteiro anterior do atual INICIO agora apontará para NIL.	
Escolhe-se agora retirar o elemento 9 que está no final da lista, logo, o ponteiro FIM passa a apontar para o endereço anterior do número que será excluído e o ponteiro próximo do número que apontava para o número 9 passa a apontar para NIL.	

A tabela abaixo mostra os passos na execução de uma lista dinâmica duplamente encadeada e ordenada.

Estrutura de dados do tipo lista duplamente encadeada e ordenada para armazenar um número. Como a lista é ordenada, o programa é que decide onde inserir novos números: no início, no fim ou no meio da lista.	
Inicialmente a estrutura de dados do tipo lista está vazia, logo, o ponteiro INICIO aponta para vazio.	
Primeiro elemento inserido na lista. Este terá o ponteiro próximo e o ponteiro anterior apontando para vazio e, por ser o primeiro elemento, os ponteiros INICIO e FIM passam a apontar para a nova posição de memória alocada.	
Segundo elemento inserido na lista, o número 6; logo, a inserção será no início da lista, assim, o ponteiro próximo do novo elemento passa a ter o valor do INICIO, o ponteiro anterior do elemento que era INICIO vai apontar para a nova posição e o INICIO passa a ter o valor da nova posição de memória alocada.	

Terceiro elemento inserido na lista, o número 9; logo a inserção será no final da lista, assim, o ponteiro próximo do elemento FIM passa a ter o valor da nova posição de memória, o ponteiro anterior da nova posição apontará para o FIM e o FIM passa a ter o valor da nova posição de memória alocada.	
Quarto elemento inserido na lista, o número 8; logo, a inserção será no meio da lista, assim, o ponteiro próximo do elemento anterior passa a apontar para a nova posição e o ponteiro próximo da nova posição passa a apontar para o valor do ponteiro próximo da posição anterior , e o valor do ponteiro anterior à posição após o número inserido apontará para a nova posição. O ponteiro anterior da nova posição apontará para o endereço anterior da posição posterior à posição inserida.	
Qualquer elemento pode ser retirado da lista. Escolhe-se retirar o elemento 7 que está no meio da lista. Os ponteiros INICIO e FIM não mudam, mas o elemento que apontava par a o 7 passa a apontar para o endereço do ponteiro próximo da posição do número 7 e o ponteiro anterior ao número, cujo ponteiro próximo do número 7 apontava, apontará para o ponteiro anterior do número 7.	
Escolhe-se agora retirar o elemento 6 que está no início da lista, logo, o ponteiro INICIO passa a apontar para o endereço do ponteiro próximo da posição do INICIO e o ponteiro anterior da nova posição INICIO passa a apontar para NIL.	
Escolhe-se agora retirar o elemento 9 que está no final da lista, logo, o ponteiro FIM passa a apontar para o endereço que apontava para o elemento que será retirado e o ponteiro próximo ao elemento que agora é o FIM passa apontar para NIL.	

Uma lista duplamente encadeada é aquela que pode ser percorrida do primeiro elemento ao último ou do último elemento ao primeiro. As aplicações 19 (lista simplesmente encadeada e não ordenada) e 20 (lista simplesmente encadeada e ordenada) utilizam implementação estática, ou seja, faz-se o uso de vetores

para o armazenamento dos elementos. Para que essas listas sejam duplamente encadeadas, basta implementar uma operação que consulte a lista do último elemento ao primeiro. A seguir está a implementação da consulta que poderia ser utilizada nas aplicações 19 e 20.

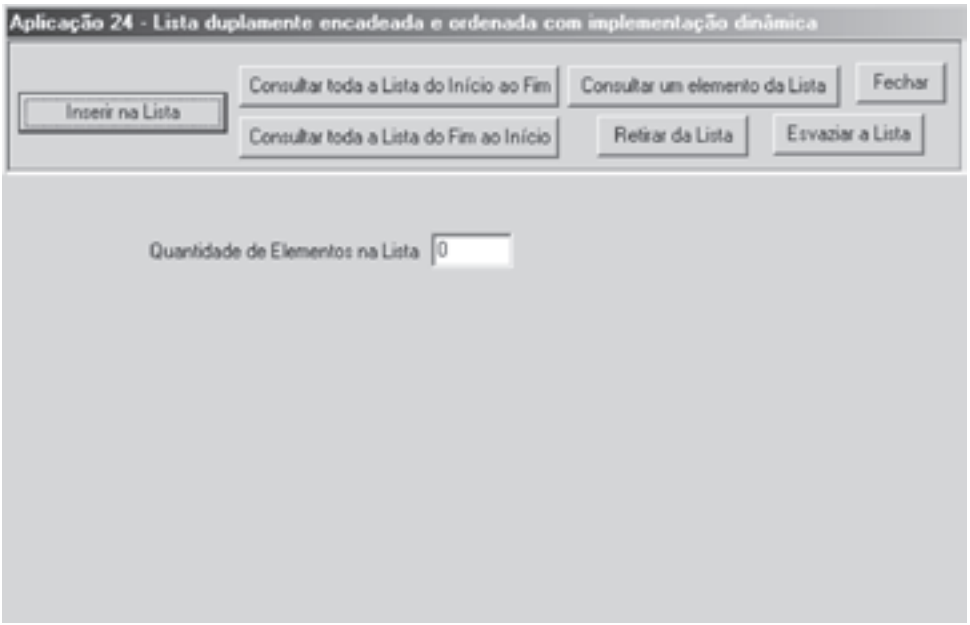
O código da *Consulta dos Elementos de uma Lista do Último ao Primeiro* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
    i:integer;
begin
if qtde = 0
then showmessage('A lista está vazia')
else begin
      texto:='';
      for i:= qtde downto 1 do
      begin
        texto:=texto+ '   '+inttostr(lista[i]);
      end;
      showmessage('A lista toda do último ao primeiro: '+texto);
     end;
end;
```

Implementação de uma lista dinâmica duplamente encadeada e não ordenada

A Figura 5.1 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista duplamente encadeada e não ordenada: insere no início da lista, insere no final da lista, retira da lista, consulta um elemento da lista, consulta toda a lista do início ao fim, consulta toda a lista do fim ao início e esvazia a lista. Todas essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho da memória. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 5.1 Tela da aplicação com implementação dinâmica de uma lista duplamente encadeada e não ordenada



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da lista;
// a variável NUM armazenará um número inteiro;
// a variável PROXIMO é um ponteiro para o próximo endereço
// de memória do tipo REGISTRO;
// a variável ANTERIOR é um ponteiro para o endereço anterior
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo, anterior: pointer;
     end;

// as variáveis LISTA, INICIO, FIM, AUX e AUX2 são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável QTDE armazena a quantidade de números
// armazenados na lista

var Form1: TForm1;
    lista, inicio, fim, aux, aux2: ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
```

```
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro INICIO aponta para vazio, pois a lista está vazia
inicio:=nil;
end;
```

O código do botão *Inserir no Início da Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
num:=inputbox('Lista duplamente encadeada e não ordenada com
              implementação dinâmica','Digite o número a ser inserido
              no início da lista','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(lista);
      if inicio = nil
      then begin
           // a lista está vazia e
           // este número será o primeiro a ser inserido
           lista^.num := numero;
           lista^.proximo:=nil;
           lista^.anterior:=nil;
           inicio:=lista;
           fim:=lista;
          end
      else begin
          // já existe algum número na lista
          // e o novo número será inserido no início
          lista^.num:= numero;
          lista^.proximo:=inicio;
          inicio^.anterior:=lista;
          lista^.anterior:=nil;
          inicio:=lista;
          end;
      qtde:=qtde+1;
      edit1.text:=inttostr(qtde);
      showmessage('O número '+ num+ ' foi inserido com sucesso no
                  início da lista');
      end;
end;
```

O código do botão *Inserir no Final da Lista* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
var num: string;
```

```
    numero: integer;
begin
num:=inputbox('Lista duplamente encadeada e não ordenada com
             implementação dinâmica','Digite o número a ser inserido
             no final da lista','');
if num <> ''
 then begin
     numero:=strtoint(num);
     new(lista);
     if inicio = nil
     then begin
          // a lista está vazia
          // este número será o primeiro a ser inserido
          lista^.num := numero;
          lista^.proximo:=nil;
          lista^.anterior:=nil;
          inicio:=lista;
          fim:=lista;
         end
     else begin
         // já existe algum número na lista
         // e o novo número será inserido no final
         lista^.num:= numero;
         lista^.proximo:=nil;
         lista^.anterior:=fim;
         fim^.proximo:=lista;
         fim:=lista;
         end;
     qtde:=qtde+1;
     edit1.text:=inttostr(qtde);
     showmessage('O número '+ num+ ' foi inserido com sucesso no
                 final da lista');
     end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
     // a variável ACHOU verifica se
     // o número procurado existe na lista
     achou:=0;
     num:=inputbox('Lista duplamente encadeada e não ordenada com
                   implementação dinâmica','Digite o número a ser
```

```
                    procurado na lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             aux:=inicio;
             while (aux <> nil) do
             begin
               if aux^.num = numero
               then achou:=achou+1;
             aux:=aux^.proximo;
             end;
             if achou = 0
             then showmessage('O número '+num+' não foi encontrado na
                              lista')
             else if achou = 1
                  then showmessage('O número '+num+' foi encontrado'
                                   +inttostr(achou)+' vez')
                  else showmessage('O número '+num+' foi encontrado'
                                   +inttostr(achou)+' vezes');
           end;
    end;
end;
```

O código do botão *Consultar Toda a Lista do Início ao Fim* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
begin
if inicio =  nil
then showmessage('A lista está vazia')
else begin
     texto:='';
     aux:=inicio;
     while (aux <> nil) do
     begin
       texto:=texto+ '   '+inttostr(aux^.num);
       aux:=aux^.proximo;
     end;
     showmessage('A lista do início ao fim: '+texto);
    end;
end;
```

O código do botão *Consultar Toda a Lista do Fim ao Início* é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
var texto:string;
begin
if inicio =  nil
then showmessage('A lista está vazia')
```

```
else begin
     texto:='';
     aux:=fim;
     while (aux <> nil) do
     begin
       texto:=texto+ '   '+inttostr(aux^.num);
       aux:=aux^.anterior;
     end;
     showmessage('A lista do fim ao início: '+texto);
    end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
     // será necessário encontrar o número na lista;
     // a variável ACHOU marca se o número a ser retirado
     // da lista foi encontrado
     achou:=0;
     num:=inputbox('Lista duplamente encadeada e não ordenada com
                  implementação dinâmica','Digite o número a ser
                  retirado da lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             aux:=inicio;
             while (aux <> nil) do
             begin
               if aux^.num = numero
                 then begin
                      achou:=1;
                      qtde:=qtde-1;
                      edit1.text:=inttostr(qtde);
                      // se o número a ser excluído for o primeiro
                      if aux = inicio
                       then begin
                            inicio:=aux^.proximo;
                            inicio^.anterior:=nil;
                            dispose(aux);
                            aux:=inicio;
                            end
                       else if aux = fim
                            // se o número a ser excluído
```

```
                            // for o último
                            then begin
                                 aux2:=inicio;
                                 while (aux2^.proximo <> aux) do
                                 begin
                                   aux2:=aux2^.proximo;
                                 end;
                                 aux2^.proximo:=nil;
                                 dispose(fim);
                                 fim:=aux2;
                                 aux:=nil;
                                 end
                            else begin
                                 // se o número a ser excluído
                                 // estiver no meio da lista
                                 aux2:=inicio;
                                 while (aux2^.proximo <> aux) do
                                 begin
                                   aux2:=aux2^.proximo;
                                 end;
                                 aux2^.proximo:=aux^.proximo;
                                 aux2:=inicio;
                                 while (aux2^.anterior <> aux) do
                                 begin
                                   aux2:=aux2^.proximo;
                                 end;
                                  aux2^.anterior:=aux^.anterior;
                                 dispose(aux);
                                 aux:=aux2;
                                 end;
                       end
              else aux:=aux^.proximo;
              end;
     if achou = 1
     then showmessage('O número '+ inttostr(numero)+ ' foi retirado
                      com sucesso da lista')
     else showmessage('O número '+ inttostr(numero)+ ' não foi
                      encontrado na lista');
    end;
  end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if inicio = nil
then showmessage('A lista já está vazia')
else begin
```

```
      aux:=inicio;
      while (aux <> nil) do
      begin
        inicio:=inicio^.proximo;
        dispose(aux);
        aux:=inicio;
      end;
      showmessage('A lista foi esvaziada com sucesso');
      qtde:=0;
      edit1.text:=inttostr(qtde);
     end;
end;
```

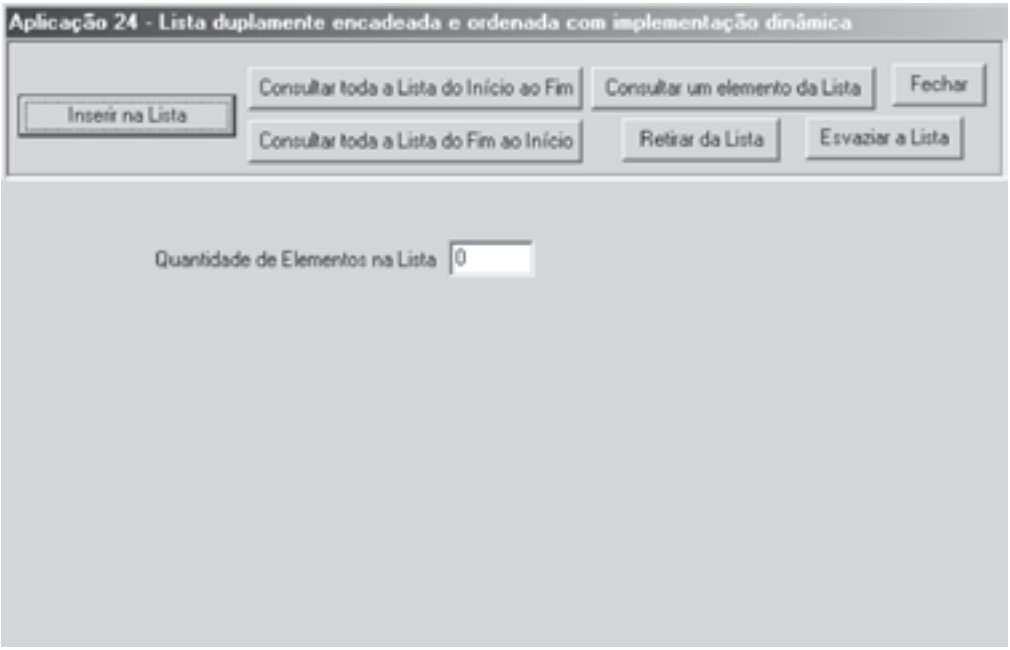
O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Implementação de uma lista dinâmica duplamente encadeada e ordenada

A Figura 5.2 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo lista duplamente encadeada e ordenada: insere na lista, retira da lista, consulta um elemento da lista, consulta toda a lista do início ao fim, consulta toda a lista do fim ao início e esvazia a lista. Todas essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação, a quantidade de dados que podem ser colocados na lista limita-se ao tamanho da memória. Após a figura têm-se os seus respectivos códigos comentados.

Figura 5.2 Tela da aplicação com implementação dinâmica de uma lista duplamente encadeada e ordenada



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada posição da lista;
// a variável NUM armazenará um número inteiro;
// a variável PROXIMO é um ponteiro para o próximo endereço
// de memória do tipo REGISTRO;
// a variável ANTERIOR é um ponteiro para o endereço anterior
// de memória do tipo REGISTRO

type registro = record
      num: integer;
      proximo, anterior: pointer;
     end;

// as variáveis LISTA, INICIO, FIM, AUX e AUX2 são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente;
// a variável QTDE armazena a quantidade de números
// armazenados na lista

var Form1: TForm1;
    lista, inicio, fim, aux, aux2: ^registro;
    qtde:integer;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
```

```
qtde:=0;
edit1.text:=inttostr(qtde);
// o ponteiro INICIO aponta para vazio pois a lista está vazia
inicio:=nil;
end;
```

O código do botão *Inserir na Lista* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero: integer;
begin
num:=inputbox('Lista duplamente encadeada e ordenada com implementação
              dinâmica','Digite o número a ser inserido na
              lista','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(lista);
      if inicio = nil
      then begin
           // a lista está vazia e
           // este número será o primeiro a ser inserido
           lista^.num := numero;
           lista^.proximo:=nil;
           lista^.anterior:=nil;
           inicio:=lista;
           fim:=lista;
          end
      else if (numero <= inicio^.num)
           then begin
                // já existe algum número na lista
                // e o novo número é menor que o primeiro da lista,
                // logo, será inserido no início da lista
                lista^.num:= numero;
                lista^.proximo:=inicio;
                inicio^.anterior:=lista;
                lista^.anterior:=nil;
                inicio:=lista;
                end
           else if (numero >= fim^.num)
                 then begin
                      // já existe algum número na lista
                      // e o novo número é maior
                      // que o último da lista
                      // logo, será inserido no final da lista
                      lista^.num:= numero;
                      lista^.proximo:=nil;
                      lista^.anterior:=fim;
                      fim^.proximo:=lista;
                      fim:=lista;
                      end
                 else begin
                      // já existe algum número na lista
```

```
                    // e o novo número será inserido
                    // na posição ordenada
                    aux:=inicio;
                    while (aux^.num < numero) do
                    begin
                    aux:=aux^.proximo;
                    end;
                    lista^.num:= numero;
                    lista^.proximo:=aux;
                    aux^.anterior:=lista;
                    aux2:=inicio;
                    while (aux2^.proximo <> aux) do
                    begin
                    aux2:=aux2^.proximo;
                    end;
                    lista^.anterior:=aux2;
                    aux2^.proximo:=lista;
                    end;
     qtde:=qtde+1;
     edit1.text:=inttostr(qtde);
     showmessage('O número '+ num+ ' foi inserido com sucesso na
                 lista');
     end;
end;
```

O código do botão *Consultar um Elemento da Lista* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
     // a variável ACHOU verifica se
     // o número procurado existe na lista
     achou:=0;
     num:=inputbox('Lista duplamente encadeada e ordenada com
                  implementação dinâmica','Digite o número a ser
                  procurado na lista','');
     if num <> ''
        then begin
             numero:=strtoint(num);
             aux:=inicio;
             while (aux <> nil) and (aux^.num <= numero) do
             begin
               if aux^.num = numero
               then achou:=achou+1;
             aux:=aux^.proximo;
             end;
             if achou = 0
             then showmessage('O número '+num+' não foi encontrado na
                             lista')
             else if achou = 1
                  then showmessage('O número '+num+' foi encontrado'
```

```
                                    +inttostr(achou)+' vez')
                  else showmessage('O número '+num+' foi encontrado'
                                    +inttostr(achou)+' vezes');
            end;
     end;
end;
```

O código do botão *Consultar toda a Lista do Início ao Fim* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
var texto:string;
begin
if inicio =  nil
then showmessage('A lista está vazia')
else begin
      texto:='';
      aux:=inicio;
      while (aux <> nil) do
      begin
        texto:=texto+ '   '+inttostr(aux^.num);
        aux:=aux^.proximo;
      end;
      showmessage('A lista do início ao fim: '+texto);
     end;
end;
```

O código do botão *Consultar toda a Lista do Fim ao Início* é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
var texto:string;
begin
if inicio =  nil
then showmessage('A lista está vazia')
else begin
      texto:='';
      aux:=fim;
      while (aux <> nil) do
      begin
        texto:=texto+ '   '+inttostr(aux^.num);
        aux:=aux^.anterior;
      end;
      showmessage('A lista do fim ao início: '+texto);
     end;
end;
```

O código do botão *Retirar da Lista* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var numero, achou: integer;
    num: string;
begin
if inicio = nil
then showmessage('A lista está vazia')
else begin
```

```
// será necessário encontrar o número na lista;
// a variável ACHOU marca se o número a ser retirado
// da lista foi encontrado
achou:=0;
num:=inputbox('Lista duplamente encadeada e ordenada com
              implementação dinâmica','Digite o número a ser
              retirado da lista','');
if num <> ''
   then begin
        numero:=strtoint(num);
        aux:=inicio;
        while (aux <> nil) do
        begin
          if aux^.num = numero
            then begin
                 achou:=1;
                 qtde:=qtde-1;
                 edit1.text:=inttostr(qtde);
                 // se o número a ser excluído for o primeiro
                 if aux = inicio
                  then begin
                       inicio:=aux^.proximo;
                       inicio^.anterior:=nil;
                       dispose(aux);
                       aux:=inicio;
                       end
                  else if aux = fim
                       // se o número a ser excluído
                       // for o último
                       then begin
                            aux2:=inicio;
                            while (aux2^.proximo <> aux) do
                            begin
                              aux2:=aux2^.proximo;
                            end;
                            aux2^.proximo:=nil;
                            dispose(fim);
                            fim:=aux2;
                            aux:=nil;
                            end
                       else begin
                            // se o número a ser excluído
                            // estiver no meio da lista
                            aux2:=inicio;
                            while (aux2^.proximo <> aux) do
                            begin
                              aux2:=aux2^.proximo;
                            end;
                            aux2^.proximo:=aux^.proximo;
                            aux2:=inicio;
                            while (aux2^.anterior <> aux) do
                            begin
                              aux2:=aux2^.proximo;
                            end;
```

```
                                  aux2^.anterior:=aux^.anterior;
                                dispose(aux);
                                aux:=aux2;
                                end;
                    end
            else aux:=aux^.proximo;
            end;
     if achou = 1
     then showmessage('O número '+ inttostr(numero)+ ' foi retirado
                      com sucesso da lista')
     else showmessage('O número '+ inttostr(numero)+ ' não foi
                      encontrado na lista');
    end;
  end;
end;
```

O código do botão *Esvaziar a Lista* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
if inicio = nil
then showmessage('A lista já está vazia')
else begin
     aux:=inicio;
     while (aux <> nil) do
     begin
       inicio:=inicio^.proximo;
       dispose(aux);
       aux:=inicio;
     end;
     showmessage('A lista foi esvaziada com sucesso');
     qtde:=0;
     edit1.text:=inttostr(qtde);
    end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

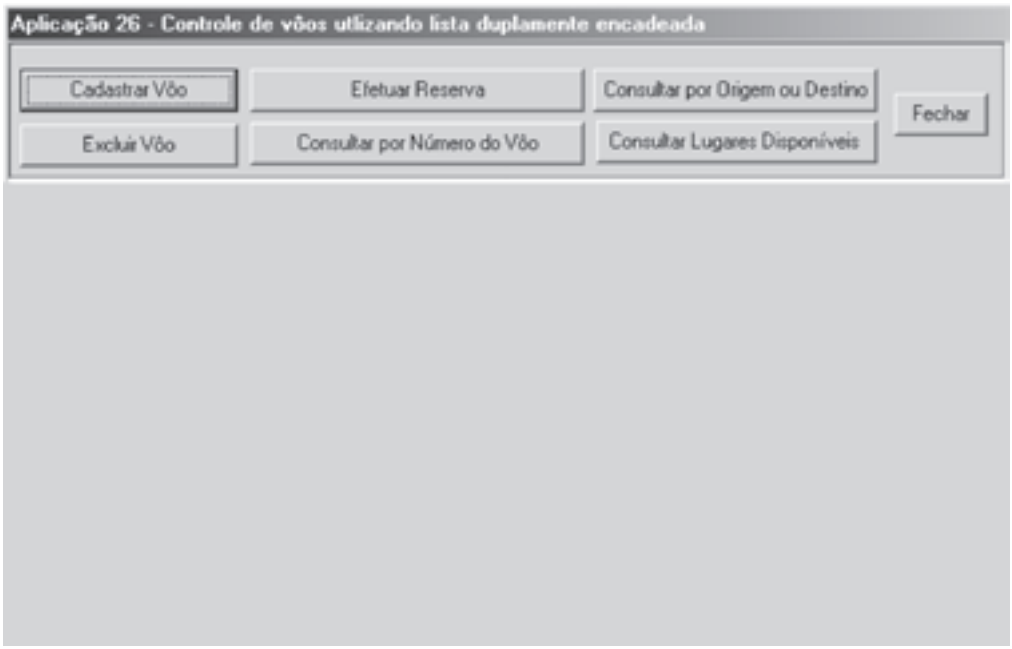
```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Aplicação de controle de vôos usando lista dinâmica duplamente encadeada e não ordenada

A Figura 5.3 mostra a tela da aplicação de um controle de vôos utilizando uma estrutura de dados do tipo lista duplamente encadeada e não ordenada com implementação dinâmica. Nesta aplicação serão cadastrados os vôos, não sendo permitido o cadastro de mais de um vôo com o mesmo número,

excluídos vôos já cadastrados, efetivação de reservas em vôos; algumas consultas implementadas são: consulta por número do vôo, consulta de vôos por origem, consulta de vôos por destino e consulta de vôos com lugares disponíveis. Após a figura têm-se seus respectivos códigos comentados.

Figura 5.3 Tela da aplicação de controle de vôos utilizando lista duplamente encadeada e não ordenada com implementação dinâmica



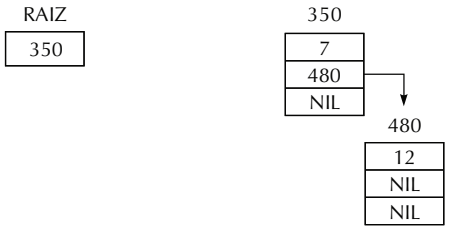
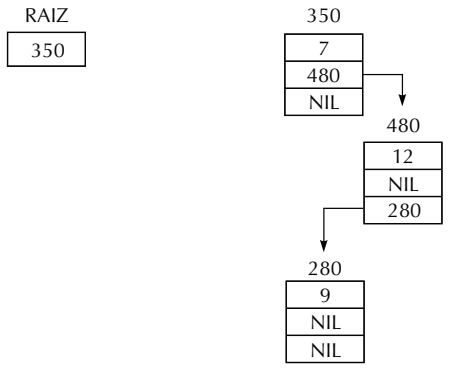
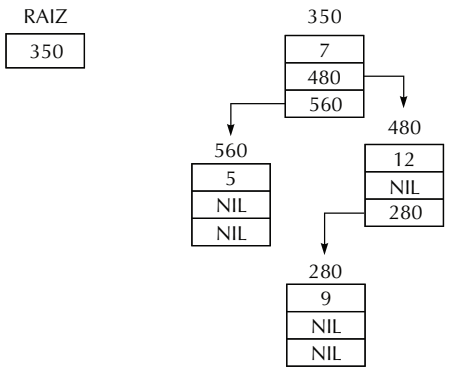
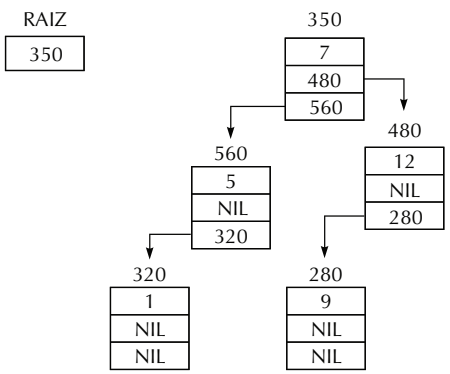
Unidade 6

Árvore binária*

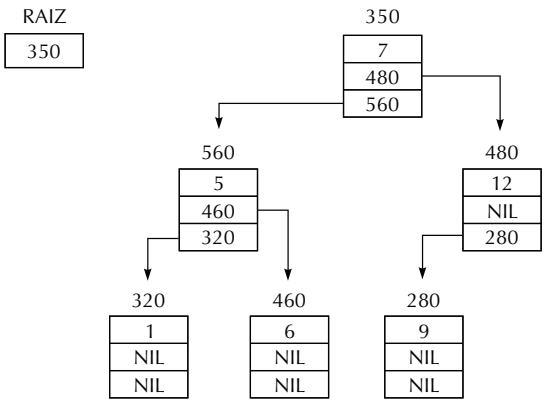
A árvore binária é uma estrutura de dados que preferencialmente armazena dados ordenados; sua idéia fundamental é que todos os seus elementos possam ser acessados. Assim, quando um elemento novo é introduzido na árvore, este é colocado à direita da raiz, se este for maior que a raiz, ou à esquerda da raiz, se for menor que a raiz. A raiz é o primeiro elemento inserido. A estrutura de dados do tipo árvore binária utiliza principalmente implementação dinâmica, ou seja, ponteiros. A tabela abaixo mostra os passos na execução de uma árvore binária dinâmica.

Estrutura de dados do tipo árvore binária para armazenar um número.	350 — endereço inicial de memória onde os dados serão armazenados espaço de memória onde o número será armazenado ponteiro para o ramo direito da árvore ponteiro para o ramo esquerdo da árvore
Inicialmente, a estrutura de dados do tipo árvore binária está vazia, logo, o ponteiro RAIZ aponta para vazio.	RAIZ: NIL
Primeiro elemento inserido na árvore. Decide-se fazer a inserção do número 7 e este terá o ponteiro direito e o ponteiro esquerdo apontando para vazio. Por ser o primeiro elemento inserido, o ponteiro RAIZ passa a apontar para a nova posição de memória alocada.	RAIZ: 350 350: 7, NIL, NIL

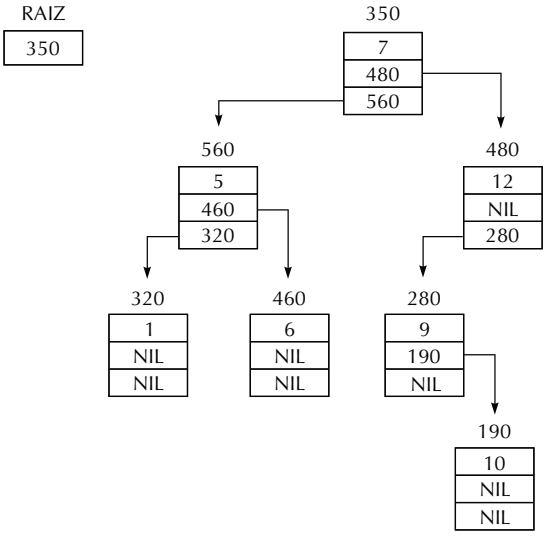
* Esta unidade corresponde ao Capítulo 2 de ASCENCIO, Ana Fernanda Gomes. *Aplicações das estruturas de dados*. São Paulo: Pearson/Prentice Hall, 2005.

Segundo elemento inserido na árvore. Decide-se fazer a inserção do número 12. Como o número 12 é maior que o número da RAIZ, este será incluído à direita da árvore, logo, o ponteiro à direita da RAIZ apontará para o valor da nova posição de memória.	
Terceiro elemento inserido na árvore. Decide-se fazer a inserção do número 9. Como o número 9 é maior que o número da RAIZ, este será incluído à direita da árvore. Caminha-se na árvore para o endereço do ponteiro à direita da RAIZ. Como o número 9 é menor que o número 12, o ponteiro à esquerda do número 12 apontará para o valor da nova posição de memória.	
Quarto elemento inserido na árvore. Decide-se fazer a inserção do número 5. Como o número 5 é menor que o número da RAIZ, este será incluído à esquerda da árvore, logo, o ponteiro à esquerda da RAIZ apontará para o valor da nova posição de memória.	
Quinto elemento inserido na árvore. Decide-se fazer a inserção do número 1. Como o número 1 é menor que o número da RAIZ, este será incluído à esquerda da árvore. Caminha-se na árvore para o endereço do ponteiro à esquerda da RAIZ. Como o número 1 é menor que o número 5, este será incluído à esquerda do número 5, logo, o ponteiro à esquerda do número 5 apontará para o valor da nova posição de memória.	

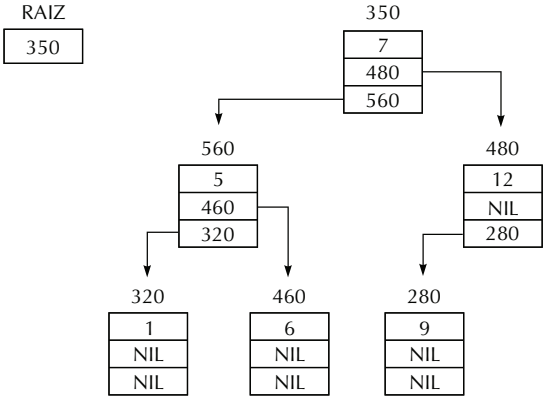
Sexto elemento inserido na árvore. Decide-se fazer a inserção do número 6. Como o número 6 é menor que o número da RAIZ, este será incluído à esquerda da árvore. Caminha-se na árvore para o endereço do ponteiro à esquerda da RAIZ. Como o número 6 é maior que o número 5, este será incluído à direita do número 5, logo, o ponteiro à direita do número 5 apontará para o valor da nova posição de memória.

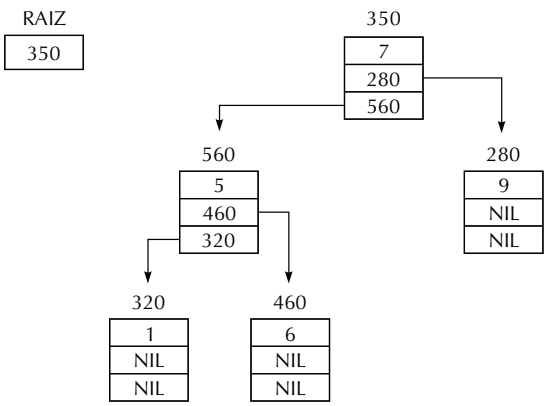
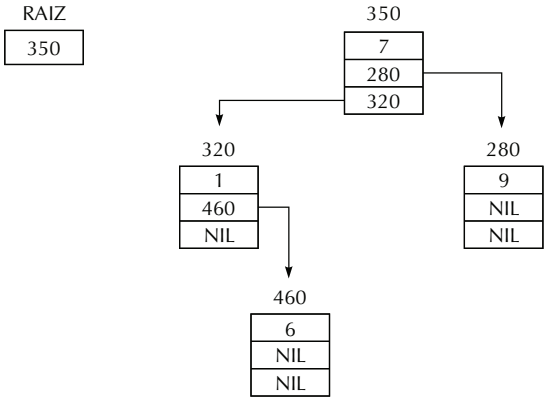


Sétimo elemento inserido na árvore. Decide-se fazer a inserção do número 10. Como o número 10 é maior que o número da RAIZ, este será incluído à direita da árvore. Caminha-se na árvore para o endereço do ponteiro à direita da RAIZ. Como o número 10 é menor que o número 12, este será incluído à esquerda do número 12. Caminha-se na árvore para o endereço do ponteiro à esquerda do número 12. Como o número 10 é maior que o número 9, este será incluído à direita do número 9, logo, o ponteiro à direita do número 9 apontará para o valor da nova posição de memória.



Escolhe-se fazer a exclusão do número 10 que é um nó-folha, ou seja, um número que não aponta para nenhum outro número. Logo, quem apontava para ele passa a apontar para vazio.

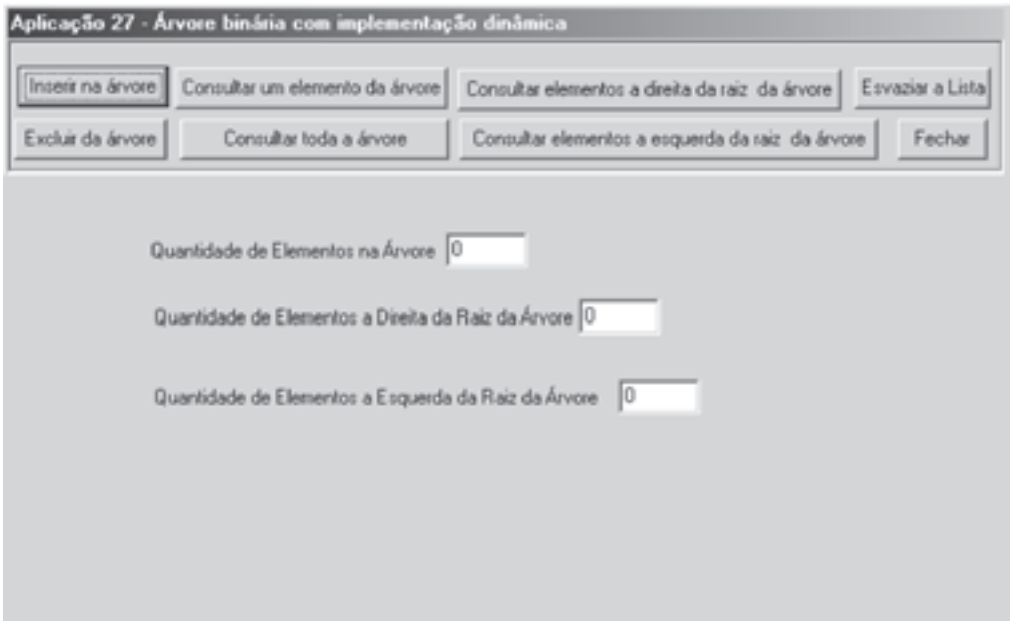


Escolhe-se fazer a exclusão do número 12 que possui apontador apenas para a esquerda. Logo, quem apontava para ele passa a apontar para o seu apontador para a esquerda. Ou seja, o ponteiro para a direita do número 7 que apontava para o número 12 vai passar a apontar para o número 9 que é o ponteiro para a esquerda do número 12.	
Escolhe-se fazer a exclusão do número 5 que é um nó que possui apontadores para a direita e para a esquerda. Logo, quem apontava para ele passa a apontar para o ramo da esquerda do nó excluído, e o ramo da direita do nó excluído será reposicionado.	

Implementação de uma árvore binária dinâmica

A Figura 6.1 mostra a tela da aplicação que implementa as seguintes operações em uma estrutura de dados do tipo árvore binária: insere na árvore, retira da árvore, consulta um elemento na árvore, consulta toda a árvore, consulta elementos à direita da raiz da árvore, consulta elementos à esquerda da raiz da árvore e esvazia a árvore. Essas operações serão realizadas pela implementação dinâmica, ou seja, utilizando-se ponteiros; logo, nesta aplicação a quantidade de dados que podem ser colocados na árvore limita-se ao tamanho da memória. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 6.1 Tela da aplicação com implementação de uma árvore binária dinâmica



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada nó da árvore;
// a variável NUM armazenará um número inteiro;
// a variável DIREITA é um ponteiro para um endereço
// de memória do tipo REGISTRO;
// a variável ESQUERDA é um ponteiro para um endereço
// de memória do tipo REGISTRO;
// definição de um novo tipo que será um apontador para
// a estrutura da árvore;
// APONTADOR será utilizado na função recursiva
// para percorrer a árvore;

type apontador = ^registro;
     registro = record
        num: integer;
        direita, esquerda: apontador;
     end;

// as variáveis ARVORE, AUX, AUX1, AUX2, ANTERIOR e RAIZ são ponteiros
// para endereços de memória do tipo REGISTRO definido anteriormente;
// a variável QTDE1 armazena a quantidade de números
// armazenados na árvore;
// a variável QTDE2 armazena a quantidade de números
// armazenados à direita da raiz da árvore;
// a variável QTDE3 armazena a quantidade de números
// armazenados à esquerda da raiz da árvore;

var Form1: TForm1;
```

```
    arvore, aux, aux1, aux2, anterior, raiz : apontador;
    qtde1, qtde2, qtde3 : integer;
    texto: string;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

```
procedure TForm1.FormActivate(Sender: TObject);
begin
qtde1:=0;
qtde2:=0;
qtde3:=0;
edit1.text:=inttostr(qtde1);
edit2.text:=inttostr(qtde2);
edit3.text:=inttostr(qtde3);
// o ponteiro RAIZ aponta para vazio, pois a árvore está vazia
raiz:=nil;
end;
```

O código do botão *Inserir na Árvore* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
var num: string;
    numero, achou: integer;
begin
num:=inputbox('Árvore binária com implementação dinâmica','Digite o
              número a ser inserido na árvore','');
if num <> ''
 then begin
      numero:=strtoint(num);
      new(arvore);
      arvore^.num:= numero;
      arvore^.direita:=nil;
      arvore^.esquerda:=nil;
      if (raiz = nil)
      then raiz := arvore
      else begin
           aux := raiz;
           achou := 0;
           while (achou = 0) do
            begin
            if arvore^.num <= aux^.num
            then begin
                 // o número será inserido à esquerda da árvore
                 if aux^.esquerda = nil
                      then begin
                             aux^.esquerda:= arvore;
                             achou := 1;
                           end
                      else aux := aux^.esquerda;
                 end
            else if arvore^.num > aux^.num
                   then begin
                        // o número será inserido à direita da árvore
                        if aux^.direita = nil
                             then begin
```

```
                                aux^.direita:= arvore;
                                achou := 1;
                                end
                           else aux := aux^.direita;
                      end
           end;
          end;
     showmessage('Número inserido na árvore com sucesso');
     qtde1:=qtde1+1;
     edit1.text:=inttostr(qtde1);
     if numero > raiz^.num
     then qtde2:=qtde2+1;
     edit2.text:=inttostr(qtde2);
     if numero < raiz^.num
     then qtde3:=qtde3+1;
     edit3.text:=inttostr(qtde3);
     end;
end;
```

O código do botão *Excluir da Árvore* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
var num:string;
    numero, achou:integer;
begin
if raiz = nil
then showmessage('A árvore está vazia')
else begin
     num:=inputbox('Árvore binária com implementação dinâmica','Digite
                  o número a ser excluído da árvore','');
     if num <> ''
     then begin
          numero:=strtoint(num);
          // a busca pelo número a ser excluído
          // começa na raiz
          aux := raiz;
          // a variável ACHOU marca se o número
          // a ser excluído está na árvore
          achou := 0;
          while (achou = 0) and (aux <> nil) do
          begin
          if aux^.num = numero
          then achou := 1
          else if aux^.num > numero
               then begin
                    // o número está à esquerda da árvore
                    aux := aux^.esquerda
                    end
               else begin
                    // o número está à direita da árvore
                    aux := aux^.direita;
                    end;
          end;
          if achou = 0
          then showmessage('O número não foi encontrado na árvore')
```

```
          else begin
               if aux <> raiz
                then begin
                     // acertando as quantidades à esquerda e à
                     // direita da árvore
                     if numero > raiz^.num
                     then qtde2:=qtde2-1;
                     if numero < raiz^.num
                     then qtde3:=qtde3-1;
                     // o número foi encontrado e será excluído;
                     // como não é a raiz
                     // é necessário encontrar o anterior
                     // para acertar os ponteiros
                     anterior := raiz;
                     while (anterior^.direita <> aux)
                     and (anterior^.esquerda <> aux) do
                     begin
                      if anterior^.num > numero
                      then begin
                           // o número está à esquerda da árvore
                           anterior := anterior^.esquerda;
                           end
                      else begin
                              // o número está à direita da árvore
                              anterior := anterior^.direita;
                            end;
                      end;
                     if (aux^.direita = nil) and (aux^.esquerda = nil)
                     then begin
                          // um número folha será excluído;
                          // acertando os ponteiros do nó anterior
                          if anterior^.direita = aux
                          then anterior^.direita := nil
                          else anterior^.esquerda := nil;
                          // apagando o número da árvore
                          dispose(aux);
                          end
                    else begin
                       // um número não folha será excluído;
                       // um número que possui filhos
                       // apenas para a direita
                       if (aux^.direita <> nil)
                       and (aux^.esquerda = nil)
                       then begin
                            if anterior^.esquerda = aux
                            then anterior^.esquerda := aux^.direita
                            else anterior^.direita := aux^.direita;
                            // apagando o número da árvore
                            dispose(aux);
                            end
                        // um número não folha será excluído;
                        // um número que possui filhos
                        // apenas para a esquerda
                        else if (aux^.esquerda <> nil)
                         and (aux^.direita = nil)
```

```
                        then begin
                             if anterior^.esquerda = aux
                             then anterior^.esquerda := aux^.esquerda
                             else anterior^.direita := aux^.esquerda;
                             // apagando o número da árvore
                             dispose(aux);
                             end
                             // um número não folha será excluído;
                             // um número que possui filhos
                             // para a esquerda e para a direita
                        else if (aux^.esquerda <> nil)
                             and (aux^.direita <> nil)
                             then begin
                                  if anterior^.direita = aux
                                  then begin
                                       anterior^.direita := aux^.direita;
                                       aux1 := aux^.esquerda;
                                       end
                                  else begin
                                       anterior^.esquerda := aux^.esquerda;
                                       aux1 := aux^.direita;
                                       end;
                                  // apagando o número da árvore
                                  dispose(aux);
                                  // recolocando o
                                  // pedaço da árvore
                                  aux := anterior;
                                  while (aux <> nil) do
                                  begin
                                  if aux^.num < aux1^.num
                                  then begin
                                       if aux^.direita = nil
                                       then begin
                                              aux^.direita := aux1;
                                              aux := nil;
                                            end
                                        else aux := aux^.direita;
                                        end
                                    else if aux^.num > aux1^.num
                                         then begin
                                              if aux^.esquerda = nil
                                              then begin
                                                   aux^.esquerda := aux1;
                                                   aux := nil;
                                                   end
                                              else aux := aux^.esquerda;
                                              end;
                                    end;
                                    end;
                    end;
                end
else begin
          // o número a ser excluído é a raiz
      if (aux^.direita = nil)
      and (aux^.esquerda = nil)
```

```
          then begin
               // a raiz não tem filhos e será excluída
               raiz := nil;
               // apagando o número;
               dispose(aux);
              end
     else begin
          // a raiz será excluída
          // e possui filhos apenas para a direita
          if (aux^.direita <> nil)
           and (aux^.esquerda = nil)
          then begin
               raiz := aux^.direita;
               // apagando o número
               dispose(aux);
               end
          else if (aux^.esquerda <> nil)
                and (aux^.direita = nil)
                then begin
                     // a raiz será excluída
                     // e possui filhos apenas para a esquerda
                     raiz := aux^.esquerda;
                     // apagando o número
                     dispose(aux);
                     end
                else if (aux^.esquerda <> nil)
                     and (aux^.direita <> nil)
                     then begin
                          // a raiz será excluída
                          // e possui filhos para a direita
                          // e para a esquerda
                          raiz := aux^.direita;
                          aux1 := aux^.esquerda;
                          // apagando o número
                          dispose(aux);
                          // recolocando o pedaço da árvore
                          aux := raiz;
                          while (aux <> nil) do
                          begin
                          if aux^.num < aux1^.num
                          then begin
                               if aux^.direita = nil
                                then begin
                                     aux^.direita := aux1;
                                     aux := nil;
                                    end
                                    else aux := aux^.direita;
                                   end
                           else if aux^.num > aux1^.num
                                 then begin
                                      if aux^.esquerda = nil
                                      then begin
                                           aux^.esquerda := aux1;
                                           aux := nil;
```

```
                                        end
                                    else aux := aux^.esquerda;
                                   end;
                               end;
                             end;
            end;
    end;
    showmessage('Número excluído com sucesso');
     qtde1:=qtde1-1;
     edit1.text:=inttostr(qtde1);
     edit2.text:=inttostr(qtde2);
     edit3.text:=inttostr(qtde3);
    end;
   end;
   end;
end;
```

O código do botão *Consultar um Elemento da Árvore* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
var num: string;
    numero, achou: integer;
begin
if raiz = nil
then showmessage('A árvore está vazia')
else begin
     num:=inputbox('Árvore binária com implementação dinâmica','Digite
                 o número a ser localizado na árvore','');
     if num <> ''
     then begin
          numero:=strtoint(num);
          // a busca pelo número a ser localizado
          // começa na raiz
          aux := raiz;
          // a variável ACHOU marca se o número
          // a ser localizado está na árvore
          achou := 0;
          while (achou = 0) and (aux <> nil) do
          begin
            if aux^.num = numero
              then achou := 1
              else if aux^.num > numero
              then  begin
                 // o número está à esquerda da árvore
                 aux := aux^.esquerda;
              end
          else  begin
                 // o número está à direita da árvore
                 aux := aux^.direita;
              end;
          end;
          if achou = 0
```

```
          then showmessage('O número não foi encontrado na árvore')
          else showmessage('O número foi encontrado na árvore');
        end;
     end;
end;
```

O código do botão *Consultar Toda a Árvore* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
texto:='';
if raiz = nil
  then showmessage('A árvore está vazia')
  else begin
       aux2 := raiz;
       mostrartudo(aux2);
       showmessage('Toda a árvore = '+texto);
      end;
end;

Procedure mostrartudo(var aux2: apontador);
begin
  if aux2 <> nil
  then begin
          mostrartudo(aux2^.esquerda);
          mostrartudo(aux2^.direita);
          texto:= texto + '  '+inttostr(aux2^.num);
       end;
end;
```

O código do botão *Consultar Elementos à Direita da Raiz da Árvore* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
begin
texto:='';
if raiz = nil
  then showmessage('A árvore está vazia')
  else begin
       aux2 := raiz;
       mostrardireita(aux2);
       showmessage('Toda a árvore à direita = '+texto);
      end;
end;

Procedure mostrardireita(var aux2: apontador);
begin
  if aux2 <> nil
  then begin
       mostrardireita(aux2^.direita);
       if (aux2 <> raiz)
       then mostrardireita(aux2^.esquerda);
       texto:= texto + '  '+inttostr(aux2^.num);
      end;
end;
```

O código do botão *Consultar Elementos à Esquerda da Raiz da Árvore* é apresentado a seguir.

```
procedure TForm1.Button8Click(Sender: TObject);
begin
texto:='';
if raiz = nil
  then showmessage('A árvore está vazia')
  else begin
        aux2 := raiz;
        mostraresquerda(aux2);
        showmessage('Toda a árvore à esquerda = '+texto);
       end;
end;

Procedure mostraresquerda(var aux2: apontador);
begin
  if aux2 <> nil
  then begin
        mostraresquerda(aux2^.esquerda);
        if (aux2 <> raiz)
          then mostraresquerda(aux2^.direita);
        texto:= texto + '  '+inttostr(aux2^.num);
       end;
end;
```

O código do botão *Esvaziar a Árvore* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
begin
 if (raiz = Nil)
  then showmessage('A árvore está vazia')
  else begin
       aux := raiz;
       while (raiz <> Nil) do
       begin
        if (aux^.direita = Nil) and (aux^.esquerda = Nil)
           then begin
                // a raiz não tem filhos e será excluída
                raiz := Nil;
                dispose (aux);
                end
        else begin
             // a raiz será excluída
             // e possui filhos apenas para a direita
                 if (aux^.direita <> Nil) and (aux^.esquerda = Nil)
                      then begin
                           raiz := aux^.direita;
                           dispose(aux);
                           aux := raiz;
                           end
                 else if (aux^.esquerda <> Nil)
                       and (aux^.direita = Nil)
                       then begin
                            // a raiz será excluída
```

```
                        // e possui filhos apenas para a esquerda
                        raiz := aux^.esquerda;
                        dispose(aux);
                        aux := raiz;
                        end
                      else if (aux^.esquerda <> Nil) and
                           (aux^.direita <> Nil)
                          then begin
                               // a raiz será excluída
                               // e possui filhos para a direita
                               // e para a esquerda
                               raiz := aux^.direita;
                               aux1 := aux^.esquerda;
                               // recolocando o pedaço da árvore
                               dispose(aux);
                               aux := raiz;
                               while (aux <> Nil) do
                               begin
                               if (aux^.num < aux1^.num)
                               then begin
                                    if (aux^.direita = Nil)
                                    then begin
                                           aux^.direita := aux1;
                                           aux := Nil;
                                           end
                                    else aux := aux^.direita;
                                    end
                                else if (aux^.num > aux1^.num)
                                  then begin
                                       if (aux^.esquerda = Nil)
                                       then begin
                                            aux^.esquerda := aux1;
                                            aux := Nil;
                                            end
                                       else aux := aux^.esquerda;
                                       end;
                                end;
                               aux := raiz;
                              end;
           end;
      end;
      showmessage('Árvore esvaziada com sucesso');
      qtde1:=0;
      edit1.text:=inttostr(qtde1);
      qtde2:=0;
      edit2.text:=inttostr(qtde2);
      qtde3:=0;
     edit3.text:=inttostr(qtde3);
 end;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// fecha o formulário
close;
end;
```

Aplicação de fluxo de caixa usando árvore binária

A Figura 6.2 mostra a tela da aplicação de um controle de fluxo de caixa utilizando uma estrutura de dados do tipo árvore binária com implementação dinâmica. Nesta aplicação são cadastradas as operações, excluídas as operações já cadastradas e realizadas algumas consultas implementadas como: consulta de todas as operações a partir de uma data e consulta de saldo entre duas datas. A opção Fechar esvazia a árvore de operações. Após a figura, têm-se os seus respectivos códigos comentados.

Figura 6.2 Tela da aplicação de controle de fluxo de caixa utilizando árvore binária com implementação dinâmica



A seguir é apresentada a declaração das variáveis globais.

```
// definição do registro que será armazenado em cada nó da árvore;
// a variável DATA armazenará a data da operação;
// a variável TIPO armazenará o tipo da operação;
// ou seja, 1 para operação de crédito
// e 2 para operação de débito;
// a variável VALOR armazenará o valor da operação;
// a variável DIREITA é um ponteiro para um endereço
```

```
// de memória do tipo REGISTRO;
// a variável ESQUERDA é um ponteiro para um endereço
// de memória do tipo REGISTRO;

// definição de um novo tipo que será um apontador para
// a estrutura da árvore;
// este apontador será utilizado na função recursiva que
// percorre a árvore

type apontador = ^registro;
     registro = record
        data: tdate;
        tipo: integer;
        valor: real;
        direita, esquerda: apontador;
     end;

// as variáveis ARVORE, AUX, AUX1, AUX2, ANTERIOR e RAIZ são ponteiros
// para endereços de memória do tipo REGISTRO
// definido anteriormente

var Form1: TForm1;
    arvore, aux, aux1, aux2, anterior, raiz : apontador;
    dt,dt1,dt2:tdate;
    achou:integer;
    saldo: real;
```

No evento *OnActivate* do formulário tem-se o seguinte código.

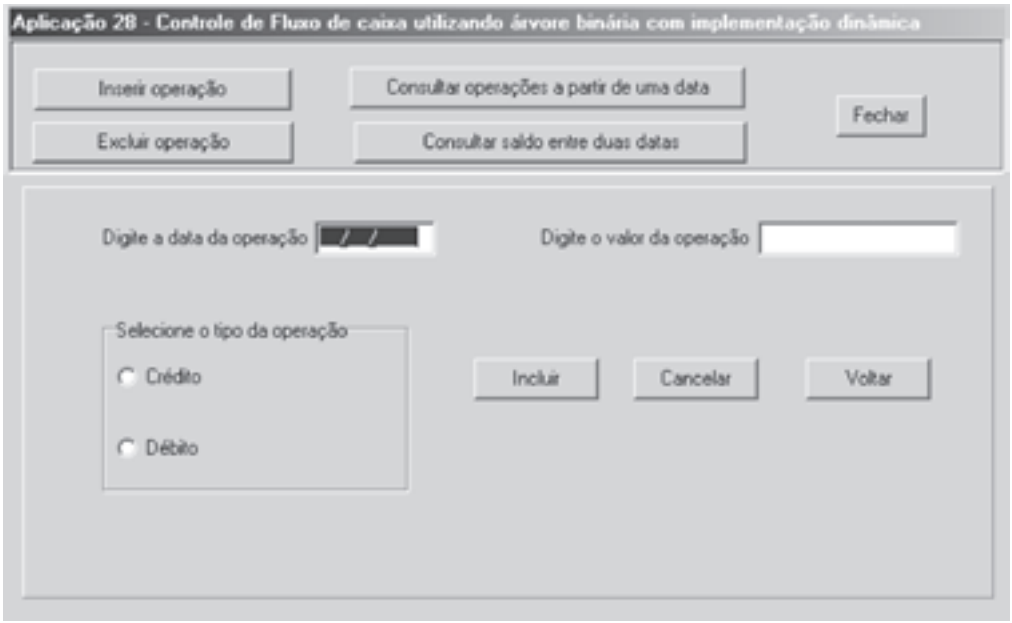
```
procedure TForm1.FormActivate(Sender: TObject);
begin
// o ponteiro RAIZ aponta para vazio, pois a árvore está vazia
raiz:=nil;
end;
```

O código do botão *Inserir Operação* é apresentado a seguir.

```
procedure TForm1.Button1Click(Sender: TObject);
begin
// desabilita a painel com os botões da aplicação
panel1.enabled:=false;
// mostra o painel para a inclusão de uma operação
panel3.visible:=true;
// limpa os componentes do painel de inclusão de novas operações
maskedit1.clear;
maskedit1.setfocus;
radiogroup1.itemindex:=-1;
edit1.clear;
end;
```

Quando o botão *Inserir Operação* é clicado, a tela da Figura 6.3 é apresentada.

Figura 6.3 Tela de inserção de novas operações na árvore



O código do botão *Incluir* é apresentado a seguir.

```
procedure TForm1.Button6Click(Sender: TObject);
var achou: integer;
begin
if maskedit1.text = '  /  /    '
then begin
     showmessage('É necessário digitar a data da operação');
     maskedit1.setfocus;
    end
else if radiogroup1.itemindex = -1
     then showmessage('É necessário selecionar o tipo da operação')
     else if edit1.text = ''
          then begin
               showmessage('É necessário digitar o valor da
                           operação');
               edit1.setfocus;
              end
          else begin
               // verificando se já existe uma operação
               // cadastrada nesta data;
               // a variável ACHOU marca se a operação
               // da referida data foi localizada
               achou := 0;
               if raiz <> nil
               then begin
                    // a busca pela operação na referida data
                    // começa na raiz
                    aux := raiz;
                    while (achou = 0) and (aux <> nil) do
```

```
                    begin
                    if aux^.data = strtodate(maskedit1.text)
                     then achou := 1
                     else if aux^.data > strtodate(maskedit1.text)
                     then  begin
                           // a operação está à esquerda da árvore
                           aux := aux^.esquerda;
                           end
                     else  begin
                           // a operação está à direita da árvore
                           aux := aux^.direita;
                           end;
                     end;
                   end;
                if achou = 1
                 then begin
                      showmessage('Já existe operação cadastrada
                                  nesta data');
                      maskedit1.setfocus;
                    end
               else begin
                     new(arvore);
                    arvore^.data:= strtodate(maskedit1.text);
                    if radiogroup1.itemindex = 0
                    then arvore^.tipo:=1
                    else arvore^.tipo:=2;
                    arvore^.valor:=strtofloat(edit1.text);
                    arvore^.direita:=nil;
                    arvore^.esquerda:=nil;
                    if (raiz = nil)
                     then raiz := arvore
                     else begin
                           aux := raiz;
                           achou := 0;
                           while (achou = 0) do
                           begin
                      if arvore^.data <= aux^.data
                      then begin
                            // a operação será inserida
                            // à esquerda da árvore
                            if aux^.esquerda = nil
                            then begin
                                  aux^.esquerda:= arvore;
                                  achou := 1;
                                 end
                            else aux := aux^.esquerda;
                            end
                       else if arvore^.data > aux^.data
                                 then begin
                                       // a operação será inserida
                                       // à direita da árvore
                                       if aux^.direita = nil
                                      then begin
                                           aux^.direita:= arvore;
```

```
                                        achou := 1;
                                        end
                                    else aux := aux^.direita;
                                    end
                              end;
                            end;
     showmessage('Operação inserida na árvore com sucesso');
     maskedit1.clear;
     maskedit1.setfocus;
     radiogroup1.itemindex:=-1;
     edit1.clear;
    end;
  end;
end;
```

O código do botão *Cancelar* é apresentado a seguir.

```
procedure TForm1.Button7Click(Sender: TObject);
begin
maskedit1.clear;
maskedit1.setfocus;
radiogroup1.itemindex:=-1;
edit1.clear;
end;
```

O código do botão *Voltar* é apresentado a seguir.

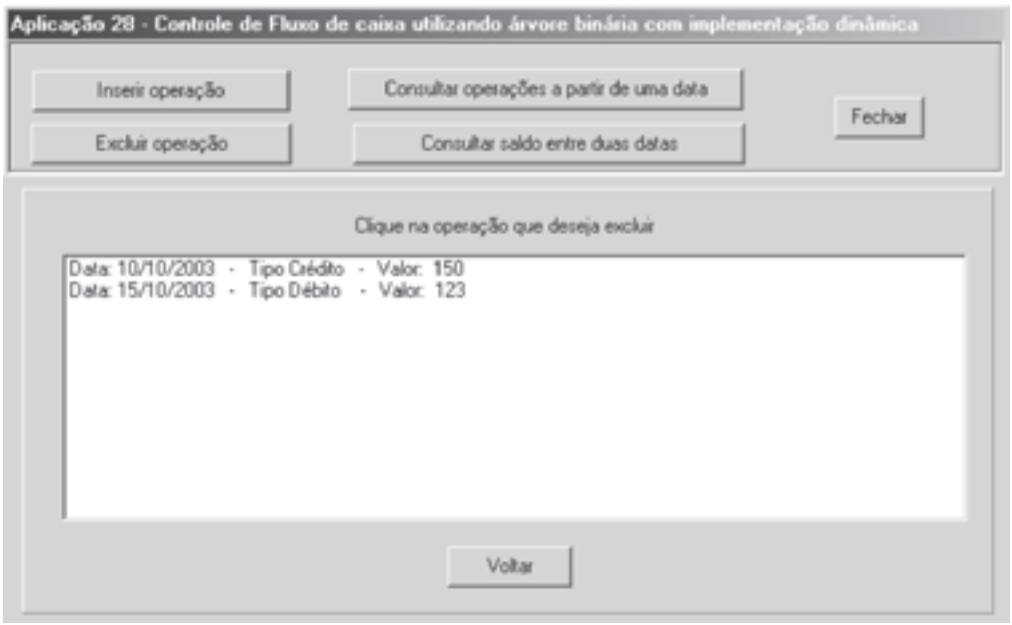
```
procedure TForm1.Button8Click(Sender: TObject);
begin
panel1.enabled:=true;
panel3.visible:=false;
end;
```

O código do botão *Excluir Operação* é apresentado a seguir.

```
procedure TForm1.Button2Click(Sender: TObject);
begin
panel1.enabled:=false;
panel4.visible:=true;
listbox1.clear;
if raiz = nil
  then showmessage('A árvore está vazia')
  else begin
       aux2 := raiz;
       mostrartudo(aux2);
      end;
end;
```

Quando o botão *Excluir Operação* é clicado, a tela da Figura 6.4 é apresentada.

Figura 6.4 Tela para a seleção da operação a ser excluída da árvore



O código quando o usuário clica sobre a operação que deseja excluir é apresentado a seguir.

```
procedure TForm1.ListBox1Click(Sender: TObject);
var achou:integer;
    dt:tdate;
    texto:string;
begin
// a busca pela operação a ser excluída
// começa na raiz
aux := raiz;
// o ponteiro AUX terá o endereço da operação a ser excluída;
// a variável ACHOU marca quando o endereço foi encontrado
achou:=0;
// Pegando a data da operação a ser excluída
texto:= copy(listbox1.items.strings[listbox1.itemindex],7,10);
dt:=strtodate(texto);
while (achou = 0) do
 begin
 if aux^.data = dt
 then achou := 1
 else if aux^.data > dt
     then  begin
           // a operação está à esquerda da árvore
           aux := aux^.esquerda;
           end
     else  begin
          // a operação está à direita da árvore
          aux := aux^.direita;
```

```
end;
if aux <> raiz
 then begin
      // a operação foi encontrada e será excluída;
      // como não é a raiz, é necessário encontrar o anterior
      // para acertar os ponteiros
      anterior := raiz;
      while (anterior^.direita <> aux)
      and (anterior^.esquerda <> aux) do
       begin
       if anterior^.data > dt
        then begin
             // a operação está à esquerda da árvore
             anterior := anterior^.esquerda;
            end
        else begin
             // a operação está à direita da árvore
             anterior := anterior^.direita;
            end;
        end;
        if (aux^.direita = nil) and (aux^.esquerda = nil)
          then begin
               // uma operação folha será excluída;
               // acertando os ponteiros do nó anterior
               if anterior^.direita = aux
               then anterior^.direita := nil
               else anterior^.esquerda := nil;
               // apagando a operação da árvore
               dispose(aux);
               end
          else begin
              // uma operação não folha será excluída;
               // uma operação que possui filhos
               // apenas para a direita
               if (aux^.direita <> nil)
               and (aux^.esquerda = nil)
               then begin
                    if anterior^.esquerda = aux
                    then anterior^.esquerda := aux^.direita
                    else anterior^.direita := aux^.direita;
                    // apagando a operação da árvore
                    dispose(aux);
                    end
               // uma operação não folha será excluída;
               // uma operação que possui filhos
               // apenas para a esquerda
               else if (aux^.esquerda <> nil)
                    and (aux^.direita = nil)
                        then begin
                            if anterior^.esquerda = aux
                            then anterior^.esquerda := aux^.esquerda
                            else anterior^.direita := aux^.esquerda;
                            // apagando a operação da árvore
```

```
                          dispose(aux);
                            end
                          // uma operação não folha será excluída;
                          // uma operação que possui filhos
                          // para a esquerda e para a direita
                          else if (aux^.esquerda <> nil)
                               and (aux^.direita <> nil)
                               then begin
                                    if anterior^.direita = aux
                                    then begin
                                         anterior^.direita :=
                                             aux^.direita;
                                         aux1 := aux^.esquerda;
                                         end
                                    else begin
                                         anterior^.esquerda := aux^.esquerda;
                                         aux1 := aux^.direita;
                                         end;
                                   // apagando a operação da árvore
                                   dispose(aux);
                                   // recolocando o pedaço
                                   // da árvore
                                   aux := anterior;
                                   while (aux <> nil) do
                                   begin
                                   if aux^.data < aux1^.data
                                   then begin
                                        if aux^.direita = nil
                                         then begin
                                               aux^.direita := aux1;
                                               aux := nil;
                                               end
                                           else aux := aux^.direita;
                                        end
                                   else if aux^.data > aux1^.data
                                        then begin
                                             if aux^.esquerda =
                                                          nil
                                              then begin
                                                  aux^.esquerda := aux1;
                                                  aux := nil;
                                                  end
                                             else aux := aux^.esquerda;
                                             end;
                                   end;
                                 end;
              end;
    end
else begin
     // a operação a ser excluída é a raiz
     if (aux^.direita = nil)
     and (aux^.esquerda = nil)
         then begin
```

```
            // a raiz não tem filhos e será excluída
            raiz := nil;
            // apagando a operação
            dispose(aux);
           end
       else begin
           // a raiz será excluída
           // e possui filhos apenas para a direita
            if (aux^.direita <> nil)
            and (aux^.esquerda = nil)
                then begin
                     raiz := aux^.direita;
                     // apagando a operação
                     dispose(aux);
                     end
                  else if (aux^.esquerda <> nil)
                       and (aux^.direita = nil)
                       then begin
                            // a raiz será excluída
                            // e possui filhos apenas
                            // para a esquerda
                            raiz := aux^.esquerda;
                            // apagando a operação
                            dispose(aux);
                           end
                       else if (aux^.esquerda <> nil)
                            and (aux^.direita <> nil)
                            then begin
                                 // a raiz será excluída
                                 // e possui filhos
                                 // para a direita
                                 // e para a esquerda
                                 raiz := aux^.direita;
                                 aux1 := aux^.esquerda;
                                 // apagando a operação
                                 dispose(aux);
                                 // recolocando o pedaço
                                 // da árvore
                                 aux := raiz;
                                 while (aux <> nil) do
                                 begin
                                 if aux^.data < aux1^.data
                                 then begin
                                      if aux^.direita = nil
                                      then begin
                                           aux^.direita := aux1;
                                           aux := nil;
                                          end
                                     else aux := aux^.direita;
                                     end
                                  else if aux^.data>aux1^.data
                                       then begin
                                            if aux^.esquerda = nil
                                            then begin
```

```
                                                  aux^.esquerda := aux1;
                                                   aux := nil;
                                                       end
                                                     else aux := aux^.esquerda;
                                                  end;
                                       end;
                                       end;
                  end;
             end;
     showmessage('Operação excluída com sucesso');
     listbox1.clear;
     if raiz = nil
     then showmessage('A árvore está vazia')
     else begin
          aux2 := raiz;
          mostrartudo(aux2);
          end;
end;
```

Quando o usuário clica sobre a operação, ocorre uma chamada a *procedure mostrartudo,* que tem seu código descrito a seguir.

```
Procedure mostrartudo(var aux2: apontador);
var t, texto:string;
begin
  if aux2 <> nil
  then begin
       mostrartudo(aux2^.direita);
       mostrartudo(aux2^.esquerda);
        if aux2^.tipo = 1
        then t:='   -   Tipo Crédito   -   '
        else t:='   -   Tipo Débito    -   ';
        texto:='';
        texto:='Data: '+datetostr(aux2^.data)+t+'Valor:  '
                  +floattostr(aux2^.valor);
           form1.listbox1.items.add(texto);
       end;
end;
```

O código do botão *Voltar* é apresentado a seguir.

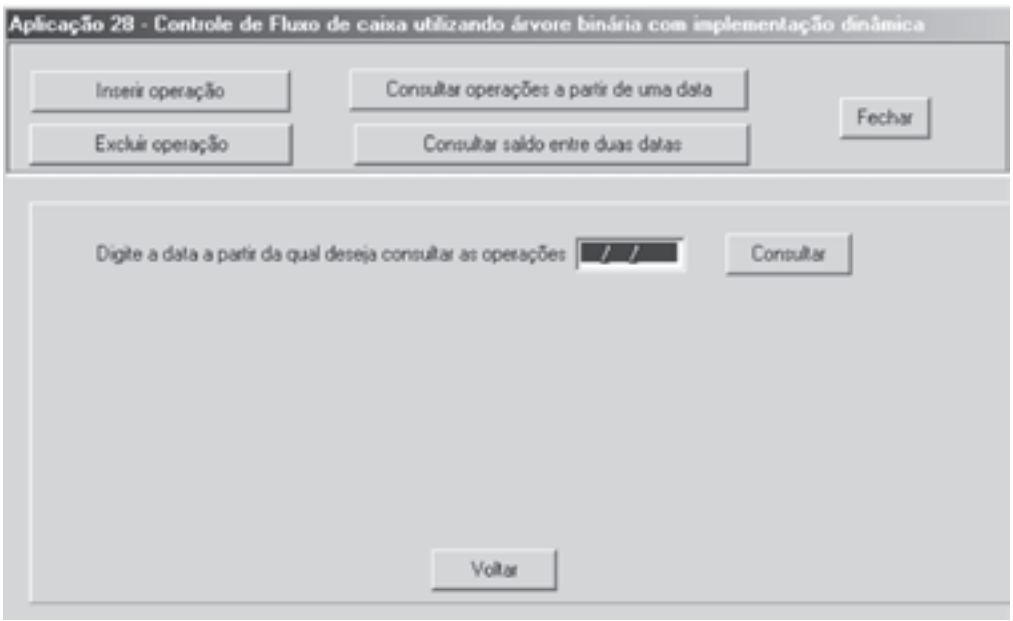
```
procedure TForm1.Button9Click(Sender: TObject);
begin
panel4.visible:=false;
panel1.enabled:=true;
end;
```

O código do botão *Consultar Operações a Partir de uma Data* é apresentado a seguir.

```
procedure TForm1.Button5Click(Sender: TObject);
begin
panel1.enabled:=false;
panel5.visible:=true;
maskedit2.clear;
maskedit2.setfocus;
listbox2.visible:=false;
end;
```

Quando o botão *Consultar Operações a Partir de uma Data* é clicado, a tela a Figura 6.5 é apresentada.

Figura 6.5 Tela para a digitação da data a partir da qual as operações serão consultadas



O código do botão *Consultar* é apresentado a seguir.

```
procedure TForm1.Button10Click(Sender: TObject);
begin
if maskedit2.text = '  /  /    '
then begin
      showmessage('É necessário digitar a data da consulta');
      maskedit2.setfocus;
     end
else if raiz = nil
then showmessage('Nenhuma operação foi cadastrada')
else begin
     dt:=strtodate(maskedit2.text);
     // a variável ACHOU marca se alguma operação foi
```

```
     // cadastrada a partir da data digitada
     achou:=0;
     listbox2.clear;
     aux2 := raiz;
     mostrartudomaior(aux2);
     if achou = 0
     then begin
          listbox2.visible:=false;
          showmessage('Nenhuma operação cadastrada a partir da data
                      digitada');
          maskedit2.setfocus;
          end
     else listbox2.visible:=true;
     end;
end;
```

Este código faz uma chamada a *procedure mostrartudomaior*, que tem seu código descrito a seguir.

```
procedure mostrartudomaior(var aux2: apontador);
var t, texto:string;
begin
  if aux2 <> nil
  then begin
       mostrartudomaior(aux2^.direita);
       mostrartudomaior(aux2^.esquerda);
       if (aux2^.data >= dt)
       then begin
            achou:=1;
            if aux2^.tipo = 1
            then t:='   -   Tipo Crédito   -   '
            else t:='   -   Tipo Débito    -   ';
            texto:='';
            texto:='Data: '+datetostr(aux2^.data)+t+'Valor:  '
                      +floattostr(aux2^.valor);
           form1.listbox2.items.add(texto);
           end;

      end;
end;
```

O código do botão *Voltar* é apresentado a seguir.

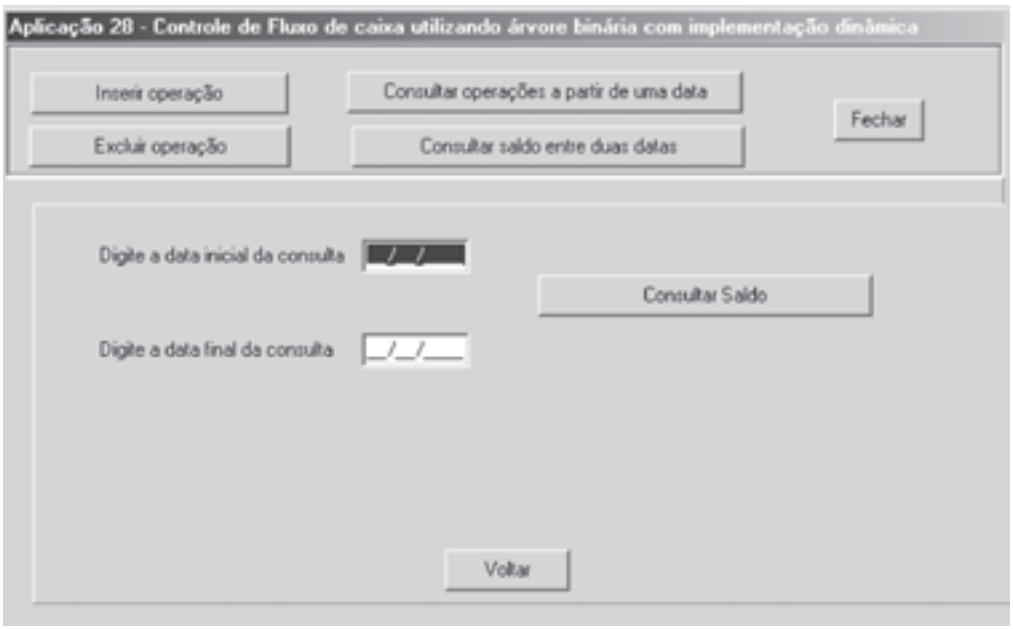
```
procedure TForm1.Button11Click(Sender: TObject);
begin
panel1.enabled:=true;
panel5.visible:=false;
end;
```

O código do botão *Consultar Saldo entre Duas Datas* é apresentado a seguir.

```
procedure TForm1.Button3Click(Sender: TObject);
begin
panel1.enabled:=false;
panel6.visible:=true;
maskedit3.clear;
maskedit3.setfocus;
maskedit4.clear;
end;
```

Quando o botão *Consultar Saldo entre Duas Datas* é clicado, a tela da Figura 6.6 é apresentada.

Figura 6.6 Tela para a digitação das datas entre as quais o saldo será calculado



O código do botão *Consultar Saldo* é apresentado a seguir.

```
procedure TForm1.Button12Click(Sender: TObject);
begin
if maskedit3.text = '  /  /    '
then begin
     showmessage('É necessário digitar a data inicial da consulta');
     maskedit3.setfocus;
    end
else if maskedit4.text = '  /  /    '
then begin
     showmessage('É necessário digitar a data final da consulta');
     maskedit4.setfocus;
    end
else if strtodate(maskedit3.text) > strtodate(maskedit4.text)
then begin
```

```
      showmessage('É necessário que a data inicial seja menor ou igual
                  a data final');
      maskedit3.setfocus;
     end
else if raiz = nil
      then showmessage('Nenhuma operação foi cadastrada')
      else begin
           dt1:=strtodate(maskedit3.text);
           dt2:=strtodate(maskedit4.text);
           saldo:=0;
           aux2 := raiz;
           consultartudoentredatas(aux2);
           showmessage('Saldo = '+floattostr(saldo));
           maskedit3.clear;
           maskedit3.setfocus;
           maskedit4.clear;
          end;
end;
```

O código anterior faz uma chamada a *procedure consultartudoentredatas* que tem o código descrito a seguir.

```
procedure consultartudoentredatas(var aux2: apontador);
begin
  if aux2 <> nil
  then begin
       consultartudoentredatas(aux2^.direita);
       consultartudoentredatas(aux2^.esquerda);
       if (aux2^.data >= dt1) and (aux2^.data <= dt2)
       then begin
            if aux2^.tipo = 1
            then saldo:=saldo + aux2^.valor
            else saldo:=saldo - aux2^.valor
           end;
      end;
end;
```

O código do botão *Voltar* é apresentado a seguir.

```
procedure TForm1.Button13Click(Sender: TObject);
begin
panel1.enabled:=true;
panel6.visible:=false;
end;
```

O código do botão *Fechar* é apresentado a seguir.

```
procedure TForm1.Button4Click(Sender: TObject);
begin
// é necessário esvaziar a árvore aqui
 aux := raiz;
  while (raiz <> Nil) do
    begin
       if (aux^.direita = Nil) and (aux^.esquerda = Nil)
           then begin
```

```
                // a raiz não tem filhos e será excluída
                raiz := Nil;
                dispose (aux);
              end
        else begin
              // a raiz será excluída
              // e possui filhos apenas para a direita
              if (aux^.direita <> Nil) and (aux^.esquerda = Nil)
              then begin
                    raiz := aux^.direita;
                    dispose(aux);
                    aux := raiz;
                    end
              else if (aux^.esquerda <> Nil)
                    and (aux^.direita = Nil)
                    then begin
                          // a raiz será excluída
                          // e possui filhos apenas para a esquerda
                          raiz := aux^.esquerda;
                          dispose(aux);
                          aux := raiz;
                         end
                    else if (aux^.esquerda <> Nil)
                          and (aux^.direita <> Nil)
                            then begin
                                 // a raiz será excluída
                                 // e possui filhos para a direita
                                 // e para a esquerda
                                 raiz := aux^.direita;
                                 aux1 := aux^.esquerda;
                                 // recolocando o pedaço da árvore
                                 dispose(aux);
                                 aux := raiz;
                                 while (aux <> Nil) do
                                 begin
                                  if (aux^.data < aux1^.data)
                                  then begin
                                          if (aux^.direita = Nil)
                                          then begin
                                               aux^.direita := aux1;
                                               aux := Nil;
                                               end
                                          else aux := aux^.direita;
                                        end
                                   else if (aux^.data > aux1^.data)
                                         then begin
                                              if (aux^.esquerda = Nil)
                                         then begin
                                              aux^.esquerda:= aux1;
                                              aux := Nil;
                                             end
                                         else aux := aux^.esquerda;
```

```
                                        end;
                                    end;
                                aux := raiz;
                            end;
            end;
      end;
      showmessage('Todas as operações cadastradas foram excluídas');
close;
end;
```